Day 02
Physical design and Verilog foundations

VLSI Design Flow RTL to GDS
Lessons 07 to 12

Layout, verification, manufacturing, Tcl and the first two Verilog lessons.

Contents and bookmarks link to sections; figures and page footers return to contents.

Includes **12 complete handwritten pages**. Find an original page or test your understanding.

Course: Prof. Sneh Saurabh, IIIT Delhi, NPTEL. Unix and Tcl tutorials: Jasmine Kaur. Worked examples state their analytical assumptions.

Contents

Revision route 3

Lesson 07 Physical design and closure 3

From connectivity to geometry 3
Floorplanning power and placement 5
Clock tree synthesis 8
Global routing detailed routing and closure 11
Definitions and mechanisms in Lesson 07 13
Application and validation in Lesson 07 14
Complete original notebook pages for Lesson 07 15

Lesson 08 Verification and manufacturing test 19

Verification simulation and formal methods 19
Timing and physical verification 21
Defects faults and test patterns 23
Yield fault coverage and escapes 26
Automatic test equipment and design for test 29
Definitions and mechanisms in Lesson 08 30
Application and validation in Lesson 08 31
Complete original notebook pages for Lesson 08 32

Lesson 09 From layout to a packaged chip 36

Mask data preparation and mask writing 36
Resolution enhancement 38
Wafer fabrication packaging and screening 41
Definitions and mechanisms in Lesson 09 42
Application and validation in Lesson 09 43

Complete original notebook pages for Lesson 09 . . . 44

Lesson 10 Tcl commands and automation . . . 46

Commands variables and substitution . . . 46
Control flow and procedures . . . 48
File channels and external commands . . . 49
Definitions and mechanisms in Lesson 10 . . . 50
Application and validation in Lesson 10 . . . 51

Lesson 11 Verilog values types and hardware modeling . . . 51

What a hardware description language must represent . . . 51
Lexical rules and four-state values . . . 52
Sized literals padding truncation and signed values . . . 54
Nets variables vectors arrays and strings . . . 57
Recall checks . . . 59
Definitions and mechanisms in Lesson 11 . . . 59
Application and validation in Lesson 11 . . . 60
Complete original notebook pages for Lesson 11 . . . 61

Lesson 12 Verilog structure events and assignments . . . 62

Modules ports hierarchy and parameters . . . 62
Operators and bit-level examples . . . 64
Processes event controls and four-state edges . . . 67
Functions and tasks . . . 70
Continuous blocking and nonblocking assignment . . . 73
System tasks and the next study boundary . . . 76
Definitions and mechanisms in Lesson 12 . . . 76
Application and validation in Lesson 12 . . . 77
Complete original notebook pages for Lesson 12 . . . 79

Revision checks and explained answers . . . 80

Check 01: Utilization does not prove routability . . . 80
Check 02: Clock delay and skew are separate . . . 80
Check 03: Yield is not fault coverage . . . 80
Check 04: Follow Tcl substitution . . . 80
Check 05: Truncate a sized literal deliberately . . . 80
Check 06: Predict nonblocking updates . . . 81

Handwritten page finder . . . 81

Sources and references . . . 82

Primary references for the technical clarifications . . . 82

Revision route

Follow the physical-design flow from a connected netlist to placed and routed geometry, then distinguish verification evidence from manufacturing-test evidence. Read the Tcl examples by substitution stage. For Verilog, write widths and four-state values before predicting an operator or process. The worked checks below deliberately separate utilization from legality, yield from coverage, and active-region values from nonblocking updates.

Revision checks and explained answers close the day. These are authored practice examples; original handwritten page identities remain attached to their source figures.

Lesson 07 Physical design and closure

Week 2 · Lecture video

From connectivity to geometry

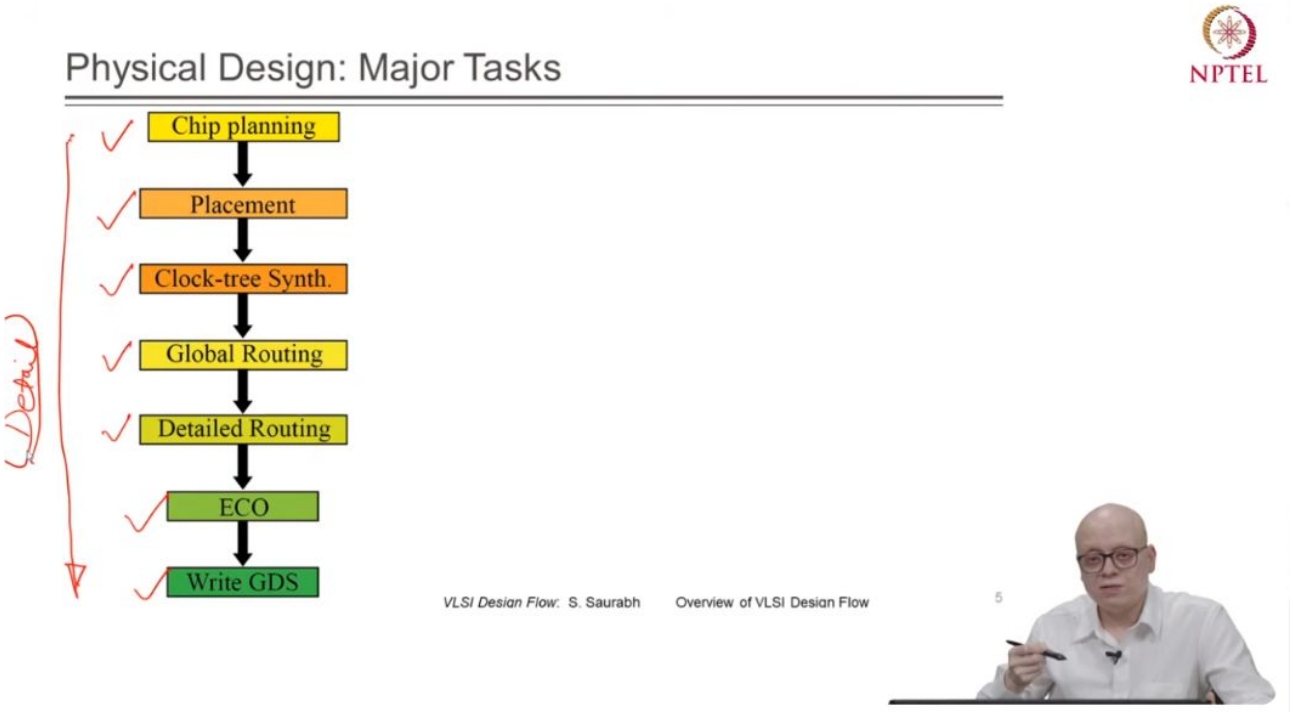


Figure 2.01. *Lecture: 10:56. Physical design adds placement, clock distribution, and routing*

Physical design converts a logical netlist into manufacturable layout geometry. The logical netlist says which pins connect; physical implementation chooses where instances sit and how metal and vias realize those connections. The tool also changes the implementation where needed, for example by inserting buffers or selecting different cell sizes, while preserving the required behavior.

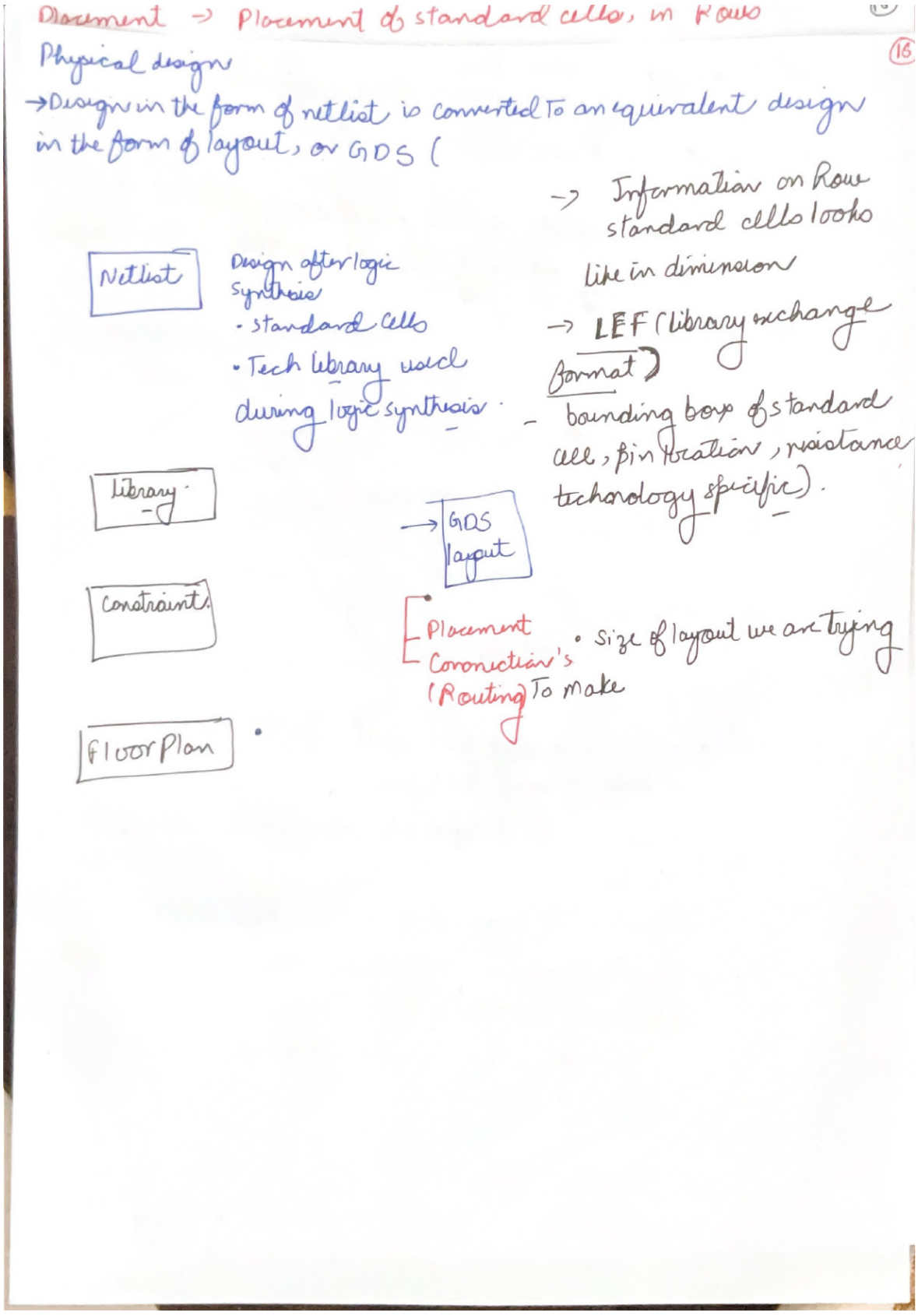


Figure 2.02. *Source: Part 1, PDF page 17.*

The input diagram needs several complementary views:

View or input	What it provides
Netlist	Instances and connectivity
Liberty	Cell function, timing, power, and related characterization
Cell/macro LEF	Abstract cell boundary, legal placement information, pin shapes, and obstructions
Technology LEF and process data	Routing layers, vias, relevant geometric rules, and technology information
Constraints	Clocks, external timing, and design intent
Floorplan	Die/core boundaries, regions, macro and I/O intent

LEF stands for **Library Exchange Format**. It is deliberately an abstract physical view: routing needs to know where a macro's pins and obstacles are without repeatedly processing all its internal transistor geometry. Detailed extraction uses appropriate process/parasitic models; a LEF file is not a substitute for all electrical or manufacturing information. GDSII carries detailed layout geometry, while implementation databases and formats such as DEF can represent placed/routed design state.

Floorplanning power and placement

Floorplanning establishes the large-scale organization of the chip: die and core size, major blocks, macro locations, I/O arrangement, and legal standard-cell regions. The **die** is the silicon boundary; the **core** is the principal implementation region inside it. **Placement** subsequently chooses legal locations for individual standard-cell instances.

Utilization is meaningful only with a stated denominator. A common early estimate divides standard-cell area by the available placement area. If cells occupy 0.60 mm^2 and the usable row area is 1.00 mm^2, utilization is 60%. The remaining space is not simply wasted: it provides flexibility for placement, buffers, clock cells, routing access, and later fixes. Excessively dense placement can cause congestion and timing detours.

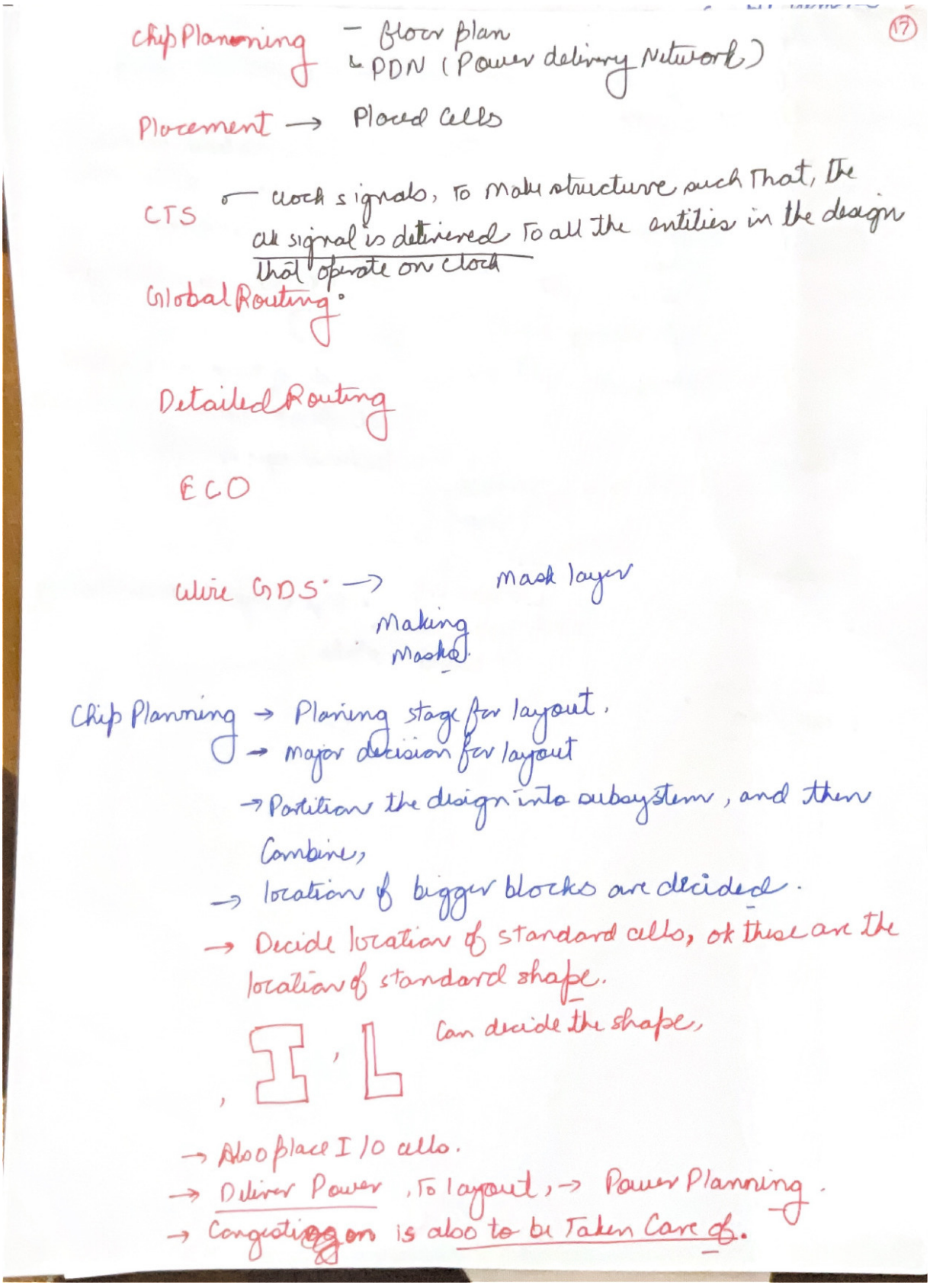


Figure 2.03. *Source: Part 1, PDF page 18.*

The note about “location of standard cells” at chip planning means **allocate regions and rows**, not finalize every cell coordinate. Those millions of individual locations belong to placement. The rectilinear L/I sketches describe flexibility available for some blocks. A hard macro with fixed

physical geometry cannot arbitrarily be reshaped; a soft block or hierarchical region can offer different aspect ratios or shapes depending on its implementation.

A **power delivery network (PDN)** distributes supply and ground through conductive structures to cells and macros. Wire resistance causes a voltage loss approximately $\Delta V = IR$ in a simple DC segment. For example, 20 mA through 0.5 Ω produces 10 mV of drop. Real networks have distributed current, transient effects, multiple paths, and reliability constraints. Lower local supply voltage can slow cells; adequate power wiring therefore supports timing as well as operation.

Congestion means routing demand is excessive relative to available routing resources in a region. A narrow channel between macros may attract many signals but offer few tracks. A globally short placement can still route poorly if it blocks pin access or concentrates too much traffic. Placement optimizes estimated wire length, timing, density, and routability before the exact wires exist; the quality of those estimates matters.

Worked example: placement area and routing capacity

With utilization defined as cell area divided by usable placement area, rearrange the definition:

$$A_{\text{placement}} = \frac{A_{\text{cells}}}{U}.$$

For 0.60 mm² of cells, a 60% target requires 1.00 mm² of usable placement area; a 75% target requires 0.80 mm². The second estimate is 20% smaller. It is an area calculation, not proof of routability. Macros, blockages, reserved regions, and the chosen utilization definition determine how this usable area relates to the total core. Later buffers and clock cells can also change the cell-area numerator.

To understand congestion, imagine one routing boundary with eight available track units and twelve units of estimated demand. The demand/capacity ratio is $12/8 = 1.5$, and overflow is four units. Average chip utilization cannot reveal this local shortage. Other layers or detours may help, but their availability, vias, pin access, and delay must also be considered. These simple units illustrate the concept; actual congestion reports depend on the router's resource model.

Moving cells apart can reduce demand concentration, while moving a macro can open a blocked channel. Strengthening the PDN may reserve additional routing space for supplies. These interactions explain why floorplanning, power planning, placement, and routing need iteration. As a concrete tool example, OpenROAD's global placer estimates congestion during routability-driven placement and increases the modeled area of cells in congested regions to encourage spreading.

Try it: if the cell area grows to 0.66 mm² while the usable placement area remains 1.00 mm², utilization becomes 66%. That percentage still does not establish whether a particular macro channel has enough tracks.

Clock tree synthesis

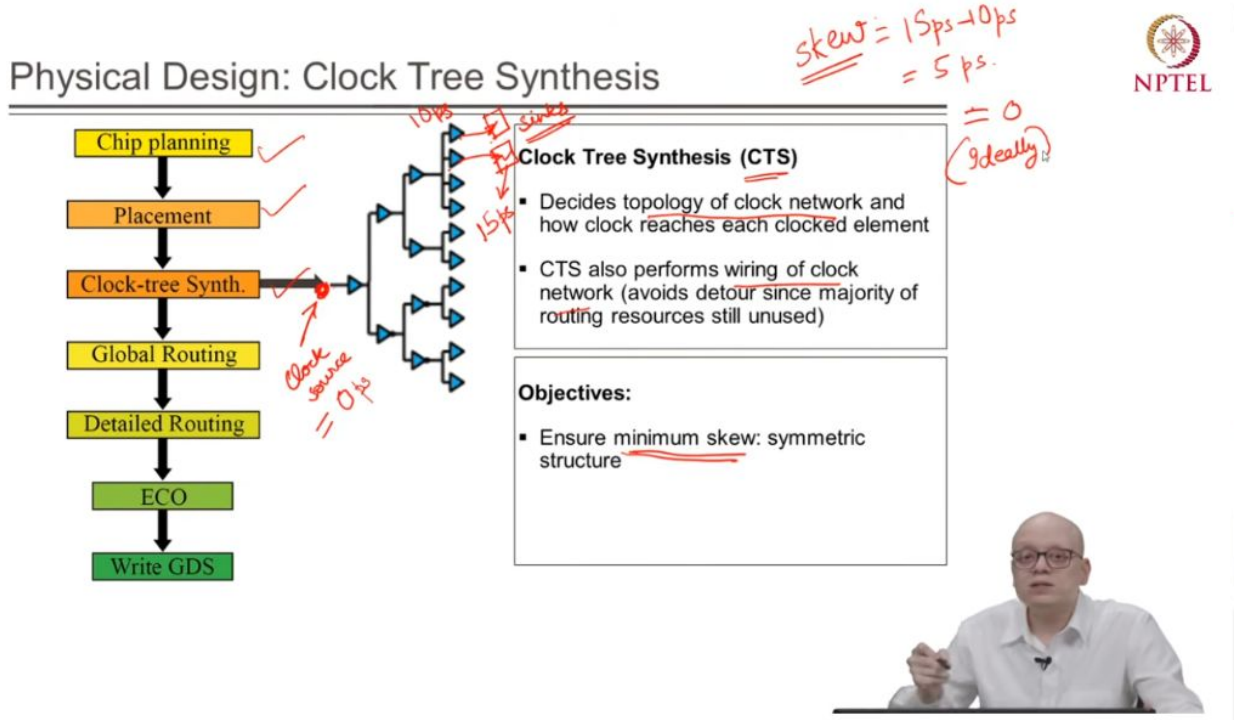


Figure 2.04. *Lecture: 32:49. Clock arrival times differ because the distribution network has delay*

Clock tree synthesis (CTS) builds a network that distributes clock events to sequential elements with controlled skew, latency, transition time, and load. A single ideal clock source cannot directly drive an arbitrarily large physical load with zero delay. Buffers and branches distribute that load.

Clock latency is the delay from the chosen clock reference to a sink. **Clock skew** is the difference in arrival times between specified sinks. If one sink receives an edge at 10 ps and another at 15 ps, their arrival difference is 5 ps. Equal latency at all sinks can give zero skew even when that common latency is large. Skew and latency are therefore different quantities.

Symmetric topology and balanced electrical loading can reduce skew, but equal drawn wire lengths alone do not guarantee equal delays. Cell delays, loading, parasitics, and variation matter. The introductory goal is small skew; practical timing optimization can also use controlled useful skew under explicit setup and hold analysis.

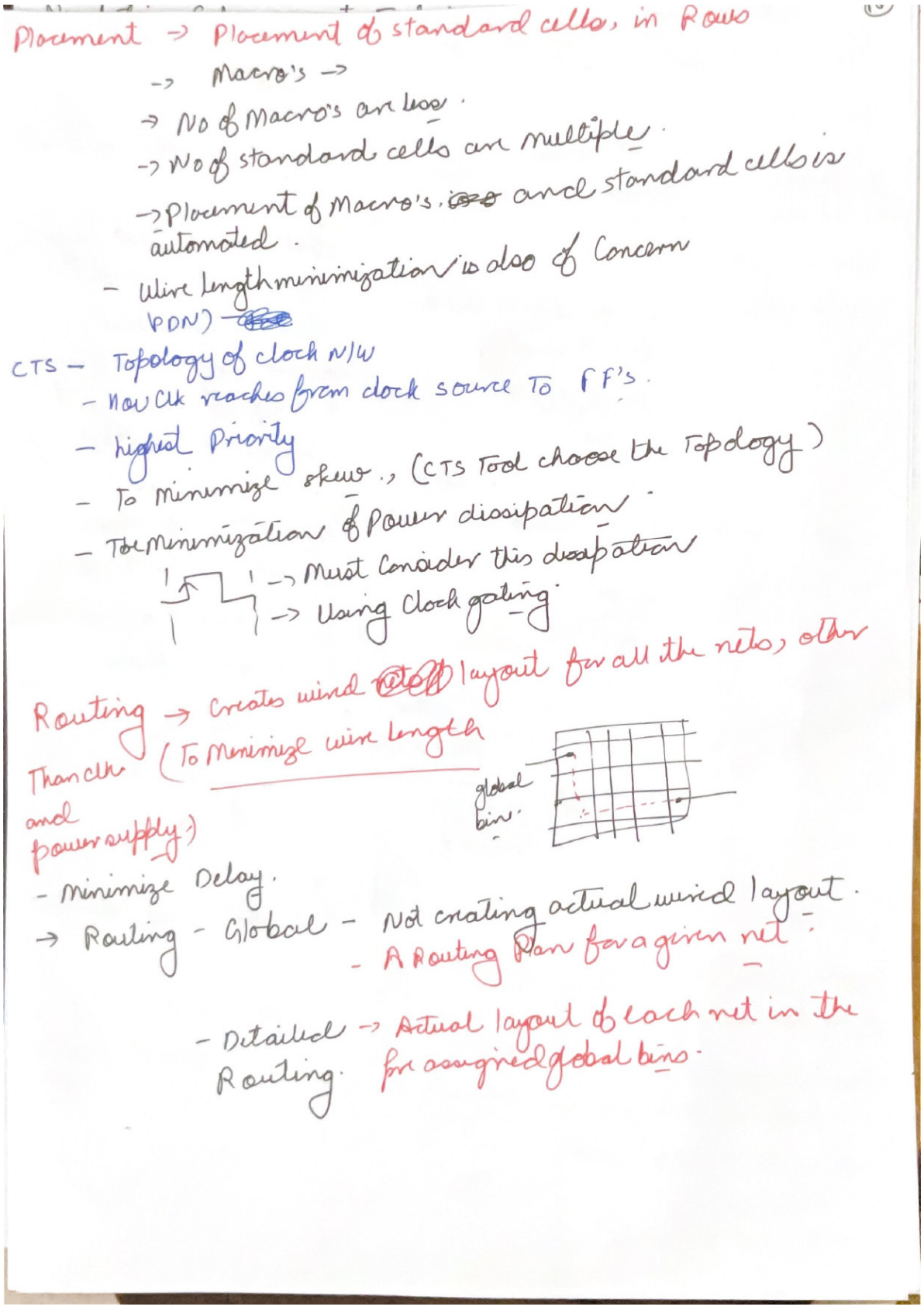


Figure 2.05. *Source: Part 1, PDF page 19.*

The high-priority clock note reflects its role in synchronous operation. Clock routing is planned before much ordinary signal routing in the lecture's flow, when resources are less constrained. The clock switches frequently and drives large capacitance, so it can consume a substantial fraction of dynamic power. The percentage depends on the design; it is not a universal constant.

Clock gating prevents unnecessary clock transitions from reaching inactive logic. The enable must be applied using a glitch-safe structure and verified behavior. A naive AND gate whose enable changes at the wrong time can create an unintended edge. Gating cells can be inserted at different points in an actual flow, so it should not be remembered as an operation that only ever occurs during CTS.

Trace the unintended edge in a naive clock gate

Assume a positive-edge-triggered destination and a clock high from 0 to 5 ns, low from 5 to 10 ns, then high again. The naive gate is $g = clk \wedge enable$. Let enable initially be zero and become one at 2 ns, while the clock is already high. The resulting gated clock rises at 2 ns even though no source-clock edge occurred then.

Time	Source clock	Enable	Naive AND output	Low-level latch state	Latch-gated output
Before 0 ns	0	0	0	0	0
0 ns, source rising edge	1	0	0	0	0
2 ns, enable rises	1	1	1	0	0
5 ns, source falls	0	1	0	1	0
10 ns, next source rise	1	1	1	1	1

The naive circuit creates a three-nanosecond high pulse from 2 to 5 ns instead of passing a complete five-nanosecond source pulse. A destination may respond to the unexpected rising edge or violate its pulse-width requirements. If enable instead falls during a passed high pulse, the same AND structure can shorten that pulse. Functional inactivity does not justify creating an arbitrary clock event.

In the ideal latch-based arrangement, the enable latch is transparent while the clock is low and holds its value while the clock is high. Its output therefore remains zero throughout the first high interval. It captures one during the following low interval and permits the complete pulse beginning at 10 ns. The SKY130 clock-gating cell model provides a concrete library example; actual cells also have characterized gating checks and implementation requirements.

The table assumes initialized latch state and ignores analog delay. An asynchronous enable still requires an appropriate synchronization and control design; a gating latch alone does not establish that requirement. Part 1 page 19's clock-gating reminder therefore concerns safe event generation as well as reducing switching activity.

Global routing detailed routing and closure

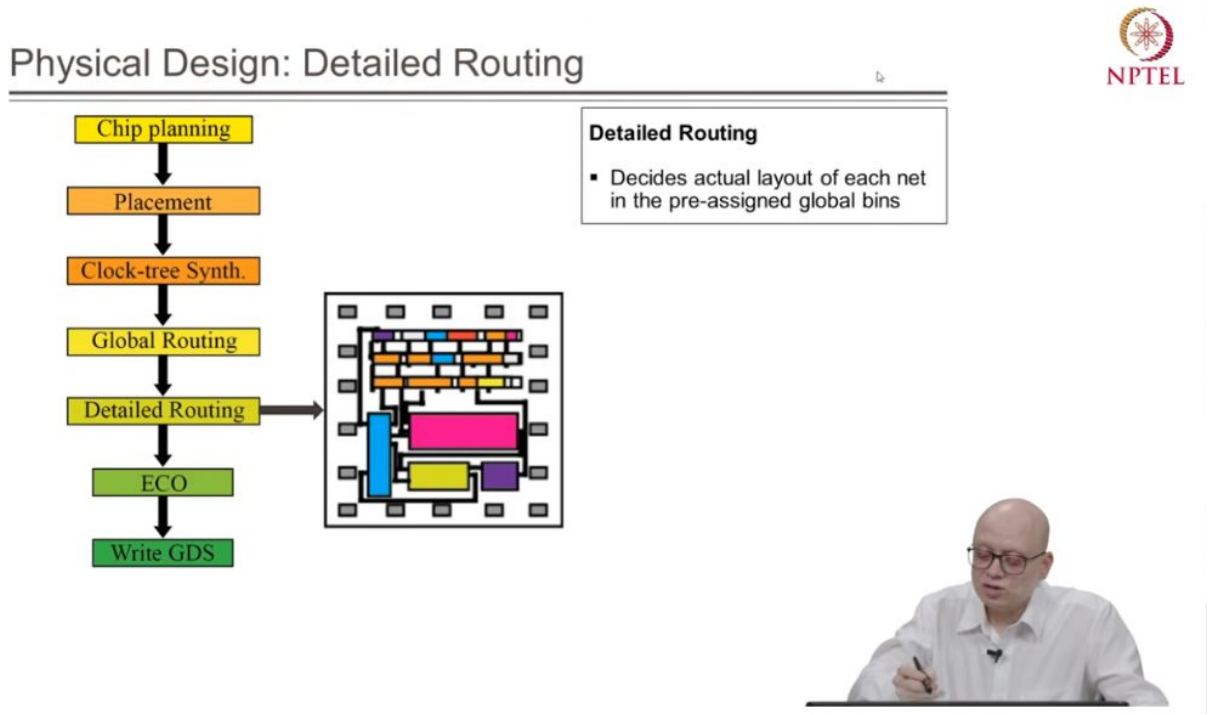


Figure 2.06. *Lecture: 41:13. Detailed routing chooses actual wires and vias within the planned regions*

Global routing plans approximate paths through routing regions and layers while accounting for capacity and congestion. **Detailed routing** assigns actual tracks, wire shapes, and vias to connect pins legally. A global route is a plan, not proof that a design-rule-clean detailed route exists. Pin access, spacing, enclosure, and competing wires can invalidate an apparently reasonable plan.

Routing seeks legal connectivity and suitable timing while managing wire length, via count, and congestion. More vias can add resistance and physical constraints; a long detour can worsen delay. After routing, extraction provides more realistic parasitic resistance and capacitance than early estimates, allowing timing and signal-integrity checks to be repeated.

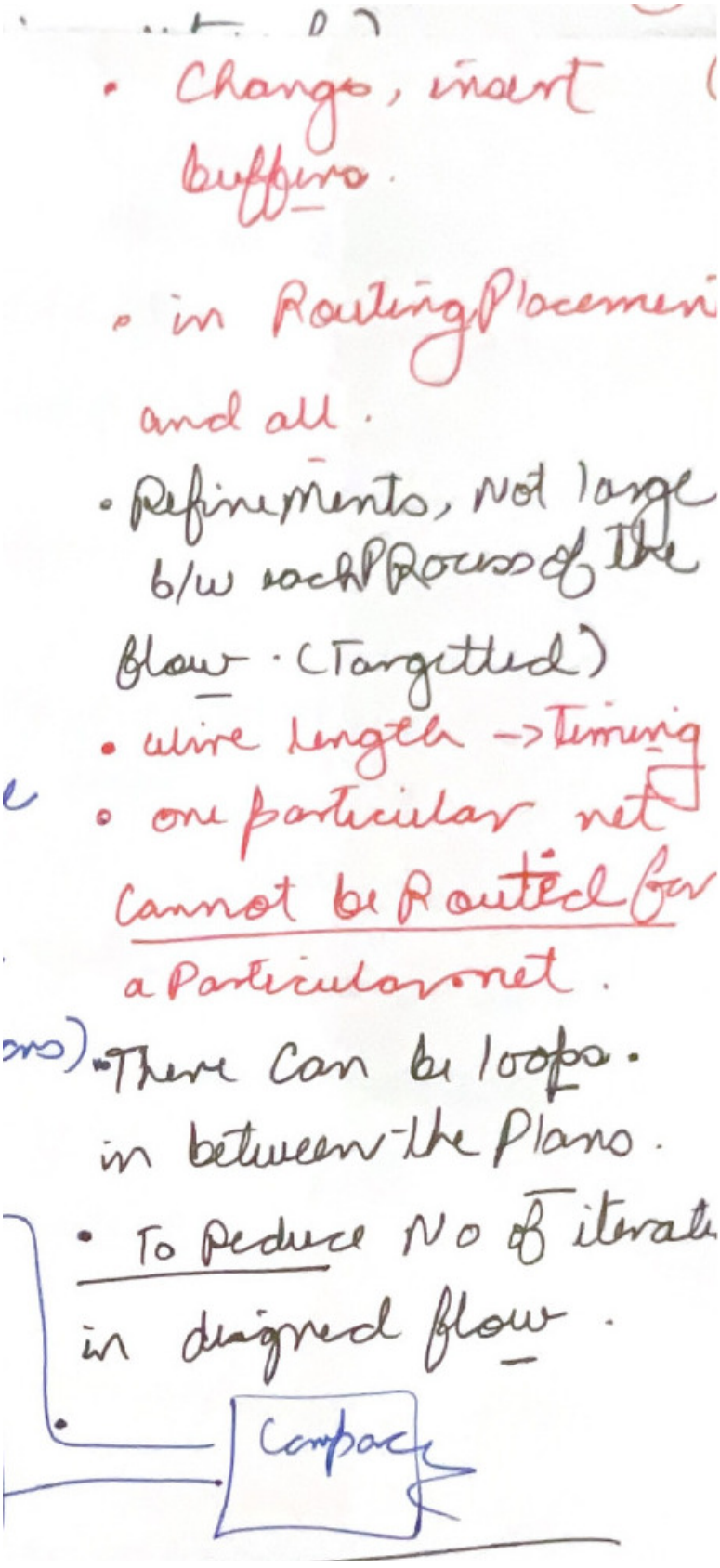


Figure 2.07. *Source: Part 1, PDF page 20, right-side ECO notes.*

An **engineering change order (ECO)** is a controlled implementation change, often used for late functional or timing fixes. Examples include resizing a cell, inserting a buffer, or changing selected logic and connections. Every such change needs the appropriate rechecks. A “small” edit in the file

can affect many paths, so its impact is judged electrically and logically, not by the number of edited lines.

Design closure means that the implementation satisfies the required set of checks and constraints for the intended operating scenarios. It can require iterations: routing may expose a congested placement, and a timing fix may worsen power or hold timing. The arrows back to earlier stages capture this feedback. **Tapeout** is release of the validated manufacturing design data to the foundry; it does not mean that fabricated parts have already passed test.

Recall checks: Can a netlist have correct connectivity but an unroutable placement? Can two clock sinks have equal latency but nonzero skew? Which checks must be revisited after inserting a buffer on a timing-critical net?

Definitions and mechanisms in Lesson 07

Physical implementation. Physical implementation converts instances and logical connectivity into positions, layers, wires and vias while satisfying the relevant electrical and geometric constraints. It also changes the implementation through operations such as buffering and resizing where the flow permits. A logical net names a connection; a physical route realizes it with finite resistance, capacitance and routing resources. The task is complete only to the extent supported by the required checks, not merely because the layout viewer displays connected shapes.

Floorplan and placement. A floorplan establishes the chip's large-scale organization: boundaries, macros, I/O intent, placement regions and infrastructure. Placement assigns locations to individual cells within the legal regions. The distinction is between organizing available space and assigning instances within it. A floorplan can allow several possible placements, and a poor macro arrangement can make every subsequent placement difficult. A fixed hard macro cannot be freely reshaped just because a floorplanning sketch uses a flexible outline.

Utilization and congestion. Utilization is an area ratio with an explicitly stated denominator, often standard-cell area divided by usable placement area. Congestion is a shortage of routing capacity relative to demand in a region or routing boundary. They are related through geometry but measure different things. Two layouts with the same utilization can have very different macro channels and pin access. Lowering average density may help routing, but it does not prove that every local bottleneck disappears.

Power delivery network. A PDN distributes power and ground to devices through connected conductive structures. Its nonzero impedance means the local supply seen by a cell can differ from the ideal source. A simple DC segment gives a voltage drop proportional to current and resistance; a real network also has distributed and transient behavior. The PDN uses routing space and must support electrical and reliability requirements. Power planning therefore influences placement, routability and timing rather than serving as a purely decorative layout layer.

Clock latency, skew and transition. Latency is the delay from a specified clock reference to a sink. Skew is an arrival-time difference between sinks. Transition time describes the edge shape at a point. A network can have equal sink latencies and therefore zero skew while all sinks receive the edge significantly after the source. CTS controls these related but distinct properties while managing load and power. Equal geometric length is only one influence on delay because buffering and electrical loading also matter.

Global routing, detailed routing and extraction. Global routing allocates approximate routes and layer resources; detailed routing chooses legal wire shapes, tracks and vias. Extraction derives interconnect electrical models from the implemented geometry using the selected technology information. These steps answer progressively different questions: is there plausible capacity, can the actual connections be drawn legally, and what electrical consequences do those shapes create? A global route is not the final wire geometry, and a connected final route still needs timing and other signoff checks.

Application and validation in Lesson 07

Physical implementation uses compatible cell abstracts, technology data and constraints to place and connect instances. The result is interpreted according to its stage: estimated placement timing and extracted post-route timing have different interconnect evidence. An ECO preserves the required function while resolving a named violation and rechecking affected paths.

Coupled setup and hold repair. Suppose one path has setup slack $+0.6$ ns and hold slack -0.2 ns. A candidate data-path buffer adds 0.25 ns to its minimum delay and 0.35 ns to its maximum delay under the modeled corners. Hold slack becomes $-0.2 + 0.25 = +0.05$ ns; setup slack becomes $0.6 - 0.35 = +0.25$ ns. This candidate passes both arithmetic checks, whereas a buffer adding 0.70 ns to maximum delay would fail setup. Actual repair also changes slew, load, placement and routing, so the min/max increments must come from the implemented analysis rather than from one nominal buffer delay.

Complete original notebook pages for Lesson 07

Part 1 original page 17

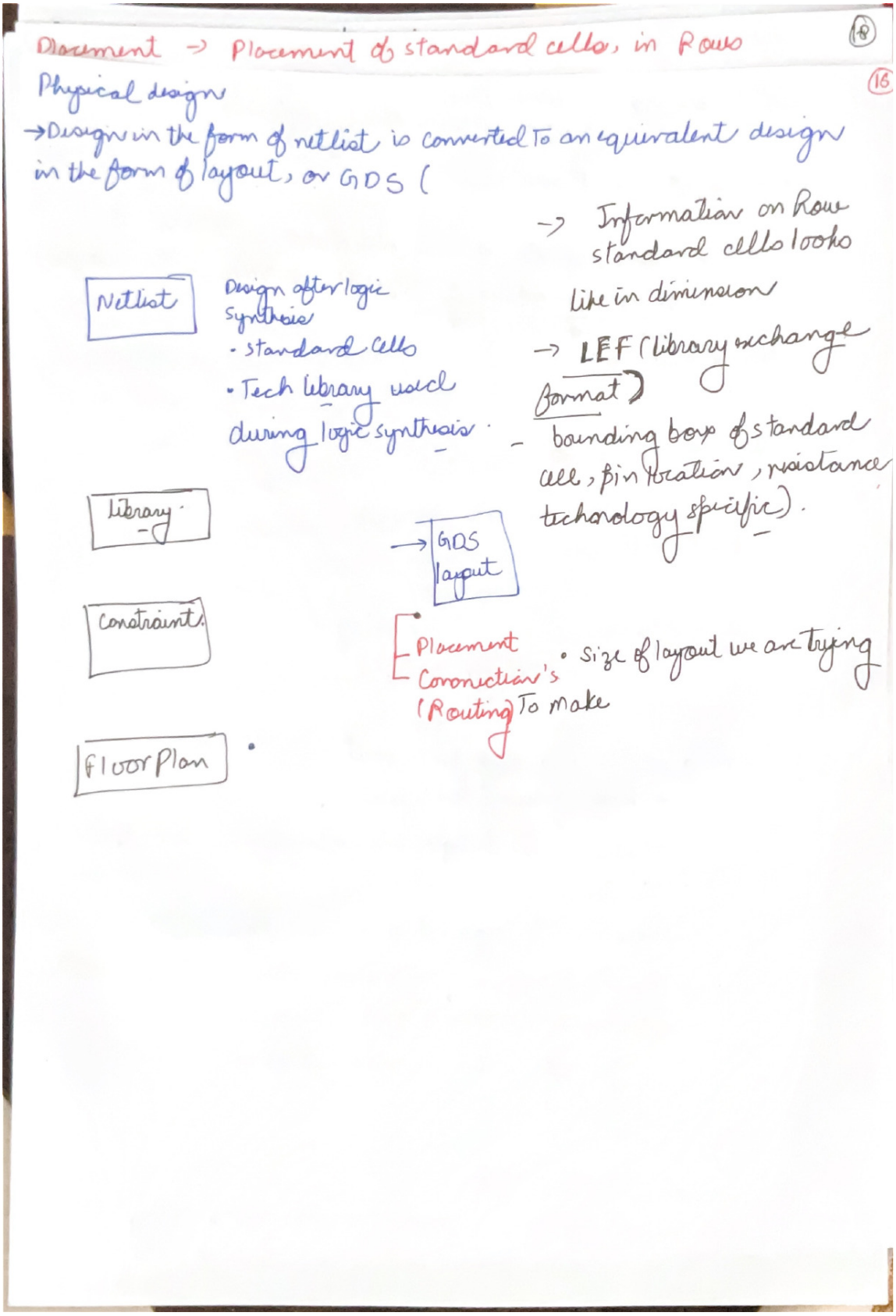


Figure 2.08. *Source: Part 1, PDF page 17. Physical design inputs and LEF.*

Explanation and worked examples: Day 2 · Lesson 07.

Part 1 original page 18

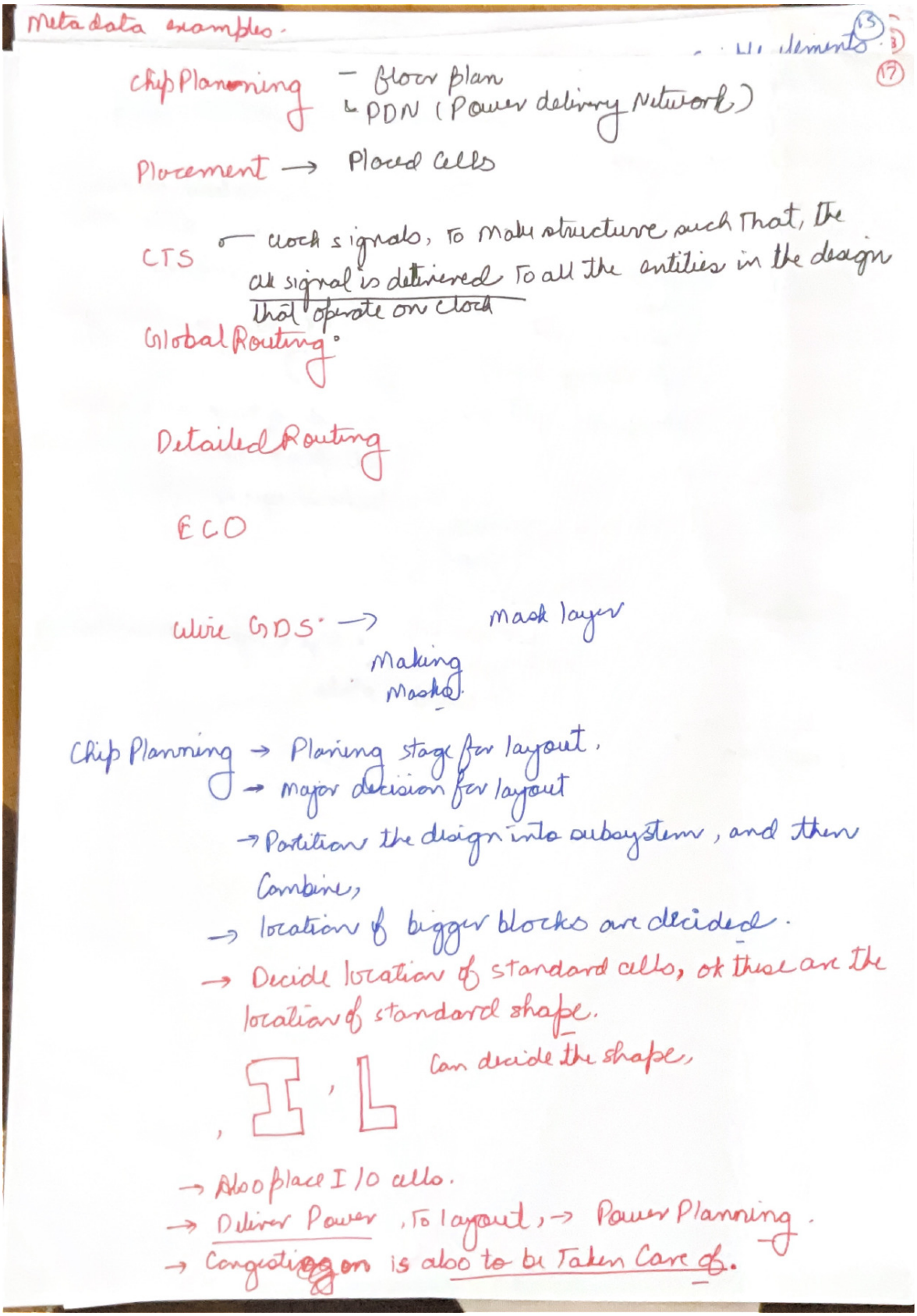


Figure 2.09. *Source: Part 1, PDF page 18. Floorplanning, PDN, placement, CTS, routing, ECO.*

Explanation and worked examples: Day 2 · Lesson 07.

Part 1 original page 19

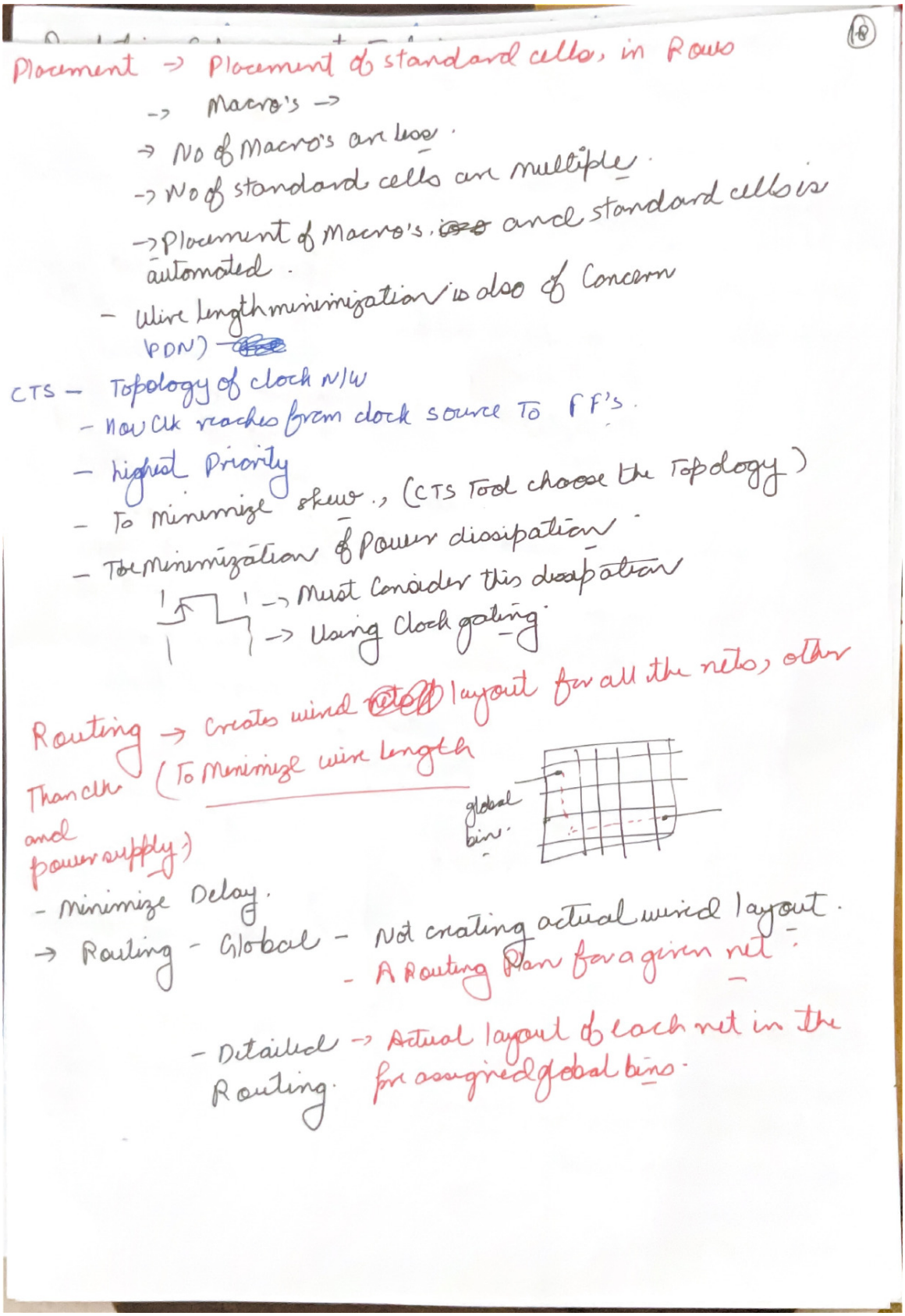


Figure 2.10. *Source: Part 1, PDF page 19. Placement, skew, clock gating, global/detailed routing.*

Explanation and worked examples: Day 2 · Lesson 07.

Part 1 original page 20

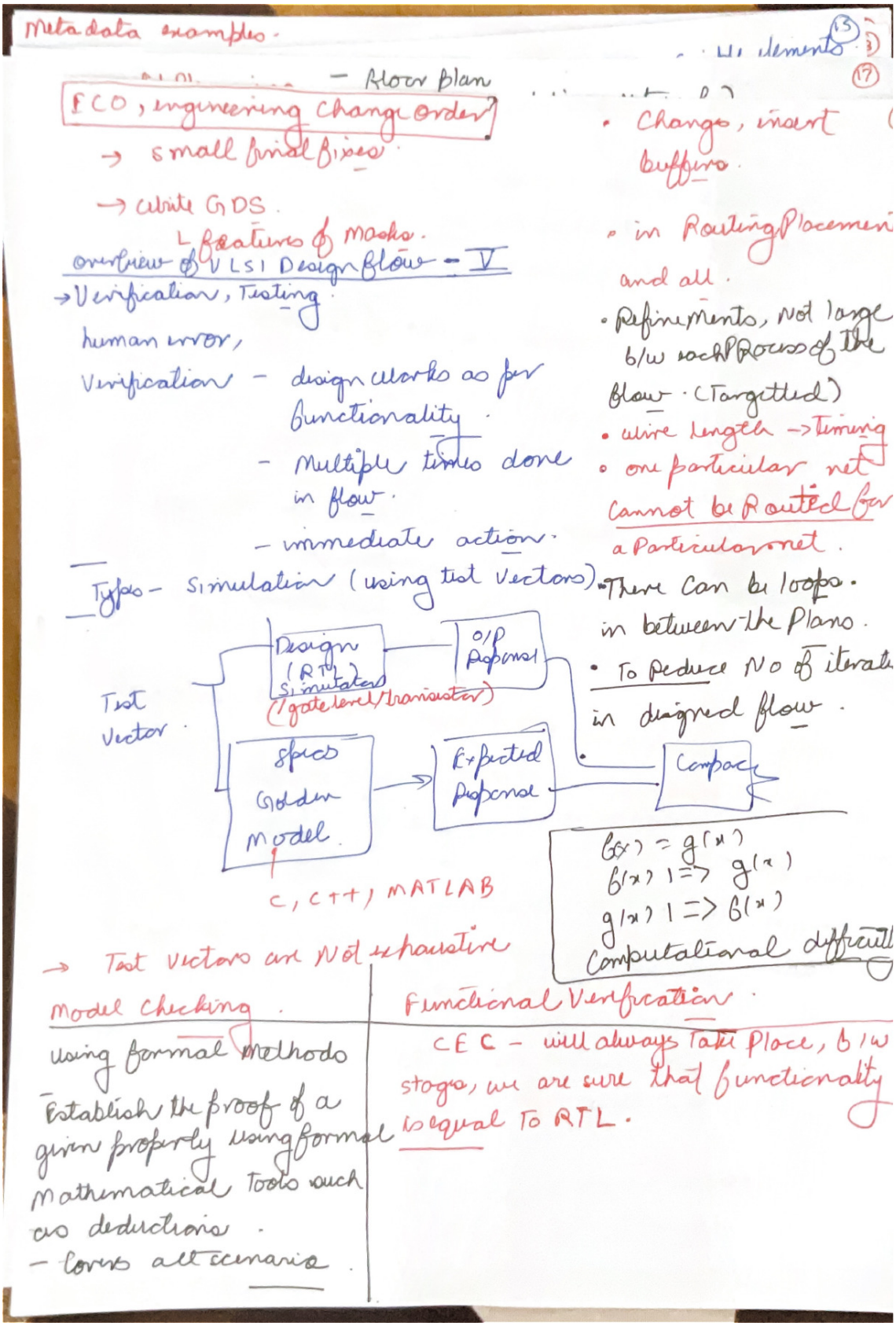


Figure 2.11. *Source: Part 1, PDF page 20. Golden model, simulation/formal verification, equivalence, ECO.*

Explanation and worked examples: Day 2 · Lesson 07 · Day 2 · Lesson 08.

Lesson 08 Verification and manufacturing test

Week 2 · Lecture video

Verification simulation and formal methods

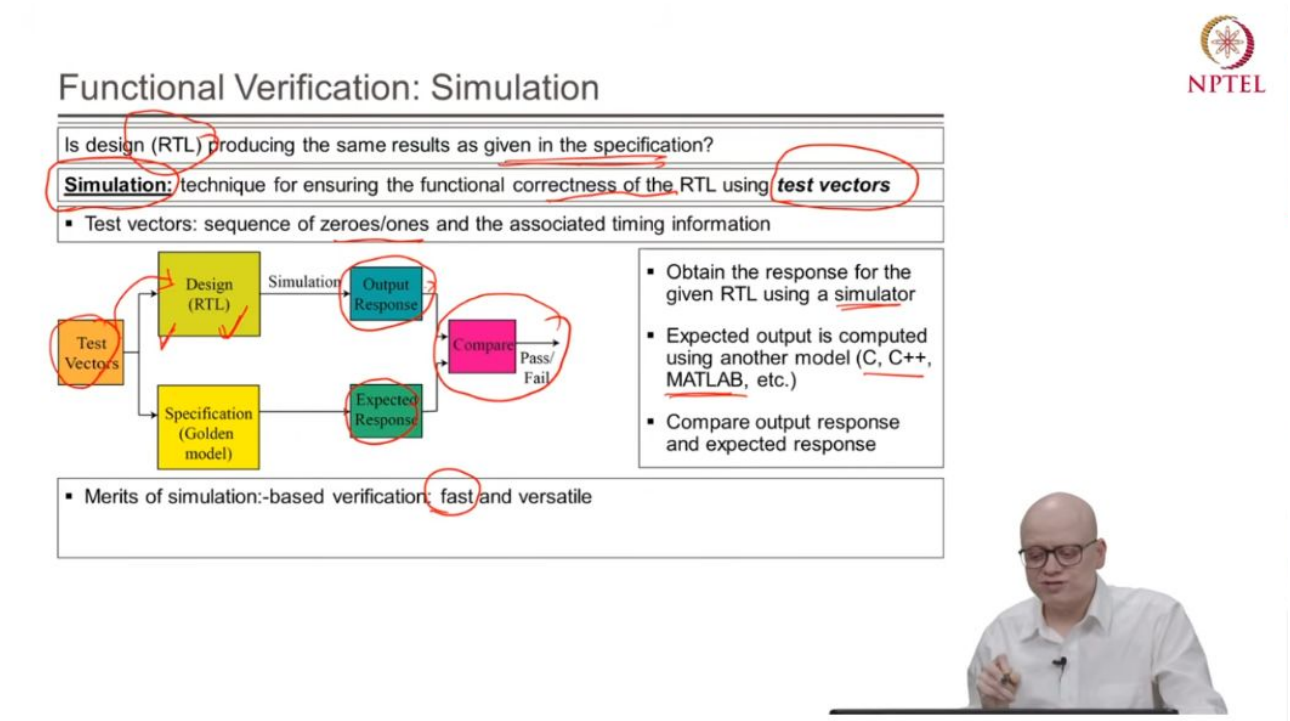


Figure 2.12. *Lecture: 9:05. Compare a design response against the expected response for the same stimulus*

Design verification checks whether a design meets its specified behavior and requirements. **Manufacturing test** checks fabricated devices for faults and acceptance criteria. Verification can find a logic error shared by every chip made from a design; manufacturing test can reject an individual die affected by a fabrication defect. Design for test prepares the design so that those later tests are effective.

In **simulation**, a simulator evaluates the design under supplied input events. A testbench applies stimulus and checks the observed response against an expected result or reference model. Stimulus includes ordering and time, not just an unordered list of zeros and ones. A sequential design's output can depend on earlier inputs and reset history.

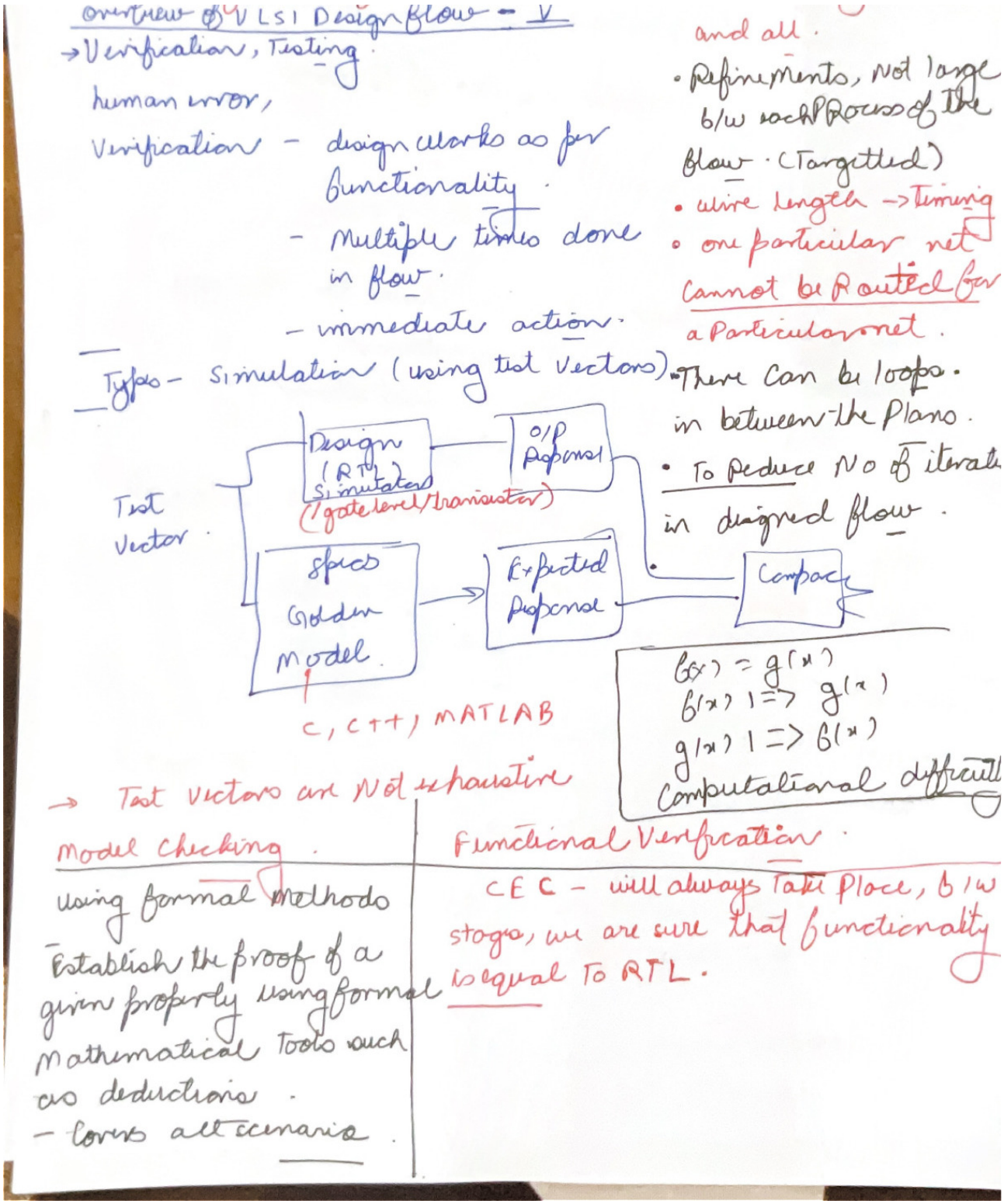


Figure 2.13. *Source: Part 1, PDF page 20, verification portion.*

The two-branch drawing is a useful verification structure: apply equivalent transactions to the implementation and a **golden/reference model**, align their outputs in time, then compare. A mismatch identifies a discrepancy; debugging determines whether the implementation, testbench, reference model, or specification is wrong. A reference model is not correct merely because it is called “golden.”

Passing a finite collection of simulation tests establishes agreement for those executed scenarios. For n independent binary inputs, a combinational truth table already has 2^n assignments; stored state

and input sequences make sequential verification much larger. Coverage measures what was exercised, but a high coverage number does not alone prove correct behavior.

Formal property checking tries to prove stated properties under explicit assumptions. A safety property might say that conflicting traffic movements are never both green. A liveness property might say an accepted request eventually receives a grant; that requires suitable assumptions about the environment and fairness. A successful proof covers the modeled cases allowed by those assumptions. An inconclusive proof is neither a pass nor a demonstrated design failure.

Equivalence checking compares two representations, such as RTL and a synthesized netlist. The lecture introduces combinational equivalence checking across transformations. Designs whose state encoding, pipeline latency, or reset behavior changes may require additional correspondence information or sequential equivalence techniques. Equivalence to an incorrect reference preserves that error, so the original specification-to-RTL verification remains necessary.

Timing and physical verification

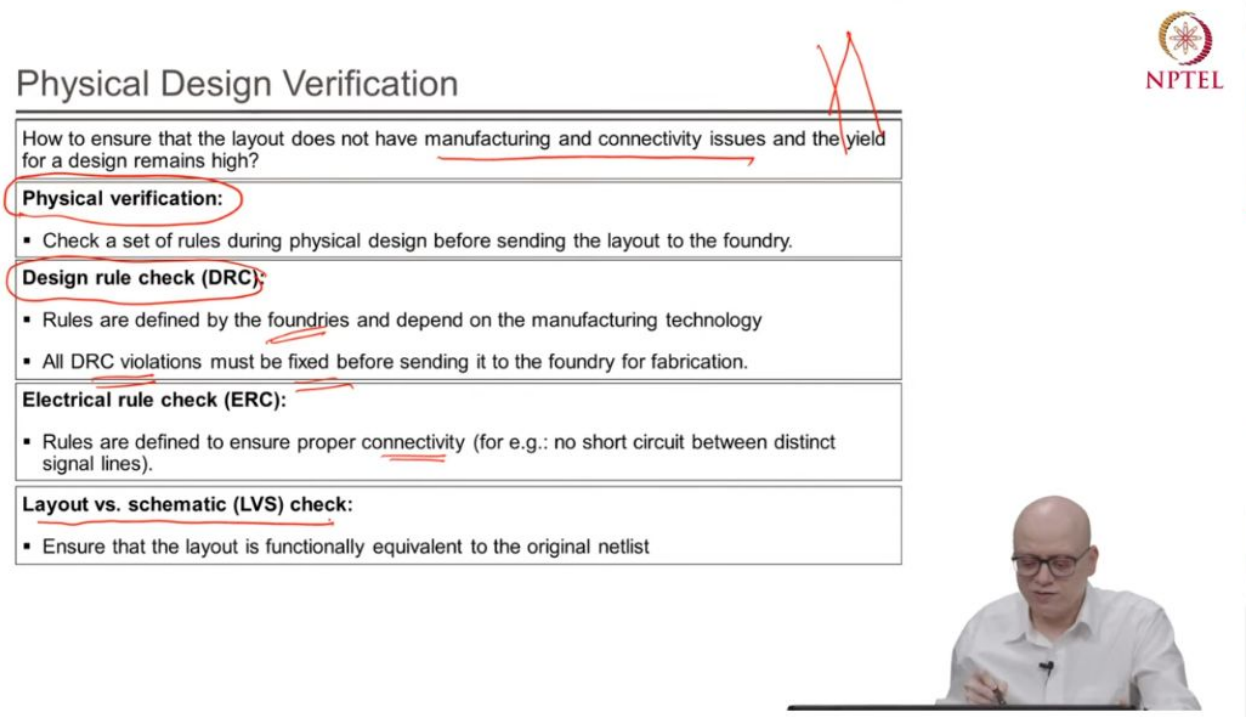


Figure 2.14. *Lecture: 25:42. Physical verification complements functional and timing checks*

Physical verification asks whether the manufactured geometry can implement the intended circuit under the selected technology rules. DRC checks geometry, LVS compares extracted devices and connections with a reference, and ERC checks electrical rules. Timing analysis answers a separate question: whether signals can arrive and remain stable when required.

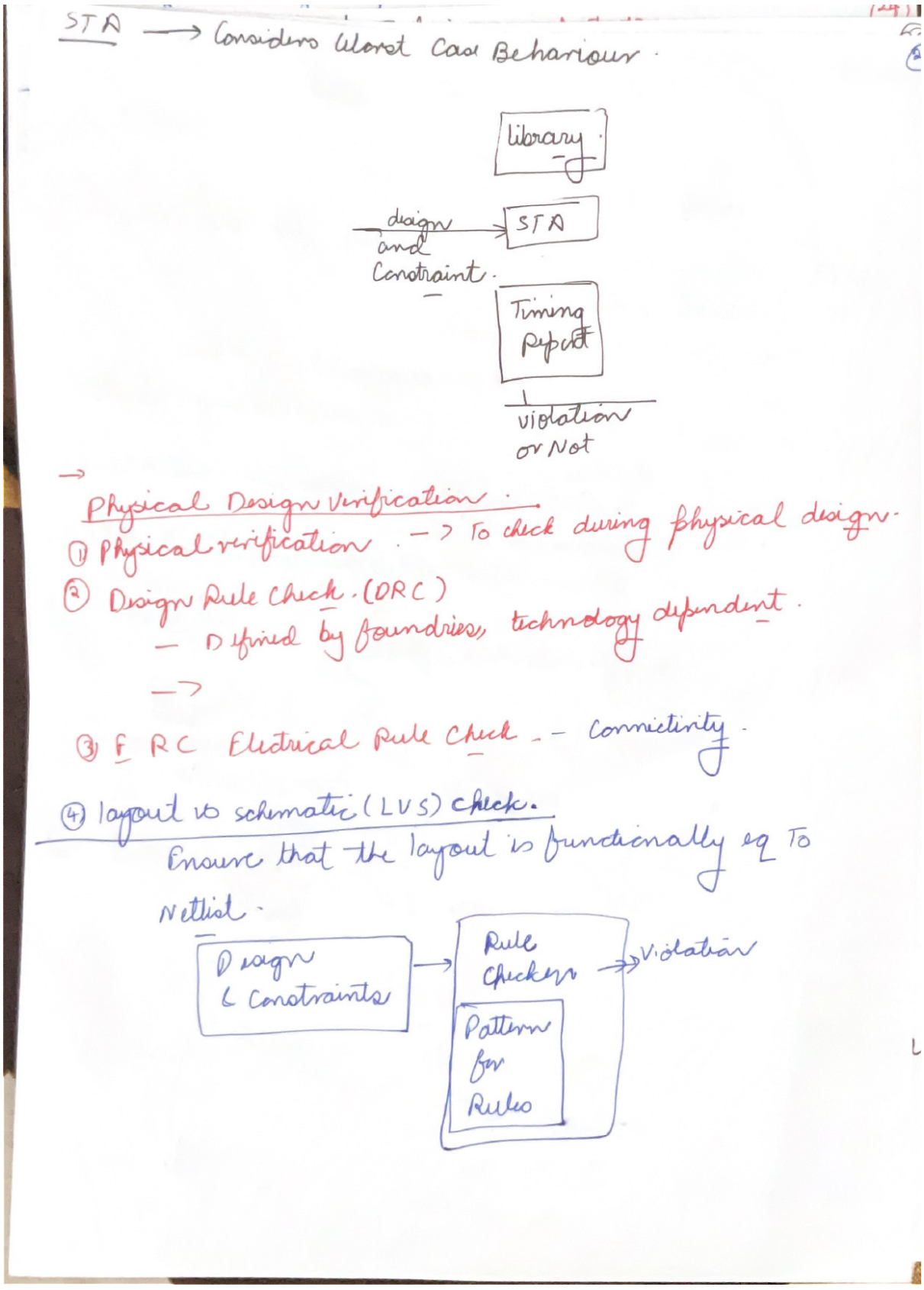


Figure 2.15. *Source: Part 1, PDF page 21.*

Static timing analysis (STA) evaluates timing paths using the netlist, cell models, interconnect estimates or extracted parasitics, clocks, and constraints. “Static” means that it does not require an explicit functional stimulus sequence to calculate each path's timing. It checks setup, hold, and other relevant requirements across chosen modes and corners.

The phrase “considers worst-case behavior” should be read **within the modeled conditions and constraints**. A passing report cannot compensate for a missing clock, incorrect exception, unmodeled operating condition, or wrong library. Pessimistic modeling can create conservative reports; incomplete modeling can still hide real problems. The setup budget in Lesson 04 shows the components of one check.

Check	Main question	What it does not establish by itself
DRC: design rule checking	Do geometric features obey the selected process rule deck?	Correct algorithm or complete electrical behavior
LVS: layout versus schematic	Does extracted layout connectivity and device information match the intended reference?	All timing requirements or complete system functionality
ERC: electrical rule checking	Are specified electrical-connection and usage rules obeyed?	Every possible physical failure
STA	Do modeled paths meet temporal constraints in the analyzed scenarios?	Functional correctness
RTL/constraint/netlist rule checks	Are suspicious constructs, conflicts, missing intent, or illegal structures present?	A proof of all design requirements

The LVS wording “functionally equal” is understandable, but the concrete check compares extracted devices/connectivity against a reference under tool rules. It is not the same task as proving a high-level algorithm. DRC rules are technology-specific. ERC content varies by tool and rule deck; shorts and opens can also be exposed through connectivity comparison. These checks overlap, but their purposes should remain distinct.

Defects faults and test patterns

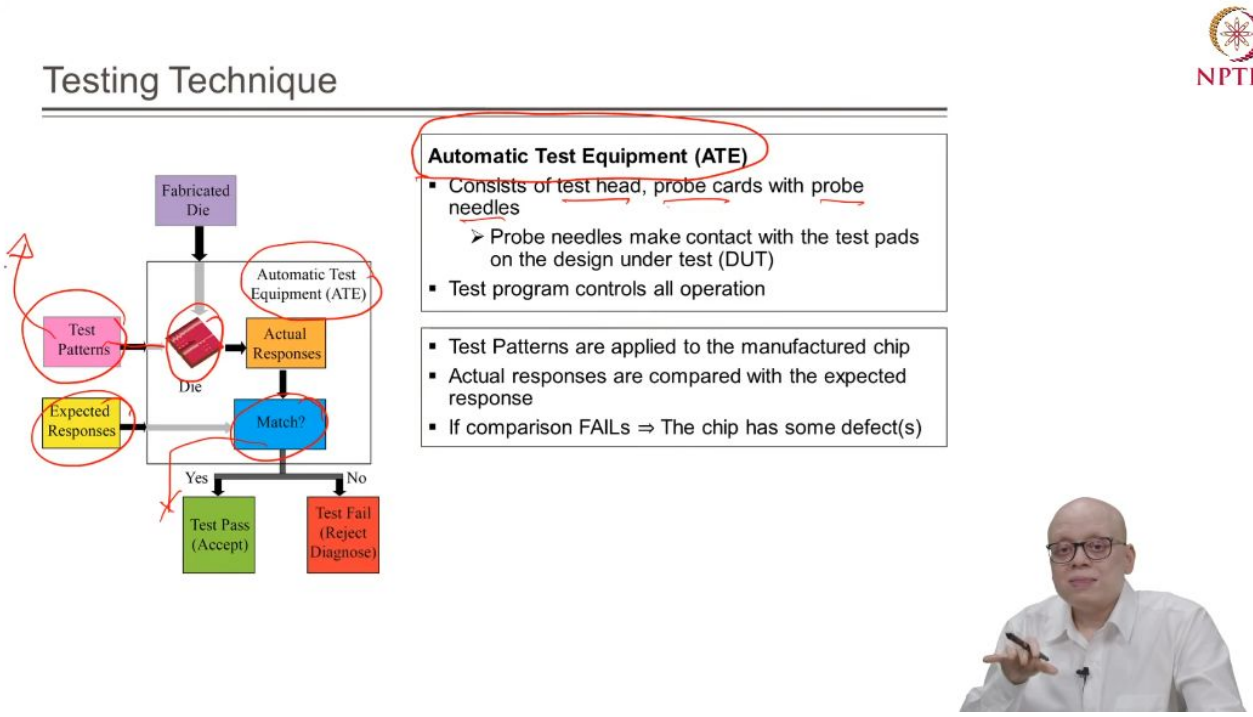


Figure 2.16. *Lecture: 50:47. Test patterns, expected responses, actual responses, and the pass/fail decision.*

Follow the two inputs to the comparison. One comes from the **actual fabricated die**, after the tester applies a pattern. The other is the **expected response** prepared for the intended circuit and test conditions. A pattern is useful for a targeted fault only when it makes the faulty response distinguishable from the expected one. Merely toggling many inputs does not establish that a hidden fault will become observable.

The probe card and needles provide electrical access at wafer test; the test program controls stimulus, timing, and measurements. “Match” therefore includes what is measured and when it is sampled. A mismatch is evidence that the tested setup did not meet the expectation. Diagnosis must still distinguish a device defect from an incorrect pattern, expectation, contact, or test condition. Passing means passing these specified tests, rather than proving the absence of every possible defect.

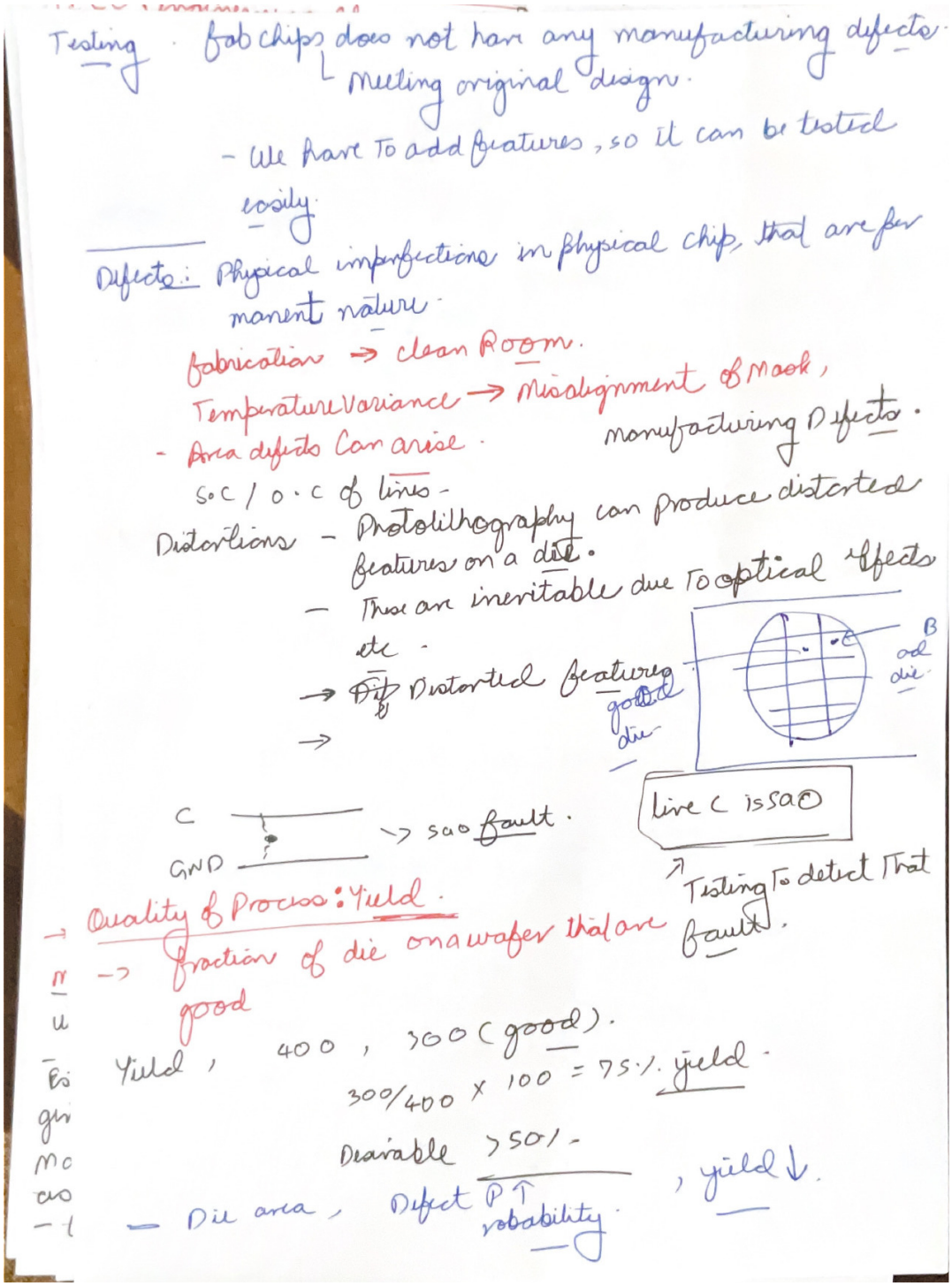


Figure 2.17. *Source: Part 1, PDF page 22.*

A **defect** is a physical imperfection. A **fault model** is an abstract representation of a possible incorrect circuit behavior caused by defects. A **failure** is an observed violation of required behavior. One physical defect can have several effects, and one fault model can abstract several different defects.

In the example, a short to ground can be modeled as a **stuck-at-0 fault** on a line. To detect that fault, a test must both excite it and propagate its effect to an observable point. If a good line would also be 0 under the chosen input, the short cannot be distinguished by that observation. If the good value is 1 but downstream logic masks it, the fault still escapes that test.

For a simple AND gate $y = a \wedge b$, testing an `a` input stuck at 0 requires setting `a=1` and `b=1`: the good output is 1 and the faulty output is 0. Setting `b=0` masks the difference. This illustrates **controllability** and **observability**, the two central practical obstacles that test structures help address.

Work out a test that activates and exposes a fault

Extend the short-to-ground example to two gates: $n = a \wedge b$, followed by $y = n \vee c$. The internal net n is the fault site; the tester observes only y. Assume settled binary inputs and a single stuck-at fault on n.

To detect n **stuck at zero**, first activate the difference: the good circuit must make $n = 1$, requiring $a = b = 1$. Next propagate it: set $c = 0$ so the OR gate does not force both outputs to one. The pattern $(a, b, c) = (1, 1, 0)$ then gives good $y = 1$ and faulty $y = 0$.

Input pattern `(a,b,c)`	**Good** `n`	**Good** `y`	`y` **with** `n` **stuck at 0**	`y` **with** `n` **stuck at 1**
`(0,0,0)`	0	0	0; not detected	1; detected
`(0,1,0)`	0	0	0; not detected	1; detected
`(1,1,0)`	1	1	0; detected	1; not detected
`(1,1,1)`	1	1	1; masked	1; not detected

The last row activates the stuck-at-zero difference internally, but `c=1` masks it at the observable output. This separates **activation** from **propagation**. To detect stuck-at-one, make the good `n` zero and keep `c=0`. Thus two carefully selected patterns, `(0,1,0)` and `(1,1,0)`, detect both faults in this deliberately limited fault set.

That result does not establish coverage for every fault on every input, wire, transistor, or clock path. A coverage percentage needs an explicit denominator. Here the denominator is exactly two modeled faults on one named net. In a sequential circuit, reaching the required internal state may also require a sequence of clocks and inputs. DFT structures improve the ability to establish and observe those states; they do not remove the need to define the faults and expected responses correctly.

Process variation, contamination, alignment error, and other mechanisms can affect manufactured structures. Optical distortion is a pattern-fidelity problem addressed through process and mask techniques; an electrical manufacturing test is not a microscope inspecting every rounded corner. If a distortion creates an electrically relevant failure, suitable electrical tests may detect its consequence. An inconsequential geometric imperfection need not fail an electrical acceptance test.

Yield fault coverage and escapes

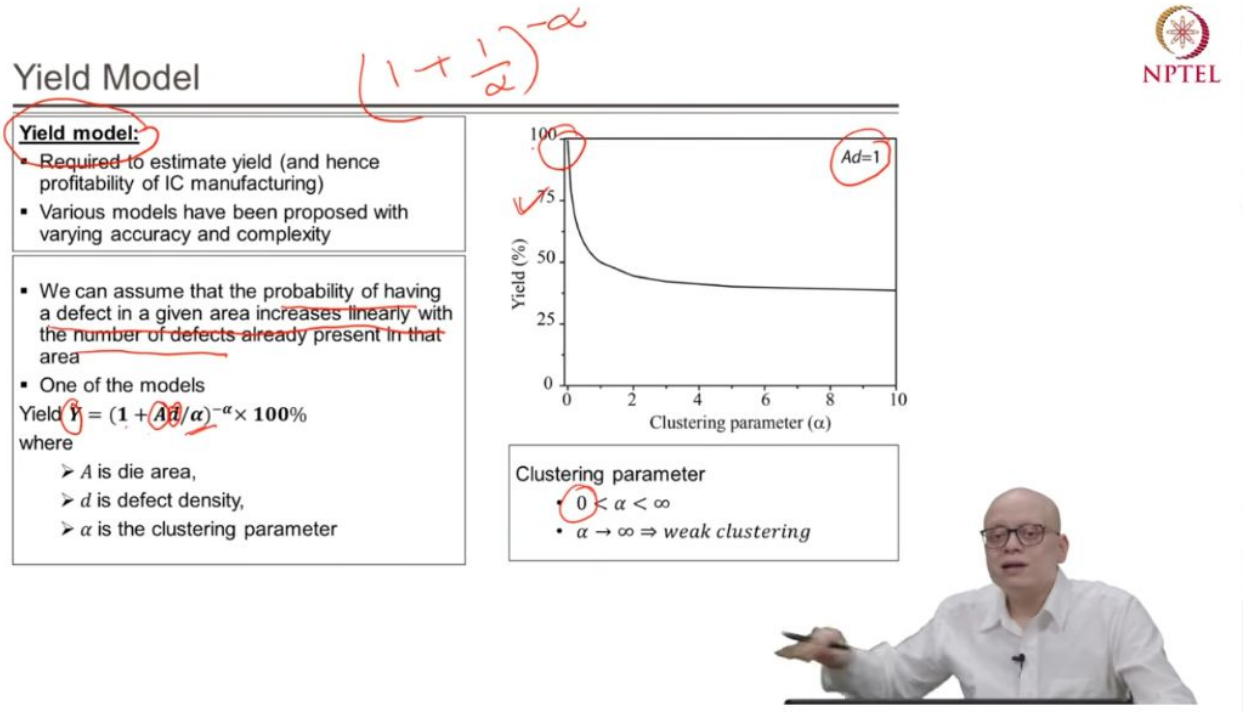


Figure 2.18. *Lecture: 47:24. Yield depends on die area, defect density, and defect clustering*

A yield model connects the fraction of acceptable dies with factors such as die area, relevant defect density, and defect clustering. Larger area usually exposes each die to more defect opportunities when the other factors are held fixed. The clustering parameter changes how those opportunities are distributed across dies; the model below makes those assumptions explicit.

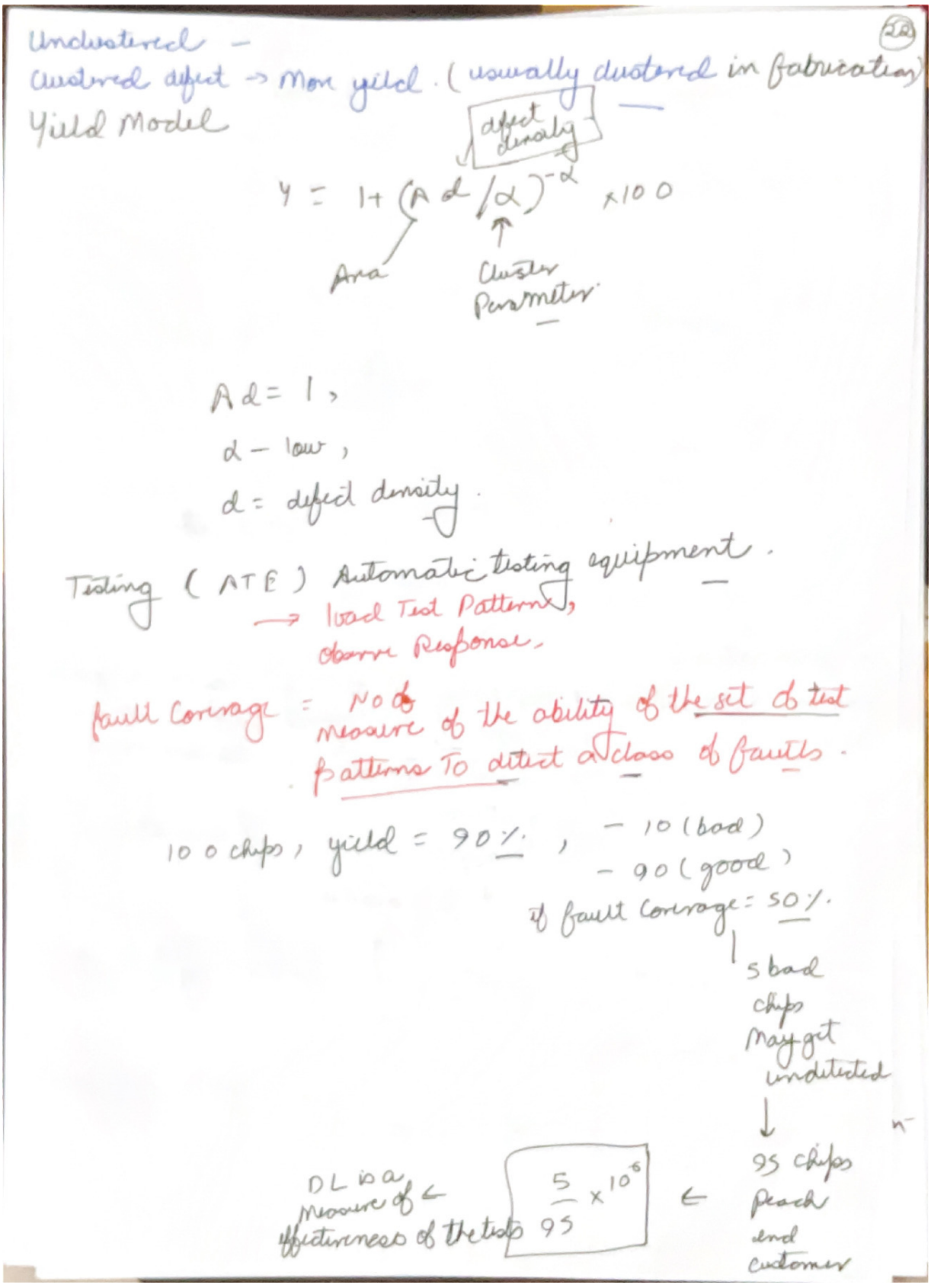


Figure 2.19. *Source: Part 1, PDF page 23.*

Yield is a fraction of manufactured units meeting specified goodness criteria at a stated stage. The previous page's example is correct: 300 good dies out of 400 gives $300/400 = 0.75$, or 75%. It does not imply that every mature process must exceed a single universal percentage; die area, design, process maturity, and acceptance criteria matter.

The clustered-defect yield model in the lecture is

$$Y = \left(1 + \frac{Ad}{\alpha}\right)^{-\alpha}.$$

The formula gives Y as a fraction; multiplying by 100 gives its numerical value in percent. Here A is die area, d is the relevant defect density per unit area, and $\alpha > 0$ is a clustering parameter. The exponent applies to the **whole parenthesis**, a detail that is easy to lose in handwriting. Ad must be dimensionless. This is a model fitted to a process and defect population, not an exact physical law for every yield mechanism.

For $Ad = 1$, $\alpha = 1$ gives $Y = 1/2 = 0.50$, or 50%; $\alpha = 2$ gives $Y = (1.5)^{-2} \approx 0.4444$, or about 44.44%. As α becomes very large, this expression approaches e^{-Ad}, giving about 36.79% for $Ad = 1$. Smaller positive α corresponds to stronger clustering in this model. The same number of defects concentrated on already-bad dies can leave more other dies untouched. Deliberately adding defects is not a yield-improvement strategy; the observation compares distributions at a stated defect density.

Fault coverage measures detected modeled faults divided by the relevant modeled-fault population, with the exact denominator stated by the report. A coverage figure depends on fault model, exclusions, and treatment of untestable faults. It is not the fraction of bad chips automatically detected in all circumstances. Even 100% stuck-at coverage does not cover every delay, analog, intermittent, or other unmodeled physical failure.

Defect level measures the fraction of bad units among units that passed test. The 100-chip example assumes 90 good and 10 bad, and further assumes the test detects exactly 5 of those 10 bad units. It then passes 95 units, of which 5 are bad:

$$DL = \frac{5}{95} \times 10^6 \approx 52{,}632 \text{ parts per million.}$$

The arithmetic is correct under that assumption. The shortcut “50% fault coverage means exactly half the bad chips are detected” is a teaching simplification, not a general equivalence. Real escape probability depends on the physical fault distribution, multiple faults per die, and the test set. The denominator is **95 passed chips**, not the original 100.

Automatic test equipment and design for test

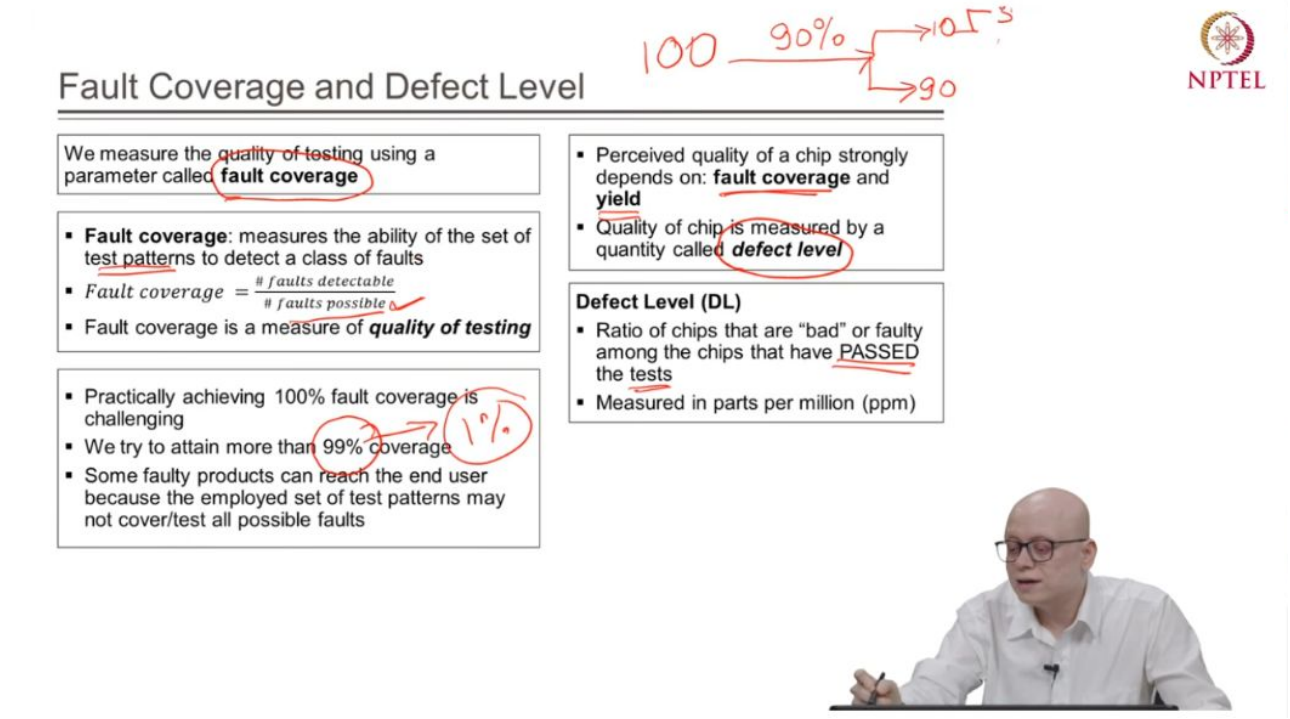


Figure 2.20. *Lecture: 55:40. Test quality affects which defective devices escape detection*

A test program screens manufactured devices by applying conditions and comparing responses. Its fault coverage describes a modeled fault population, while defect level describes bad devices among the devices that pass. The slide places those two quality measures side by side; they must not be treated as interchangeable percentages.

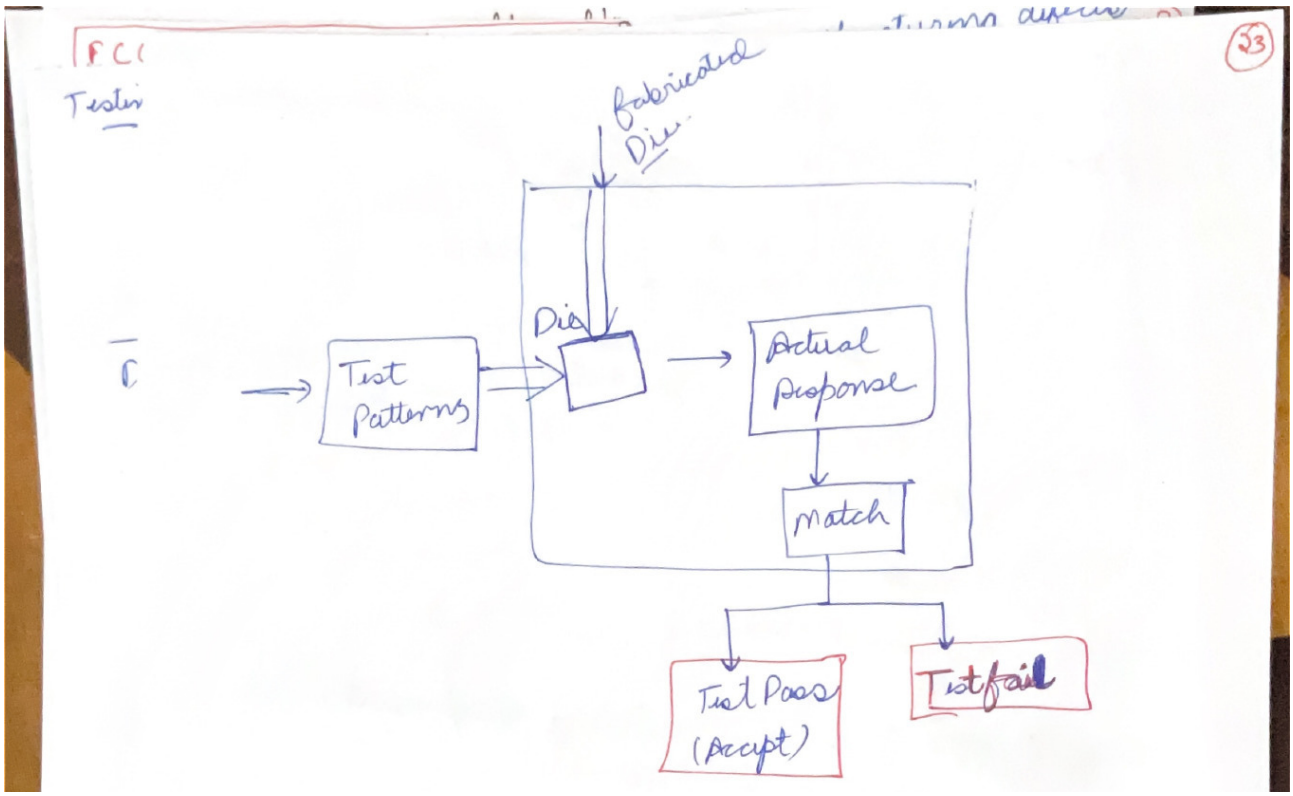


Figure 2.21. *Source: Part 1, PDF page 24, upper portion.*

Automatic test equipment (ATE) supplies test conditions and patterns, measures responses, and compares them with acceptance criteria. At wafer test, a probe interface makes electrical contact to die pads. Packaged devices are tested through a suitable package interface. The test program defines timing, stimulus, expected values, and measurements; a diagram showing a comparator is an abstraction of that larger process.

The pass/fail drawing should read “passes the specified tests,” not “proved physically perfect.” Failed devices are screened out; test diagnosis and process feedback can identify systematic problems for future production. **Design for test (DFT)** adds structures and plans that improve access to internal state and observation, such as scan or built-in test in appropriate designs. Test-pattern generation and expected-response preparation occur before manufactured devices reach the tester.

The bottom line of the final handwritten page begins “Functional Verification - Simulation.” It is preserved in the source archive and its definition is covered in this chapter. The dedicated simulation lecture is now covered in Day 03, Lesson 13.

Recall checks: Can a design pass LVS but fail timing? Can an incorrect RTL and its correctly synthesized netlist be equivalent? Why is fault coverage not interchangeable with yield?

Definitions and mechanisms in Lesson 08

Verification and manufacturing test. Design verification checks a representation against requirements or another intended representation. Manufacturing test measures fabricated devices under specified conditions to classify their acceptability. A design error can be reproduced on every die, while a fabrication defect can affect an individual unit. Test structures are planned during design, so the activities are connected, but they target different evidence. Passing a simulation is not a manufacturing-yield result, and passing a device test is not a proof of every intended behavior.

Simulation, reference model and formal property. Simulation executes the design model for supplied stimulus and compares observations with expectations. A reference model defines expected behavior at a suitable abstraction and must itself be justified. A formal property states a condition to prove under explicit assumptions. The assurance depends on what was simulated or proved, the correctness of the reference and the validity of the environment assumptions. “The test passed” needs a named test and expected result; “the property was proved” needs the property and its assumptions.

Equivalence, DRC, LVS and STA. Equivalence compares modeled behavior between representations. DRC checks geometry against a process rule deck. LVS compares extracted layout devices and connectivity with a circuit reference. STA checks modeled temporal requirements using clocks, constraints and delay information. These are complementary checks with different objects and acceptance criteria. A layout can match its reference and still miss timing; an equivalent netlist can faithfully preserve an error in the original RTL. Avoid treating any one pass as a certificate for all the others.

Defect, fault and failure. A defect is a physical imperfection. A fault is a modeled abnormal circuit behavior used to reason about consequences and test detection. A failure is an observed violation of required behavior. A short can motivate a stuck-at model, but physical defects and modeled faults do not correspond one-to-one in all cases. This distinction explains why detecting all faults in a selected model does not prove the absence of all physical defects or all possible failures.

Activation, propagation and observation. Activation makes the good and faulty circuits differ at the fault site. Propagation preserves that difference through the intervening logic. Observation makes the difference available at a measured output or accessible state. In the AND-then-OR example, a and b both high activate an internal stuck-at-zero fault, but c must be low to keep the OR gate from masking it. A pattern can activate a fault and still fail to detect it when the response is unobservable.

Yield, fault coverage and defect level. Yield counts acceptable units among manufactured units at a stated stage. Fault coverage counts detected modeled faults among the stated fault population. Defect level counts bad devices among devices that pass the test. The denominators differ, so the percentages cannot be substituted. In the 100-device teaching example, five bad devices among 95 passing devices gives the escape fraction among shipped candidates. A 50% modeled-fault coverage figure alone would not justify assuming that exactly five of ten bad devices are detected.

Application and validation in Lesson 08

Verification evidence identifies the design representation, requirements, stimulus or assumptions, and the comparison performed. Manufacturing fault detection requires both activation of the named fault and propagation to an observable output. These obligations apply to the specific modeled fault rather than to every possible physical defect.

Deriving outgoing defect level. Let the manufactured-good fraction be Y and let c be the fraction of bad units detected by the specified test. Assume no good unit is rejected and no repair occurs. Passing units form fraction $Y + (1 - Y)(1 - c)$; escaping bad units form fraction $(1 - Y)(1 - c)$. The bad fraction among shipped passing units is their ratio. For $Y = 0.9$ and $c = 0.5$, it is $0.05/0.95 \approx 0.0526$, or about 5.26%. The denominator is the passing population, not all manufactured units. Here c is an assumed bad-unit detection probability; it must not be silently replaced by a modeled stuck-at coverage percentage.

Complete original notebook pages for Lesson 08

Part 1 original page 21

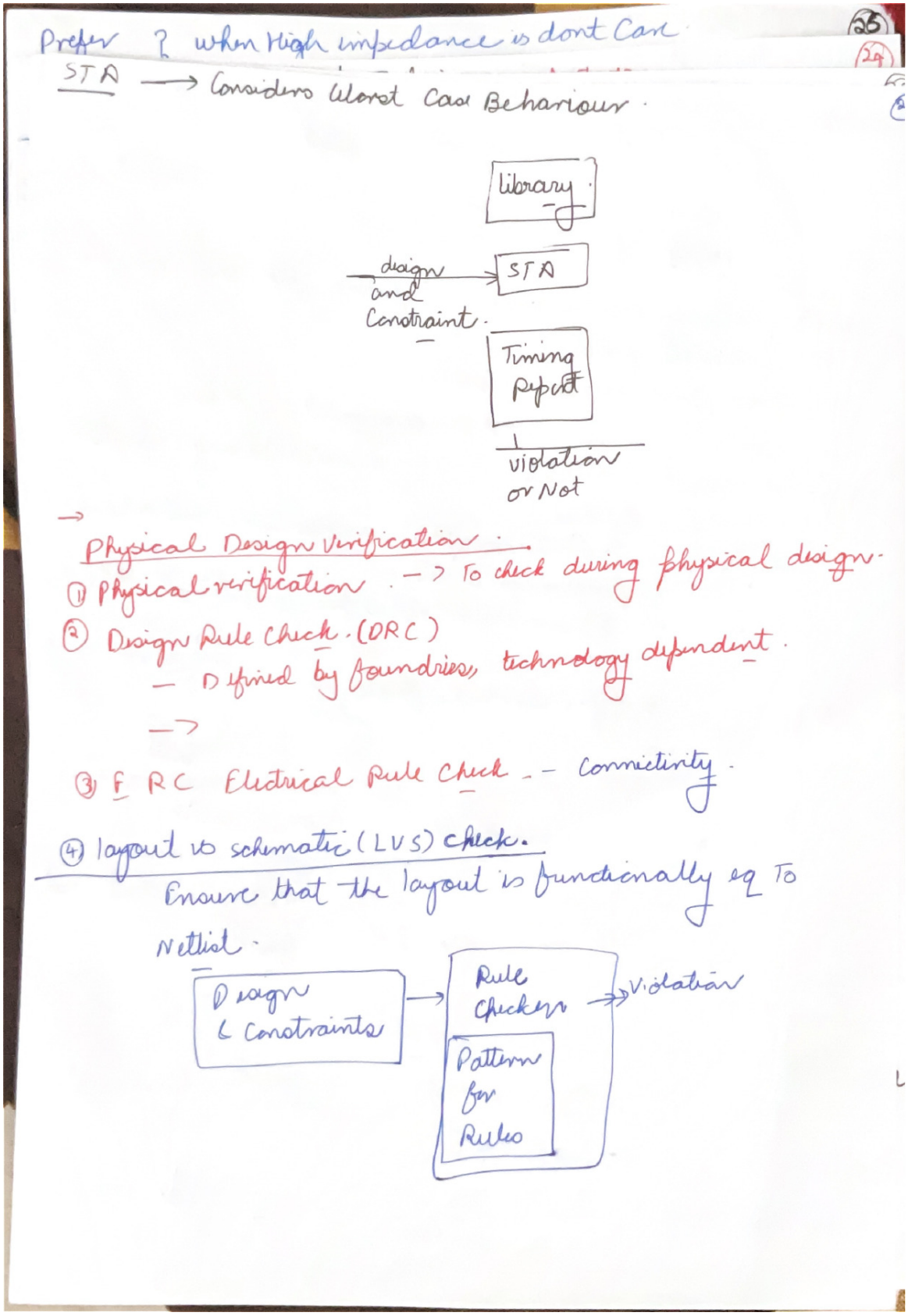


Figure 2.22. *Source: Part 1, PDF page 21. STA, DRC, ERC, LVS; small high-impedance reminder.*

Explanation and worked examples: Day 2 · Lesson 08 · Day 2 · Lesson 11.

Part 1 original page 22

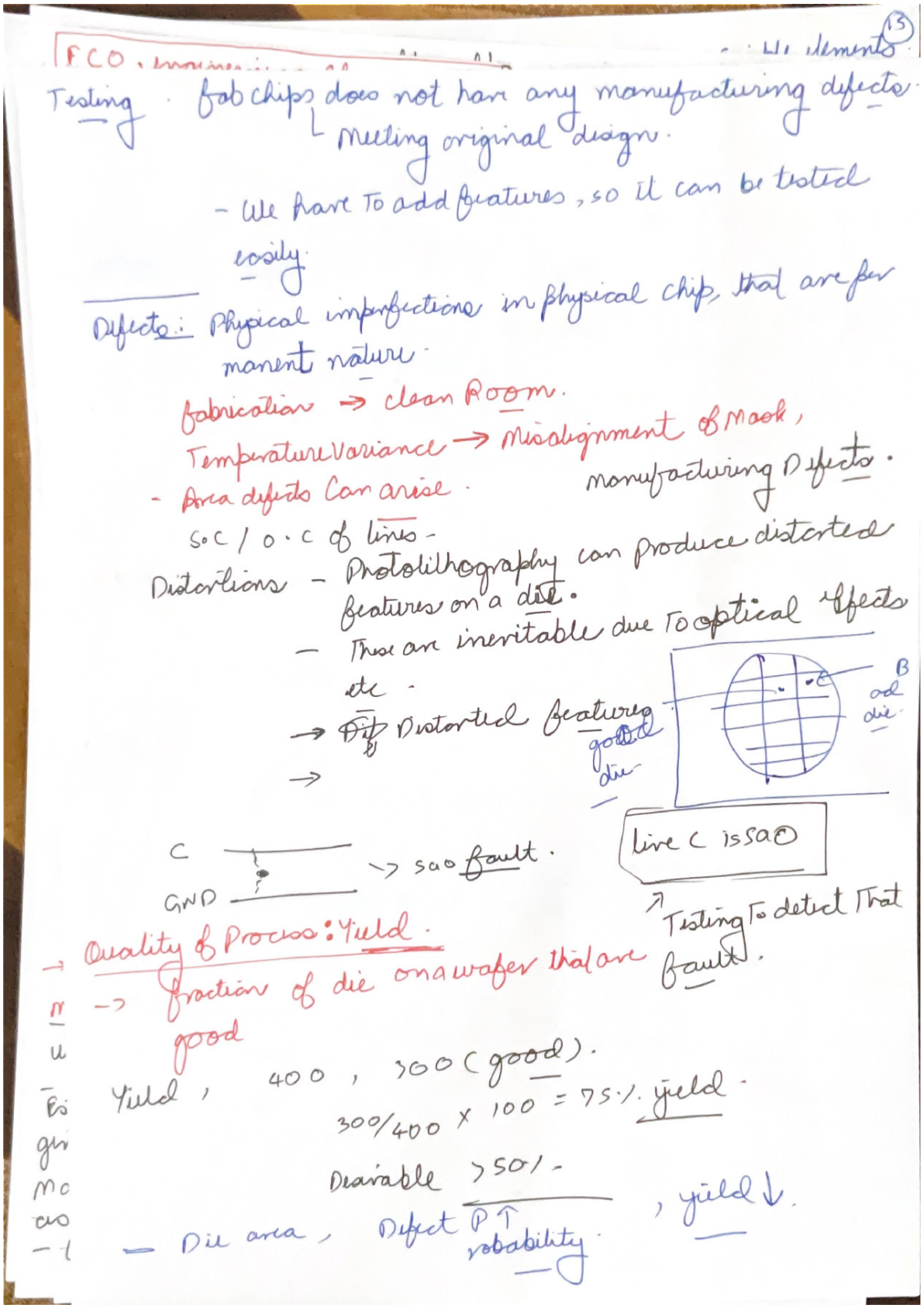


Figure 2.23. *Source: Part 1, PDF page 22. Manufacturing defects, faults, testing, yield.*

Explanation and worked examples: Day 2 · Lesson 08.

Part 1 original page 23

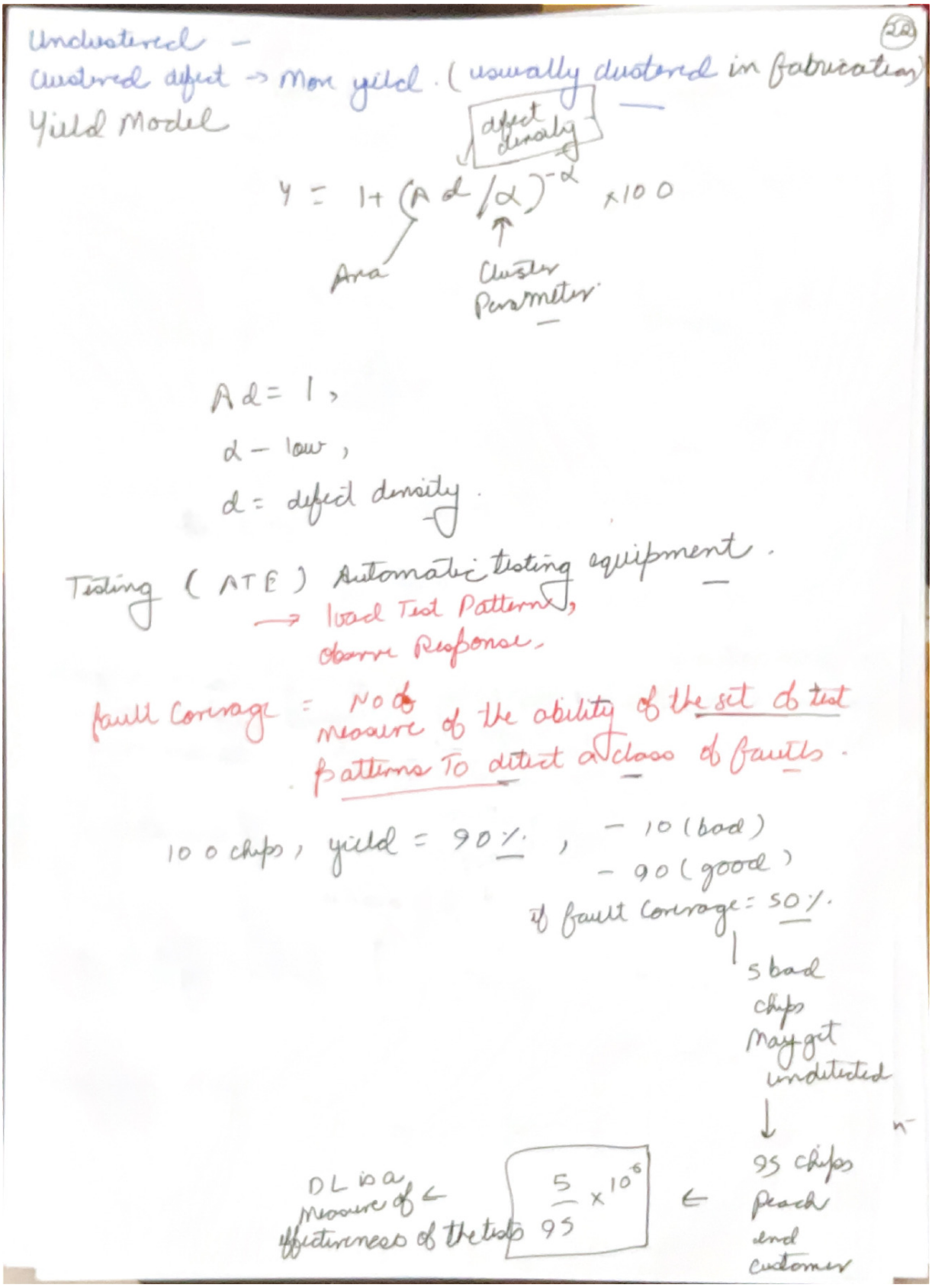


Figure 2.24. *Source: Part 1, PDF page 23. Clustered yield model, ATE, fault coverage and defect escapes.*

Explanation and worked examples: Day 2 · Lesson 08.

Part 1 original page 24

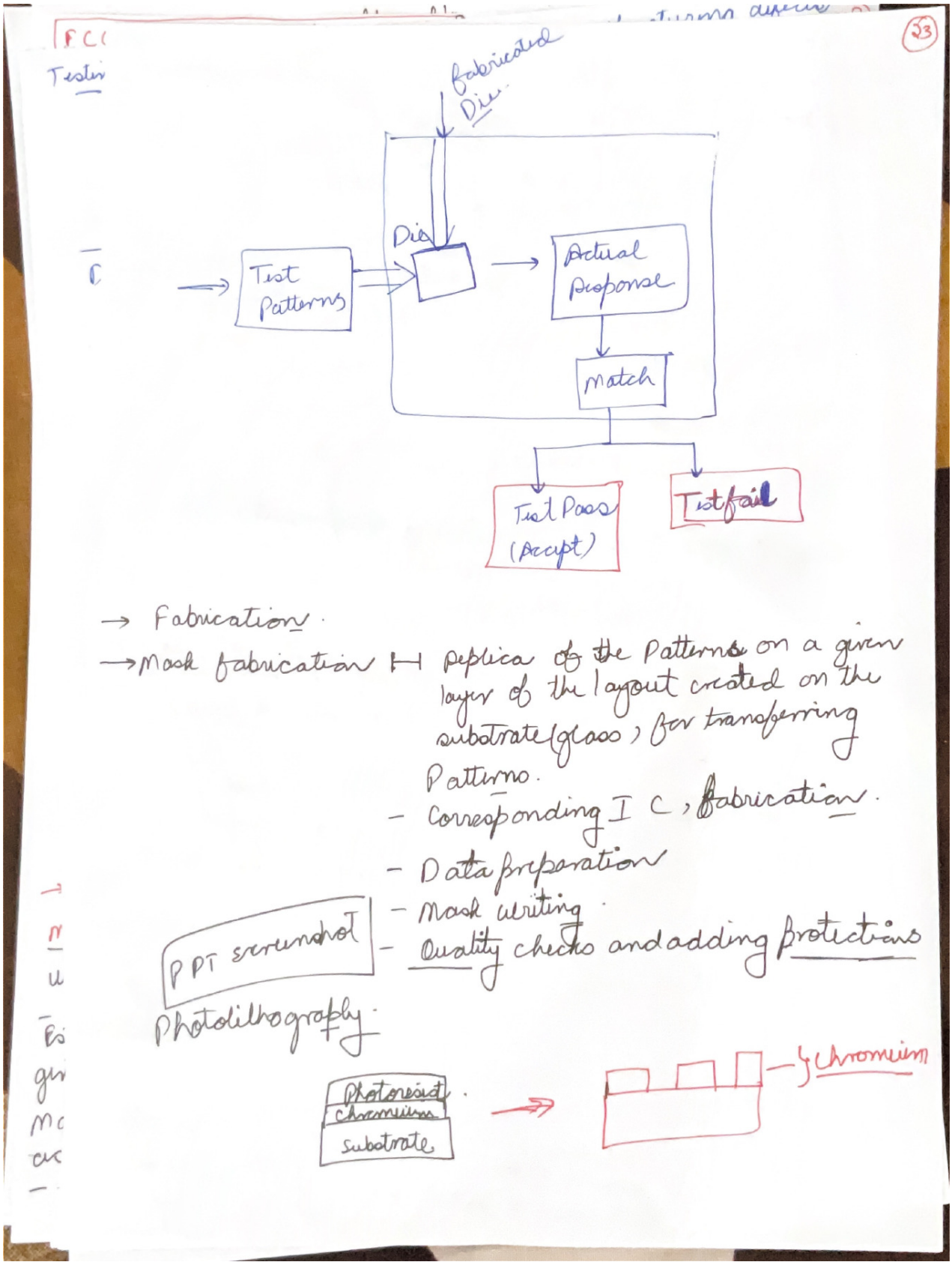


Figure 2.25. *Source: Part 1, PDF page 24. ATE pass/fail and mask preparation; screenshot reminder.*

Explanation and worked examples: Day 2 · Lesson 08 · Day 2 · Lesson 09.

Lesson 09 From layout to a packaged chip

Week 2 · Lecture video

Mask data preparation and mask writing

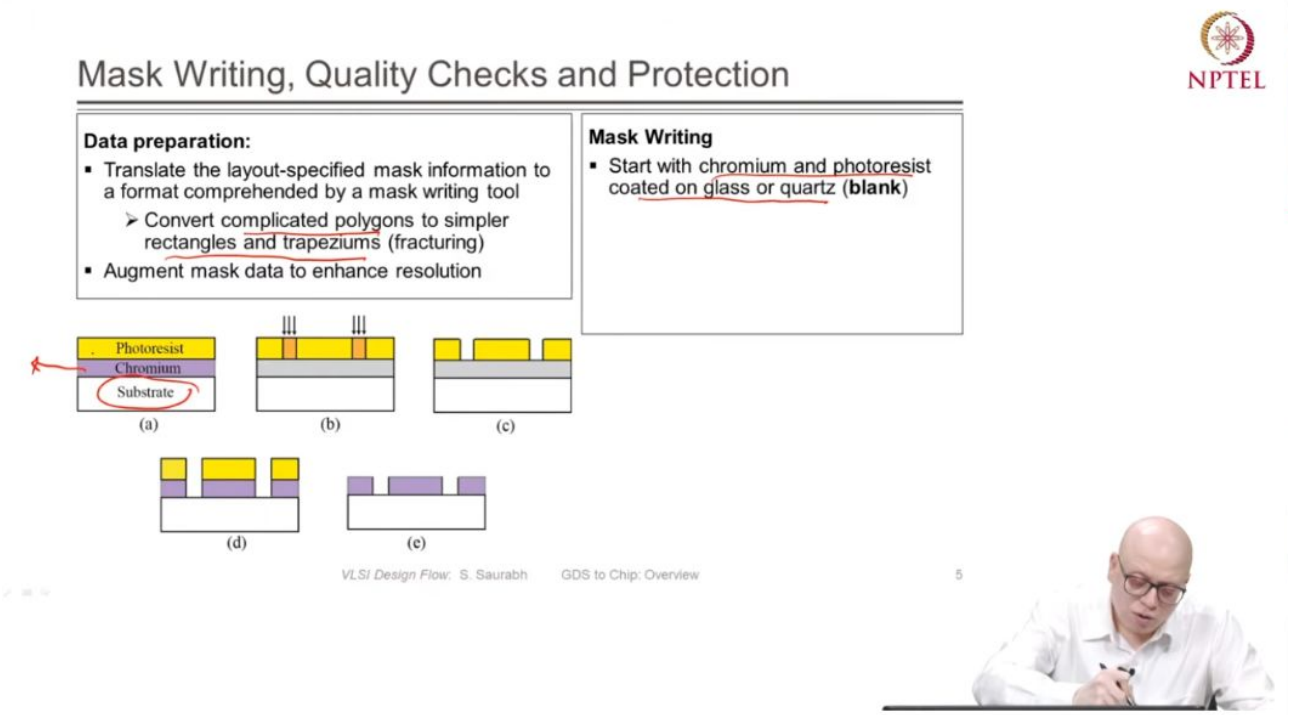


Figure 2.26. *Lecture: 5:44. Mask manufacture patterns an absorbing film on a mask blank*

Layout release is followed by manufacturing-data preparation. **Fracturing** breaks complex layout polygons into shapes supported by the selected mask-writing process. Resolution-enhancement processing can modify the mask geometry so the wafer result more closely matches the intended design. The manufactured mask therefore need not be a literal unchanged copy of every layout outline.

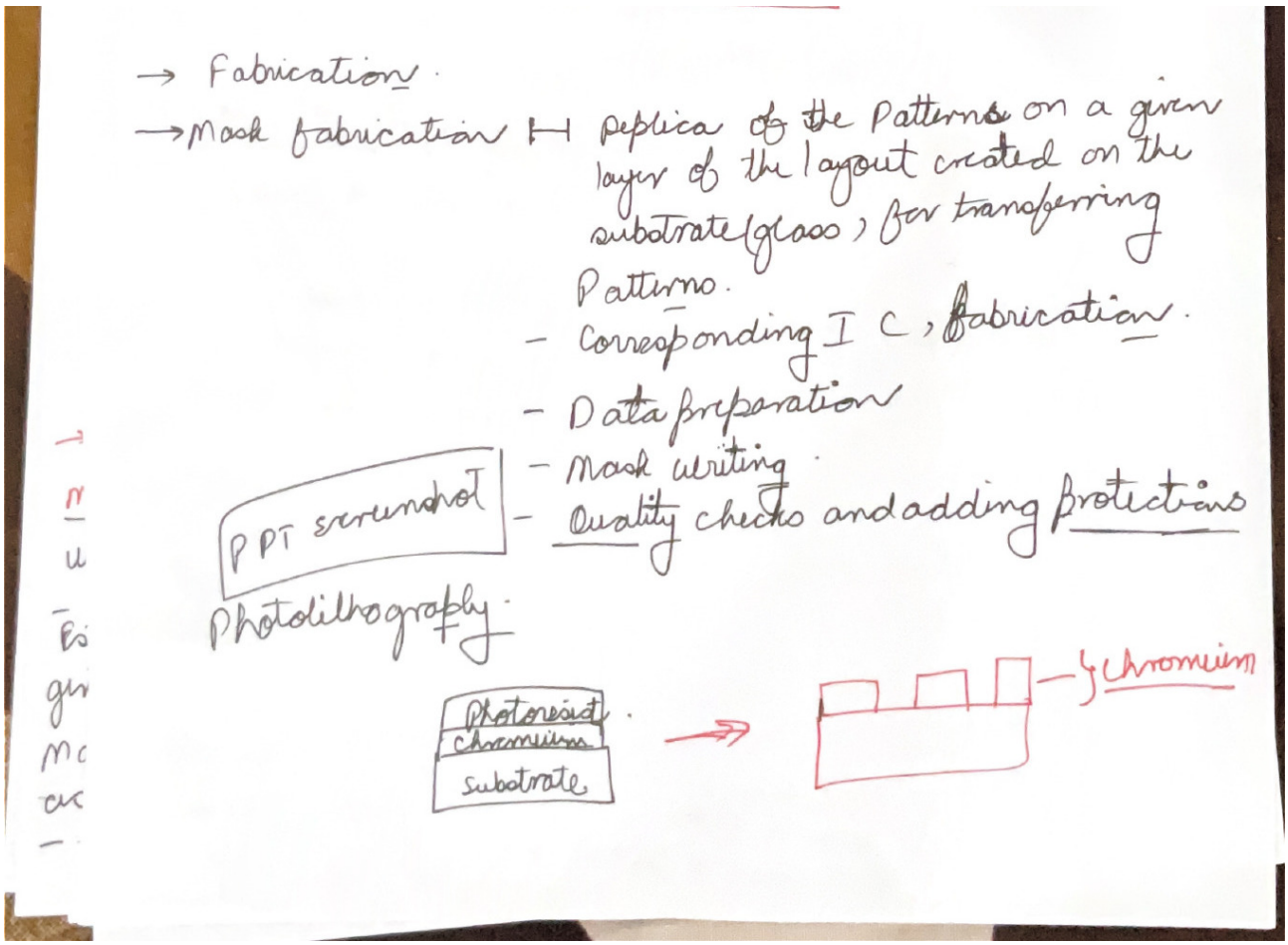


Figure 2.27. *Source: Part 1, PDF page 24, lower portion.*

The chromium-resist-substrate sketch shows the lecture's conventional transmissive-mask example. The mask process itself uses patterning:

1. Begin with a suitable transparent substrate carrying an absorbing chromium layer and resist.
2. A writing system exposes the required pattern using an appropriate laser or electron-beam process.
3. Develop the resist. For the positive-tone example, exposed resist is removed.
4. Etch the exposed chromium while remaining resist protects other regions.
5. Strip residual resist, inspect the mask, and repair qualifying defects where the process permits.
6. Add the appropriate protection, such as a pellicle, and qualify the mask for use.

This manufactures the **mask**, which is subsequently used to expose resist on wafers. Mask writing and wafer lithography are related but separate operations. A **pellicle** is a protective membrane positioned so particles are kept away from the mask's pattern plane; requirements vary with lithography technology. The lecture's chromium-on-glass example should not be generalized to all modern mask stacks.

Resolution enhancement

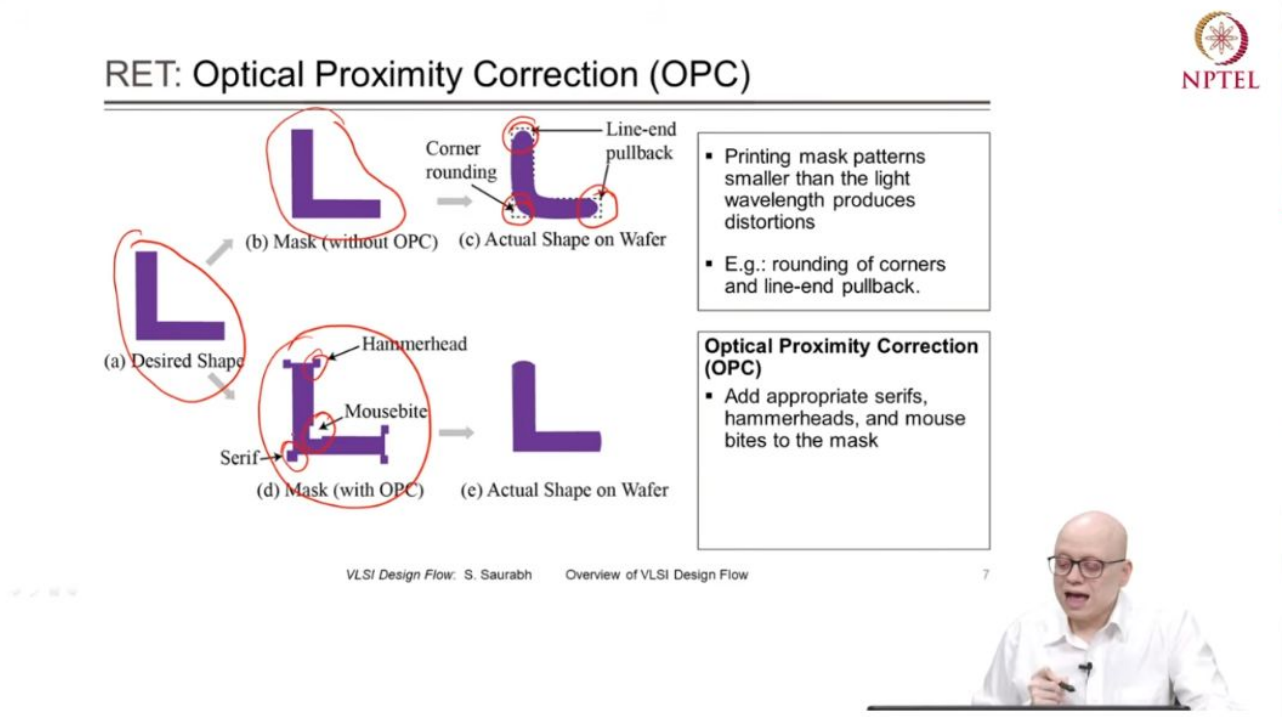


Figure 2.28. *Lecture: 13:33. Closely spaced features motivate resolution-enhancement methods*

Optical proximity correction (OPC) intentionally modifies mask patterns to compensate for predictable printing errors. Without compensation, a desired sharp corner can print rounded and a line end can pull back. Added or reshaped features such as serifs or hammerheads change the optical image so the processed wafer approaches the target geometry. Their shape is a means to produce the desired result, not necessarily a shape intended to appear identically on the final wafer.

The lecture discusses 193 nm deep-ultraviolet lithography. This is one lithography technology; not all lithography uses that wavelength. Resolution depends on wavelength, numerical aperture, illumination, process, and computational enhancement. ASML's lithography explanation provides the physical context.

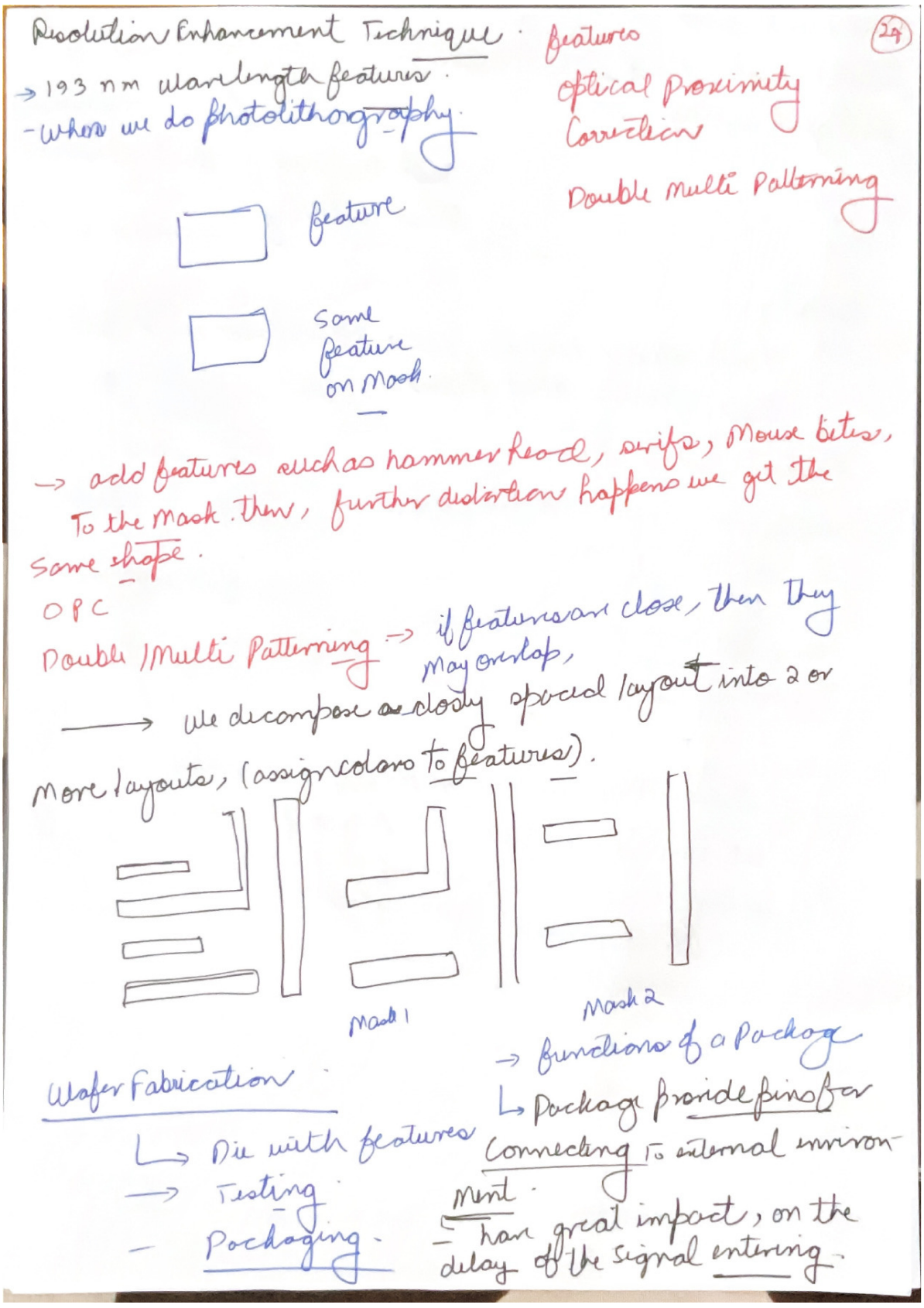


Figure 2.29. *Source: Part 2, PDF page 1; handwritten page 24.*

The L-shaped drawing captures precompensation: a modified mask is chosen because the printing process distorts it toward the intended result. The “hammerhead, serif, mouse bite” list is a set of illustrative correction shapes. Modern OPC is model-based and process-dependent, so these shapes are not manual universal recipes.

Double or multiple patterning separates features into groups that are patterned in separate steps. In the lecture's layout-coloring illustration, neighboring features too close for one exposure receive different colors. Each group has easier spacing within that exposure, while their combined result forms the dense intended pattern. The colors label processing groups; they are not literal colored material on the wafer.

This decomposition introduces extra processing, masks, overlay requirements, and possible coloring conflicts. OPC changes how a pattern prints; pattern decomposition changes which features are formed together. They solve related resolution problems through different mechanisms and can be used together.

Read the geometry: width spacing pitch and overlay

Line width is the distance across a feature. **Spacing** is the edge-to-edge gap between adjacent features. **Pitch** is the distance between corresponding points on repeating features, commonly their centerlines. For equal-width parallel lines with a uniform gap, $p = w + s$. A 20 nm line followed by a 20 nm gap has 40 nm pitch; width and pitch are different measurements.

Consider four ideal line centers at 0, 40, 80, and 120 nm. An illustrative two-mask decomposition assigns alternating lines:

Patterning group	Line-center positions	Pitch within that group
A	0 and 80 nm	80 nm
B	40 and 120 nm	80 nm
Combined target	0, 40, 80, and 120 nm	40 nm

Each group has more generous spacing, while the combined target remains dense. These numbers demonstrate decomposition; they are not a claim about a particular foundry process. Not every multiple-patterning method uses two independent exposures arranged this way.

Overlay error is misregistration between patterns that must align. If group B shifts right by 3 nm while A stays fixed, the combined centers become 0, 43, 80, and 123 nm. Adjacent center distances alternate between 43 and 37 nm. With unchanged 20 nm widths, the gaps become 23 and 17 nm. Thus a placement error can shrink one gap even though each line has the intended width. This is why the handwritten coloring example must be understood together with alignment requirements.

Check the understanding: OPC corrects predictable printing distortion in the pattern; decomposition decides which features belong to each patterning group. Identify which operation is being shown before interpreting a modified corner or a colored line.

Wafer fabrication packaging and screening

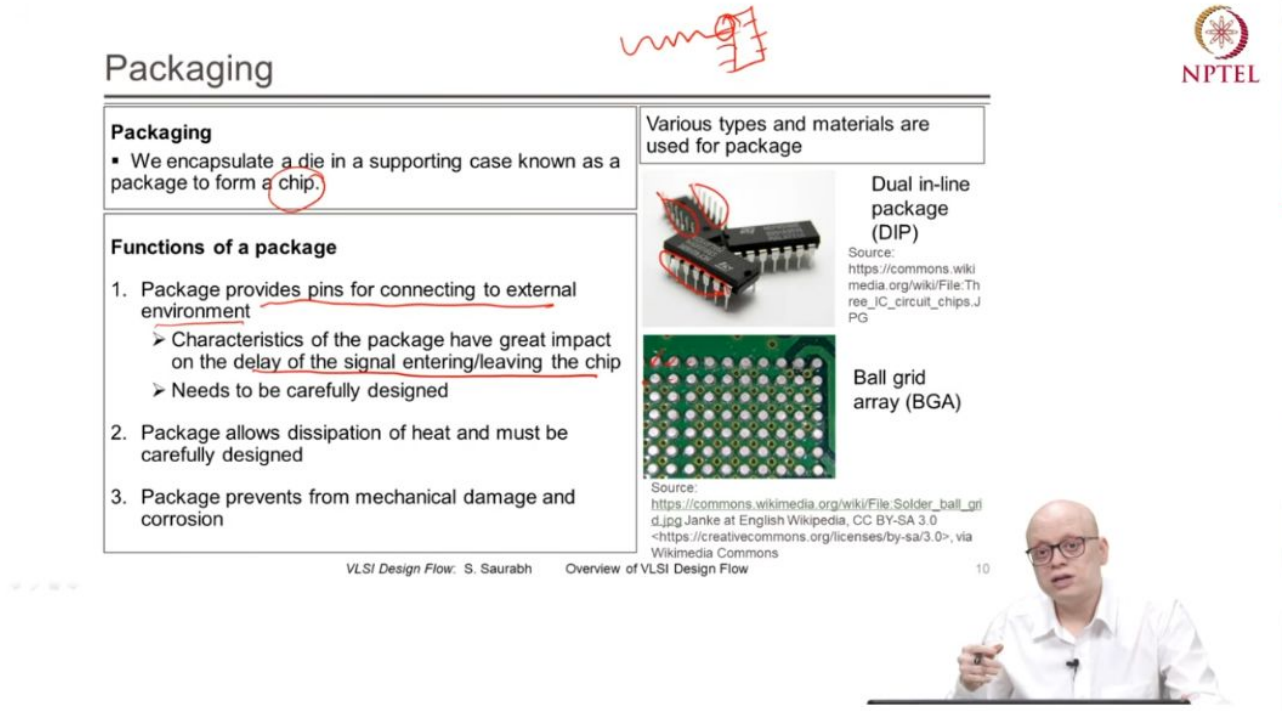


Figure 2.30. *Lecture: 22:57. A package provides electrical, thermal, and mechanical support*

Wafer fabrication repeats many operations, including deposition, oxidation, lithography, etching, implantation, cleaning, and thermal processing. **Front end of line (FEOL)** forms transistor/device structures; **back end of line (BEOL)** forms much of the interconnect above them, with contact-related processing often distinguished as middle of line. The detailed sequence depends on the process.

After fabrication, wafer test identifies acceptable dies, the wafer is diced, and suitable dies proceed to packaging. A **package** provides external electrical connections, mechanical protection, and a thermal path. It also adds parasitic resistance, inductance, and capacitance, so it affects signal and power integrity. A **dual in-line package (DIP)** places leads along two sides; a **ball grid array (BGA)** uses a grid of solder-ball connections. Package choice is an electrical and thermal design decision as well as a mechanical one.

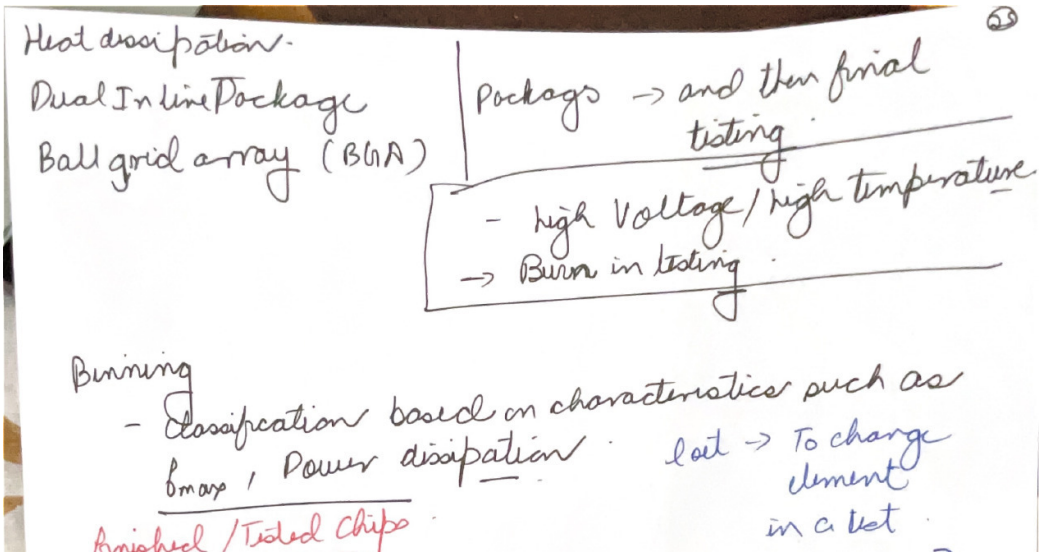


Figure 2.31. *Source: Part 2, PDF page 2, upper portion; handwritten page 25.*

The “heat dissipation” point means that generated heat must leave the die through a designed thermal path. A package may use lids, heat spreaders, or external cooling, depending on the product; not every package includes a separate heatsink. Excess temperature can reduce reliability or trigger failure well before literal silicon melting.

Final test checks the packaged part, including problems introduced or revealed by packaging. **Burn-in** is controlled electrical/thermal stress used where required to expose certain early-life weaknesses. It is an engineered screening procedure with specified limits, not an instruction to apply arbitrary high voltage. The bathtub curve is a conceptual failure-rate model: early failures, a comparatively stable useful-life region, then wear-out. Actual product reliability must be established for the product and use conditions.

Binning sorts manufactured parts by measured characteristics such as supported speed, leakage, power, or functional capability. Two dies made from the same design can have different measured performance because fabrication varies. A lower speed bin can still be a fully conforming product at its own specification. Binning does not repair a faulty design, and price is not determined by frequency alone.

Recall checks: Why can mask geometry differ from desired wafer geometry? How does multiple patterning change the spacing problem? Why is test repeated after packaging?

Definitions and mechanisms in Lesson 09

Manufacturing-data preparation and mask writing. Manufacturing-data preparation converts released layout information into data suitable for the selected patterning process. Fracturing represents shapes in a form supported by the writer, and resolution-enhancement operations may change mask geometry. Mask writing patterns the physical mask blank. Wafer exposure subsequently uses that mask in a different operation. The shared idea is controlled patterning, but the object being manufactured and the optical or writing mechanism are different at the two stages.

Optical proximity correction. OPC deliberately modifies the mask to compensate for predictable differences between the mask pattern and the wafer result. Neighboring features and imaging and process effects influence how a feature prints. A serif or hammerhead is therefore selected for its effect on the eventual printed shape, not because the designer necessarily wants that same shape on silicon. OPC is tied to a process model and target geometry; a correction copied from another context need not be valid.

Multiple patterning and decomposition. Multiple patterning builds a dense target using more than one patterning operation. In the lecture's coloring example, decomposition assigns nearby features to different groups so each group has more manageable spacing. Recombining the groups produces the dense target. This introduces alignment and process constraints as well as extra steps. OPC changes how a selected pattern prints; decomposition changes which features are processed together. They can be applied within the same manufacturing preparation flow.

Width, spacing, pitch and overlay. Width measures a feature across itself; spacing measures the edge-to-edge gap; pitch measures corresponding-point separation in a repeating pattern; overlay measures registration between patterns. For equal-width lines, pitch equals width plus spacing. A shift in one patterning group can reduce one gap and enlarge the next without changing line widths. Keep these quantities distinct when explaining why a mask or manufacturing result meets one geometric requirement but fails another.

Packaging and electrical connection. Packaging connects a die to the surrounding system while supplying mechanical protection and a thermal path. It introduces electrical parasitics and interfaces that influence signals and power delivery. The package is therefore part of the system's electrical and thermal behavior, not simply a container. Wafer testing can identify suitable dies before packaging; final testing checks the packaged product, including effects or defects introduced by those additional operations.

Burn-in and binning. Burn-in uses specified electrical and thermal stress where required to screen certain early-life weaknesses. Binning classifies parts by measured characteristics and product specifications. Neither term means repairing any arbitrary defect. A device assigned to a lower speed bin can meet that bin's specification completely. The selection of stress conditions, test limits and bins belongs to the product and manufacturing plan, so the conceptual bathtub curve cannot by itself determine the correct procedure for a particular part.

Application and validation in Lesson 09

Mask preparation and wafer exposure are interpreted as separate operations. A corrected mask feature is related to the desired wafer shape, the predicted printing error and the purpose of the correction. Pattern decomposition assigns features to processing groups; their combined result also depends on registration.

Overlay tolerance from a spacing requirement. In the 20 nm nominal-gap example, shifting one patterning group by a signed displacement δ creates adjacent gaps $20 + \delta$ and $20 - \delta$ nm. The smallest is $20 - |\delta|$. If the illustrative minimum allowed gap is 18 nm, compliance requires $20 - |\delta| \geq 18$, hence $|\delta| \leq 2$ nm. The 3 nm shift yields gaps of 23 and 17 nm and violates that assumed limit. Linewidth and overlay must be budgeted separately; this calculation holds linewidth fixed and is not a foundry rule.

Complete original notebook pages for Lesson 09

Part 2 original page 01

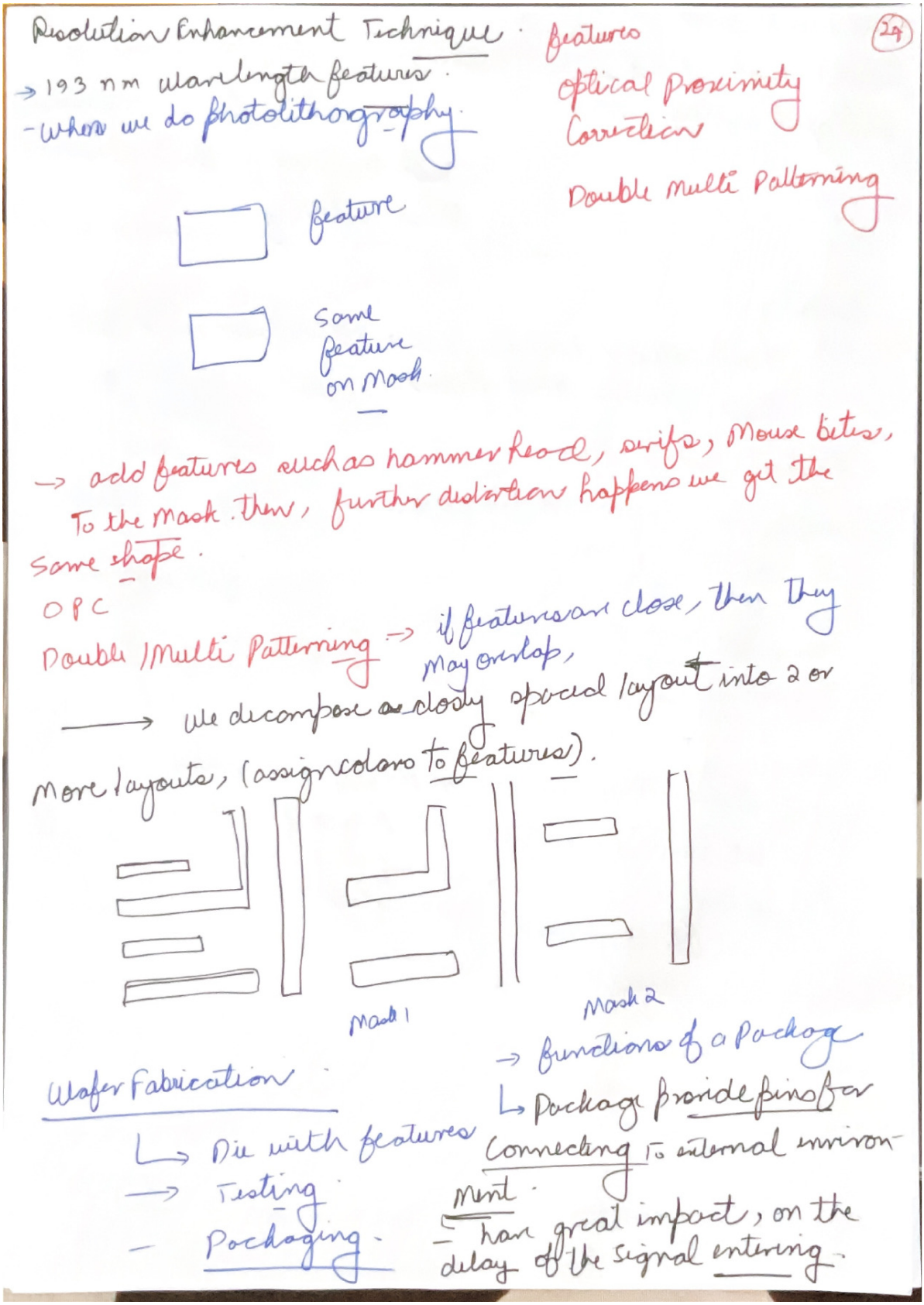


Figure 2.32. *Source: Part 2, PDF page 1. OPC, L-shaped feature, multiple patterning, fabrication/package.*

Explanation and worked examples: Day 2 · Lesson 09.

Part 2 original page 02

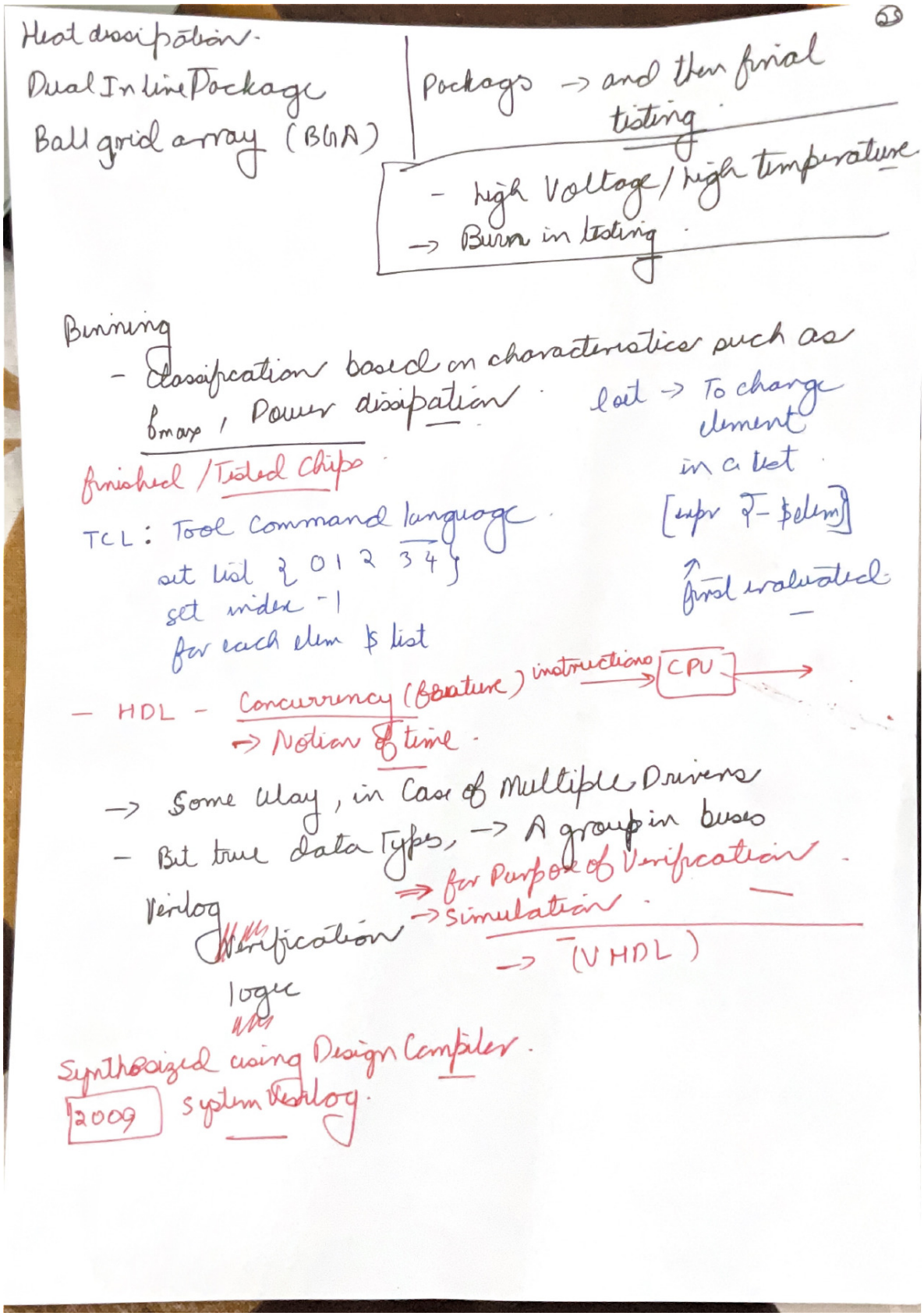


Figure 2.33. *Source: Part 2, PDF page 2. Packages/burn-in/binning; Tcl; HDL features.*

Explanation and worked examples: Day 2 · Lesson 09 · Day 2 · Lesson 10 · Day 2 · Lesson 11.

Lesson 10 Tcl commands and automation

Week 2 · Lecture video

Commands variables and substitution

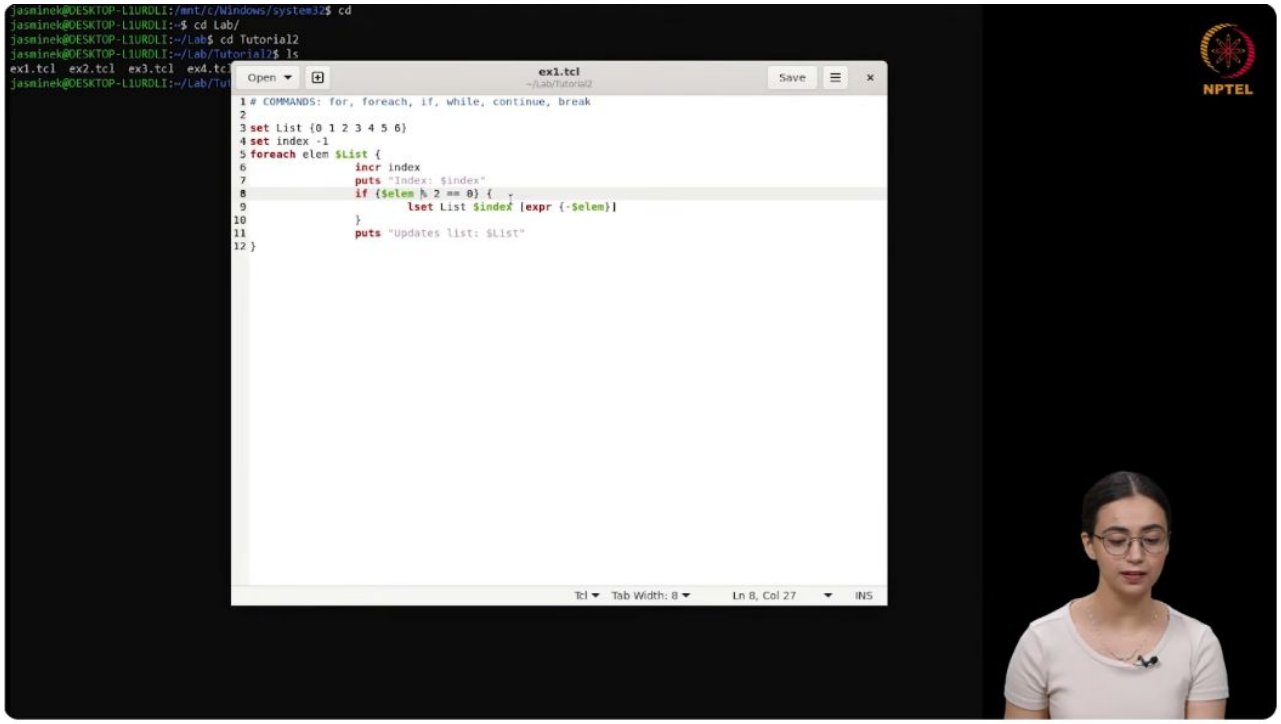


Figure 2.34. *Lecture: 3:11. Iterate over a list and negate its even elements*

Tcl means **Tool Command Language**. The tutorial, presented by Jasmine Kaur, introduces a scripting language widely embedded in EDA tools. Plain Tcl supplies language commands; an EDA application adds commands that manipulate its design database. A command such as `get_cells` is not guaranteed to exist in standalone `tclsh` merely because an EDA tool accepts it.

A Tcl command consists of a command name and words used as arguments. `set index -1` assigns the value `-1` to a variable named `index`. `$index` substitutes its value. Square brackets perform command substitution: `[expr {-$element}]` executes `expr` and substitutes its result. Braces group a word and suppress ordinary substitution at that parsing stage; the command receiving that word may later interpret it as an expression or script. Quotes group words while allowing substitutions.

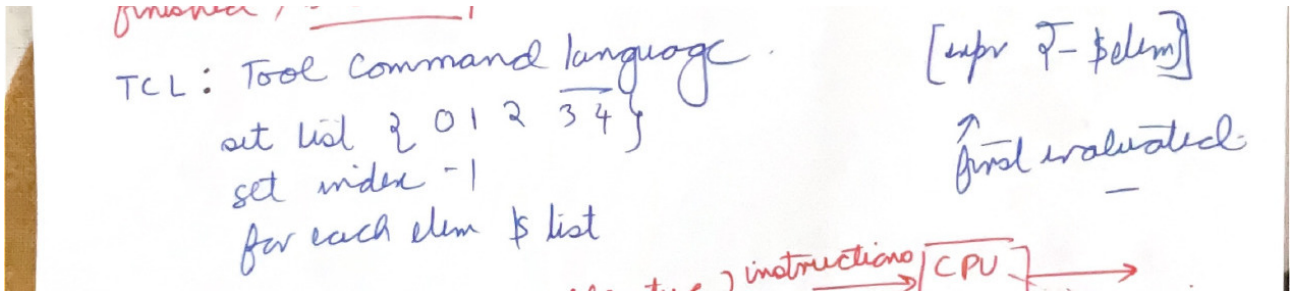


Figure 2.35. *Source: Part 2, PDF page 2, Tcl snippet.*

The “brackets first” note refers to command substitution in a word where substitution is enabled. It is not a rule that every bracket inside every braced string executes immediately. For example, `puts {[expr {1+2}]}` prints the bracketed text literally; `puts [expr {1+2}]` prints `3`. Bracing expressions is

a useful default because the expression evaluator handles their variable substitution and evaluation predictably. Tcl's `expr` manual explains expression evaluation.

This complete version of the lecture's list example uses the same operations:

```tcl
set values {0 1 2 3 4 5 6}
set index -1
foreach element $values {
    incr index
    if {$element % 2 == 0} {
        lset values $index [expr {-$element}]
    }
    puts "index=$index values=$values"
}
```

`foreach` receives the list value at loop entry and assigns each successive element to `element`. `lset` changes the named variable `values` at a zero-based index. `expr` performs arithmetic; `%` gives the integer remainder. The first negation leaves zero unchanged. The final list is `0 1 -2 3 -4 5 -6`. Modifying `values` does not rewrite the iteration list already supplied to this `foreach` invocation.

Preserve argument boundaries in an automation command

A path containing a space is one value even though its printed form contains several words. Tcl lists preserve that boundary. This standalone example constructs a copy command without executing it:

```tcl
set output {results/final netlist.v}
set command [list file copy {netlists/top.v} $output]
llength $command
lindex $command 3
```

The command list has four elements: `file`, `copy`, `netlists/top.v` and the complete output path. The last two calls return `4` and `results/final netlist.v`, respectively. The space inside the final element does not create another argument. `list` applies the grouping needed to retain the supplied values; it does not run the command named by the first element.

Tcl 8.6 argument expansion, written `{*}`, can later pass each list element as a separate command word. This is a different operation from treating an arbitrary constructed string as another script. For example, reparsing an ungrouped string containing the same output path would split it into separate words. The Tcl syntax reference distinguishes argument expansion, substitution and script evaluation. In EDA automation, check the command's actual arguments rather than assuming that a printed line preserves the intended file or object name.

Trace the list instead of memorizing the output

The index starts at -1 so that the first `incr index` makes it zero before `lset` runs. Here is the complete trace, with each list shown after that iteration's conditional update:

Index	Current `element`	Even?	`values` after the update
0	0	Yes; negating zero still gives zero	`0 1 2 3 4 5 6`

Index	Current `element`	Even?	`values` after the update
1	1	No	`0 1 2 3 4 5 6`
2	2	Yes	`0 1 -2 3 4 5 6`
3	3	No	`0 1 -2 3 4 5 6`
4	4	Yes	`0 1 -2 3 -4 5 6`
5	5	No	`0 1 -2 3 -4 5 6`
6	6	Yes	`0 1 -2 3 -4 5 -6`

At index two, `lset values $index [expr {-$element}]` receives the variable name `values`, index `2`, and replacement value `-2`. The first argument deliberately has no `$`: `lset` needs the **name of the variable to update**. Writing `lset $values ...` would substitute the list's contents and use that resulting string as a variable name, which changes the meaning and normally causes an error here. See Tcl's `lset` specification.

In contrast, `foreach element $values ...` needs the **list value to traverse**, so substitution is appropriate there. The loop variable `element` takes successive entries from that supplied list. Updating `values` inside the body does not make the next iteration revisit a modified entry. The `foreach` specification describes how list elements are assigned to loop variables.

Control flow and procedures

`if` selects a branch. `for` has initialization, condition, next-step, and body arguments. `while` repeats while its expression is true. `break` exits the enclosing loop; `continue` skips the remaining body of the current iteration and proceeds with the loop's next iteration. Their Tcl syntax still follows command-and-argument parsing, so braces and spaces have real meaning.

A procedure defines a reusable Tcl command with parameters and a body. The following returns both results as a proper Tcl list:

```tcl
proc sum_product {x y} {
    set sum [expr {$x + $y}]
    set product [expr {$x * $y}]
    return [list $sum $product]
}
puts [sum_product 10 50]
```

The output is `60 500`. `return` ends this procedure invocation and provides its result; subsequent statements in that invocation do not run. `puts` prints a value, while returning a value makes it available to the caller. A procedure that prints results is therefore different from one that returns them for another calculation. The lecture demonstrates both printing and early return; this version makes the returned data explicit.

File channels and external commands

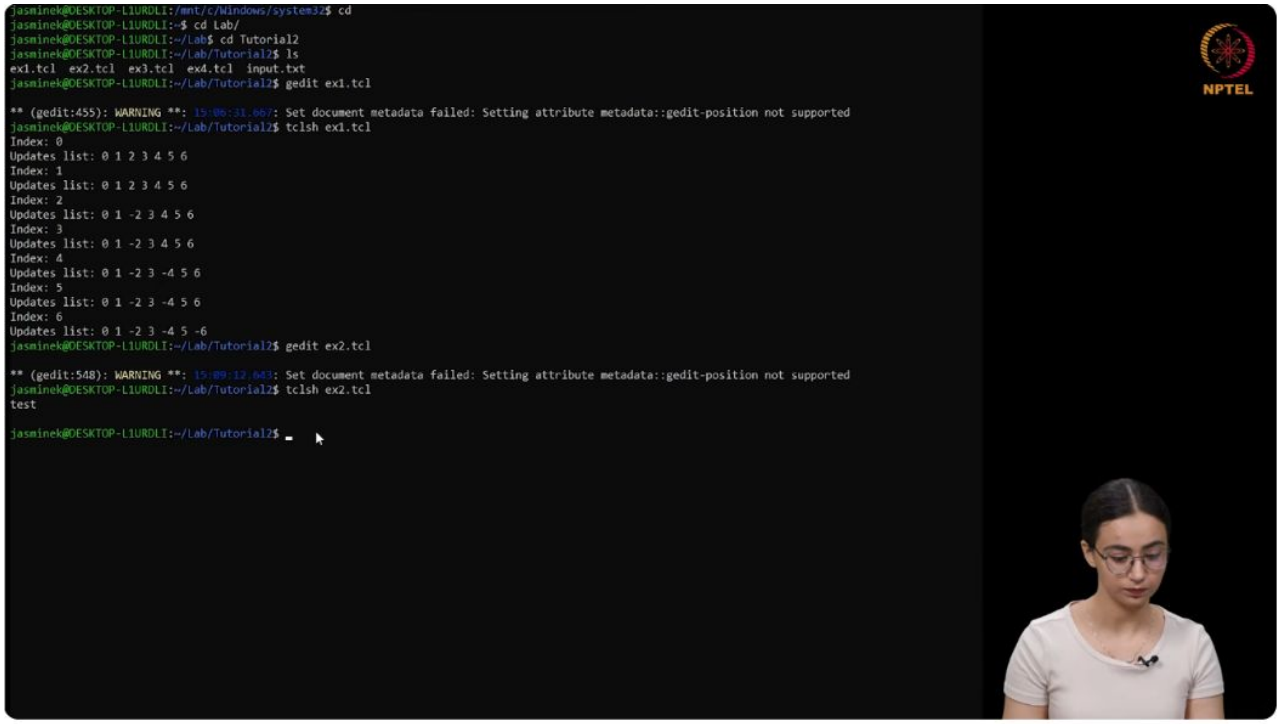


Figure 2.36. *Lecture: 7:13. The file-I/O example prints the text read back from its file*

An **open channel** is a handle for I/O. `open` returns a channel identifier; storing it in `fp` lets later commands use `$fp`. Closing a channel releases it and flushes the appropriate buffered output. The tutorial's `w+` mode permits reading and writing and truncates an existing file. Use that mode only for a file whose replacement is intended.

This example should be run in a disposable practice directory because it writes its named demonstration file:

```tcl
set fp [open "tcl_demo_output.txt" w+]
puts $fp "test"
close $fp

set fp [open "tcl_demo_output.txt" r]
set file_data [read $fp]
close $fp
puts -nonewline $file_data
```

The final terminal output is `test` followed by the newline stored in the file. `read` obtains content from the channel's current position. For line-by-line reading, `while {[gets $fp line] >= 0} { ... }` is a robust pattern; `eof` reports channel state after attempted reading, so careless end-of-file loops can mishandle the final read. In production scripts, arrange cleanup even when an intermediate command fails.

`exec` invokes an external program, for example `puts [exec ls]` on a system with `ls`. It does not turn Tcl into a Unix shell: shell syntax and built-ins are not automatically interpreted as they would be by Bash. Tcl already has a native `pwd` command and file commands, which can avoid platform-specific external tools. Save scripts conventionally with `.tcl` and run `tclsh script.tcl`, or use the relevant

EDA tool's script mechanism. The extension is a convention, not a magical property that changes the language.

Practice: change the list operation to square odd values. Trace the original value of `element`, the current `index`, and the updated list separately. Then modify `sum_product` so the caller selects either result using `lindex`.

The repository contains complete runnable Tcl examples.

Definitions and mechanisms in Lesson 10

Interpreter, command and argument. A Tcl interpreter parses command words and invokes the command named by the first word. The remaining words are arguments whose meaning depends on that command. Spaces and grouping delimit those words. An EDA tool extends Tcl with application-specific commands and a design database; standalone Tcl does not acquire those commands from the script filename. Identify the interpreter and available commands before interpreting or debugging an automation script.

Substitution and grouping. Substitution replaces syntax with values or command results when the current parsing context permits it. A dollar sign introduces variable substitution, and square brackets introduce command substitution. Braces group one word while suppressing ordinary substitution at that stage; quotes group while permitting substitution. A command such as expr can subsequently evaluate the grouped text according to its own rules. The sequence of interpretation explains the result more accurately than a blanket instruction to evaluate all brackets first.

Variable name and variable value. A command that updates a variable often needs its name; a command that consumes data needs its value. In lset values, the word values names the variable to modify. In foreach element followed by the substituted values list, the command receives the data to traverse. Adding or omitting a dollar sign changes the argument, not just the notation. State what each command expects before deciding whether substitution belongs there.

List value and iteration snapshot. A Tcl list is a structured value containing elements that can be indexed, traversed and updated using list commands. The foreach call receives its list value at invocation and assigns successive elements to the loop variable. Updating the source variable during the body does not retroactively replace the list already supplied to that invocation. In the lesson's trace, element is the current item from the original list, while values is the progressively edited list. They play different roles despite starting from the same data.

Procedure result and printed output. A procedure is a reusable Tcl command with parameters and a body. Its result is the value supplied back to its caller. Printed output is text written to a channel, often the terminal. A caller can use a returned list in another computation; text printed by puts does not automatically become that procedure's returned data. The sum_product example deliberately returns a proper list so either result can be selected reliably with list operations.

File channel and position. Opening a file returns a channel identifier representing the I/O connection, not the file's contents. Reads and writes operate through that channel and its current position and mode. Closing the channel releases resources and completes appropriate buffered output. A mode that truncates an existing file changes stored data as soon as that open succeeds. Explain the mode and lifecycle when describing a file-I/O script, because a correct arithmetic expression does not make the surrounding file operations safe or correct.

Application and validation in Lesson 10

Tcl evaluation is traced in two stages: the parser determines command words and performs permitted substitutions; the invoked command interprets those resulting words. A procedure returns structured data to its caller, whereas `puts` writes text to a channel. The list example distinguishes the iteration values from the variable changed by `lset`.

Substitution is not recursive script evaluation. After `set word {[expr {1+2}]}`, `puts $word` prints the stored bracket expression literally. Variable substitution inserts the value as data; it does not restart command substitution on the newly inserted text. By contrast, `puts [expr {1+2}]` contains brackets in the original command and evaluates them, printing 3. Braces suppress the first-stage substitutions, while `expr` can perform its own expression evaluation when invoked. A script's behavior follows these staged rules, not a global instruction to execute every visible bracket.

Source: Tcl syntax and substitution rules.

Lesson 11 Verilog values types and hardware modeling

Week 3 · Lecture video

What a hardware description language must represent

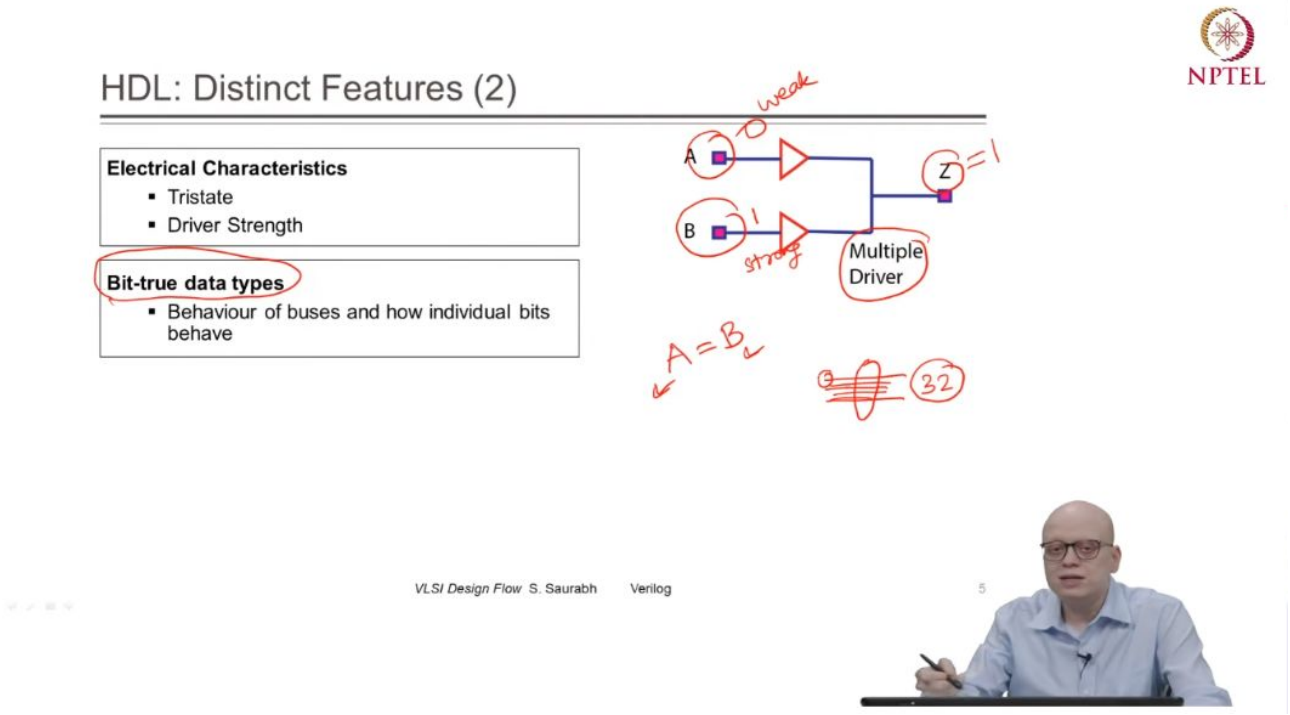


Figure 2.37. *Lecture: 9:14. Bit-accurate values and resolved drivers are distinctive HDL features*

An **HDL (hardware description language)** describes the behavior and structure of electronic hardware. Verilog lets us express modules, wires, stored state, and the changes that occur when inputs or clocks change. A simulator interprets these descriptions to predict modeled behavior; a synthesis tool interprets a supported subset to construct a circuit. These are different uses of the same source text, so a statement that simulates successfully is not automatically synthesizable.

Hardware needs **concurrency**: two adders can respond to their inputs at the same time. It needs **time**: a register reacts to a clock event, and physical gates have propagation delay. It needs

multiple-driver resolution: two connected outputs might agree, conflict, or release a line. It needs **bit-accurate modeling**: an eight-bit result cannot retain a ninth carry bit unless the description provides a place for it. A general software integer alone does not express all these properties.

RTL describes state held in registers and the combinational transformations between them. It does not give every transistor's physical layout. Synthesis and physical design progressively supply those details. A delay written as `#10` in a testbench advances simulated time; it does not order the ASIC tool to manufacture a gate with exactly that delay.

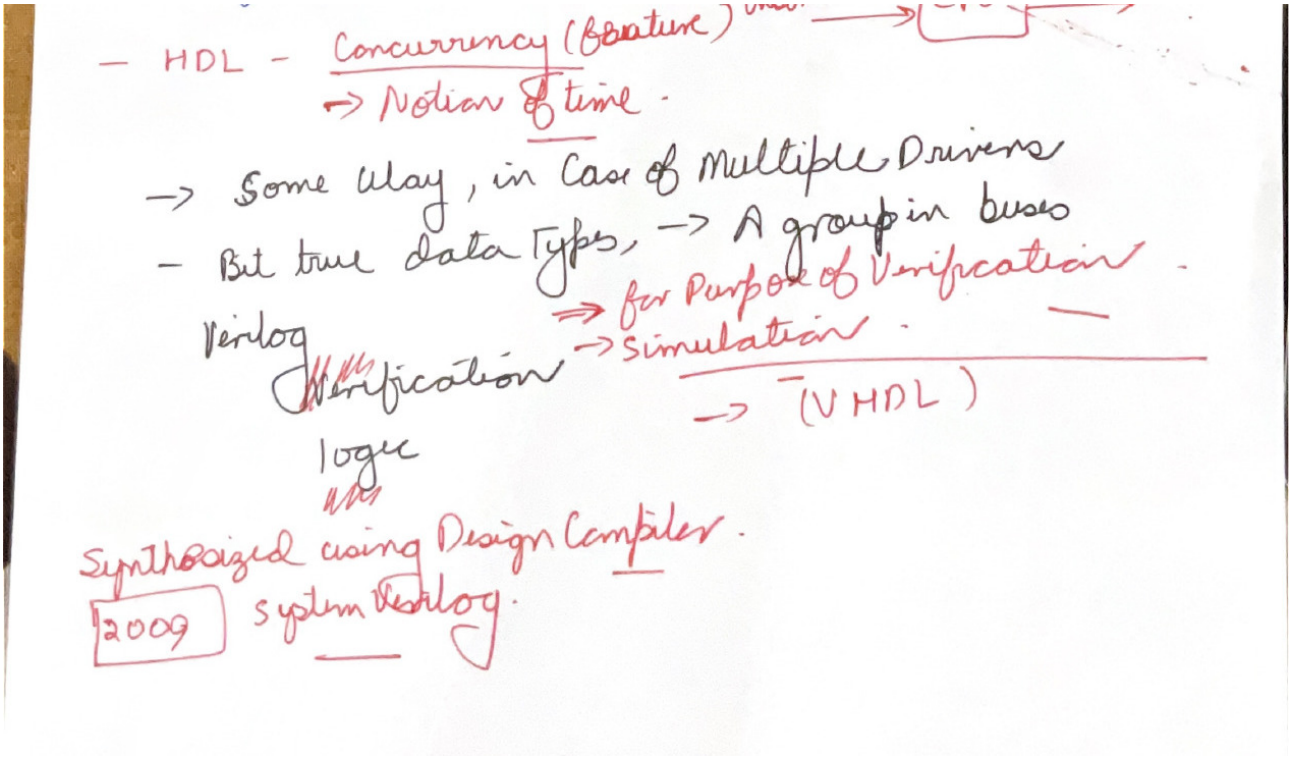


Figure 2.38. *Source: Part 2, PDF page 2, lower HDL section.*

The examples of multiple drivers and bit-true behavior are the key motivations for an HDL. Distinguish **parallel hardware** from the order of statements inside one process: different `always` blocks are concurrent, while statements within a `begin ... end` block execute in their language-defined order. That order alone does not say how many clock cycles the resulting hardware needs.

Verilog and SystemVerilog are related languages. This pair of lessons uses traditional Verilog terminology such as `wire` and `reg`. Keep those meanings clear before introducing SystemVerilog's additional types and verification features. An IEEE revision date is a language-standard milestone, not the birth date of every feature mentioned alongside it.

Lexical rules and four-state values

Verilog is **case-sensitive**: `data`, `Data`, and `DATA` are different identifiers. Keywords such as `module` and `always` are lowercase. A simple identifier begins with a letter or underscore and can subsequently contain letters, digits, underscores, and dollar signs. An escaped identifier starts with a backslash and ends at whitespace; it is useful for generated names but makes hand-written RTL harder to read. Use descriptive simple names where possible. `//` introduces a line comment; `/* ... */` encloses a block comment.

The four logic states are:

State	Meaning in simulation	Typical reason
`0`	Known logic low	A driver produces zero
`1`	Known logic high	A driver produces one
`x`	Unknown value	Uninitialized storage, conflicting drivers, or an operation with insufficiently known inputs
`z`	High impedance	A driver has released a net, or a net has no active driver

`x` does not mean that a physical circuit has a stable third digital voltage. It records uncertainty in the model. A real uninitialized flip-flop may settle to either zero or one; simulation uses `x` because the description does not establish which. Similarly, `z` describes the absence of an active drive, not a guaranteed measured voltage. Pull devices, capacitance, leakage, and other connected drivers determine what an actual released node does.

With ordinary equal-strength drivers, `0` against `1` resolves to `x`; an active `1` against a released `z` resolves to `1`. Drive strengths permit more detailed resolution, so the "conflicting drivers give `x`" rule assumes neither driver dominates. In ordinary RTL, avoid multiple procedural drivers for the same state variable.

Why an unknown input does not always make the output unknown

For a one-bit Boolean operation, ask whether both possible known values of an unknown input give the same result. If they do, that result is already determined:

Verilog expression	Result	Reason
`1'b0 & 1'bx`	`1'b0`	Zero AND either zero or one is zero
`1'b1 & 1'bx`	`1'bx`	The result depends on the unknown input
`1'b1 | 1'bx`	`1'b1`	One OR either zero or one is one
`1'b0 | 1'bx`	`1'bx`	The result depends on the unknown input
`1'bx ^ 1'bx`	`1'bx`	The simulator does not establish a known Boolean relationship between the operands

Zero is the **controlling value** for AND; one is the controlling value for OR. A controlling value determines the output regardless of the other input. For these bitwise operators, `z` is also treated as an unknown operand, so `1'b0 & 1'bz` produces zero. This differs from **net resolution**, where an active driver can determine the value of a net whose other driver is released.

The last row shows a limit of four-state simulation: an unknown value is not a symbolic variable whose relationships are tracked algebraically. Even `a ^ a` can evaluate to `x` when `a` contains `x`, although the corresponding ideal Boolean identity is zero. Therefore an `x` waveform requires

examination of initialization, drivers, and operator semantics. It is not an instruction to treat that bit as a freely chosen don't-care.

Sized literals padding truncation and signed values

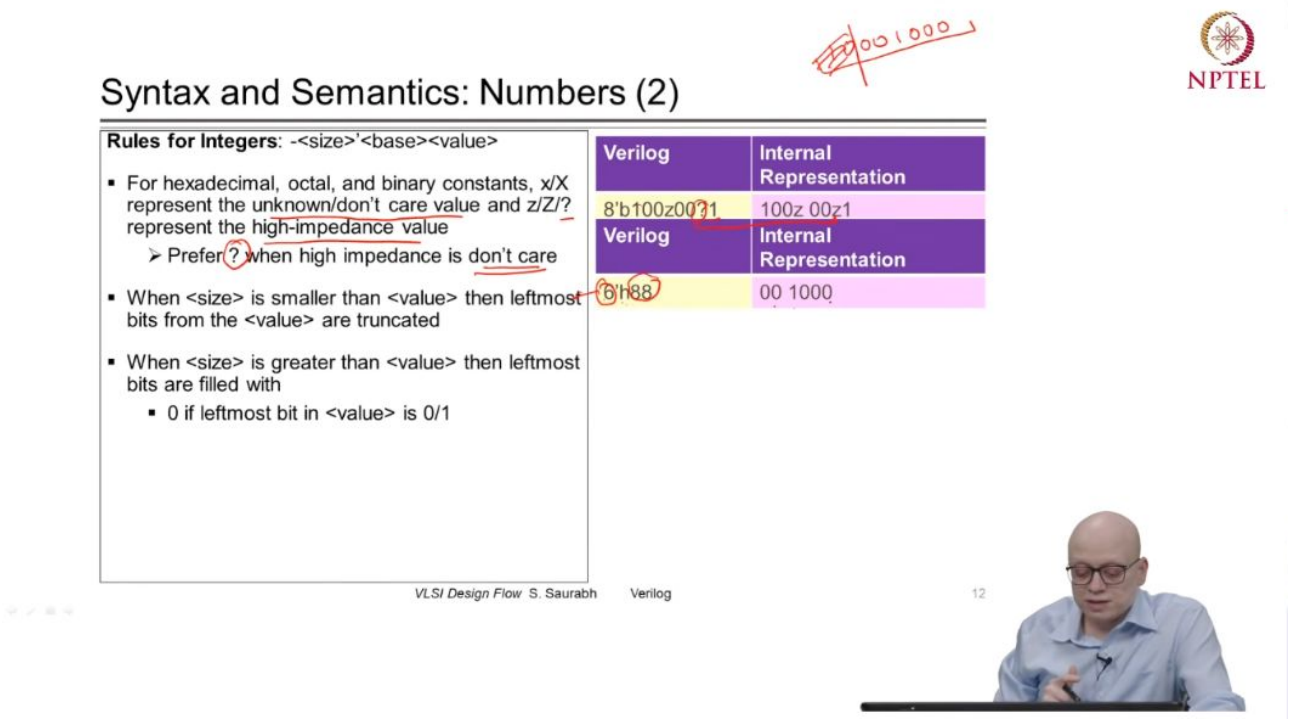


Figure 2.39. *Lecture: 33:10. Sized constants retain the declared number of bits*

A based integer literal has the form **size, apostrophe, optional signed marker, base, digits**, for example `8'hA1` or `8'shFA`. The size is a number of **bits**, independent of the chosen base. Binary uses one bit per digit, octal three, and hexadecimal four. Underscores improve readability without changing the value.

Literal or assignment	Stored bit pattern	Reason
`1'b1`	`1`	One binary bit
`8'hA1`	`10100001`	Two hexadecimal digits in eight bits
`6'o71`	`111001`	Two octal digits in six bits
`6'h88`	`001000`	`10001000` loses its two most significant bits
`8'b11`	`00000011`	Ordinary positive digits are padded on the left with zeros
`8'bz1`	`zzzzzzz1`	A leading high-impedance digit extends the unspecified upper positions

Literal or assignment	Stored bit pattern	Reason
`8'bx01`	`xxxxxx01`	A leading unknown digit similarly extends with `x`
Eight-bit assignment of `-6`	`11111010`	Two's-complement encoding modulo 256

For the last row, start with six as `00000110`, invert to `11111001`, then add one to obtain `11111010`. The **same bits** represent unsigned 250 or signed -6 depending on the declared type and expression context. A minus sign is a unary operator, not an extra digit inside a based literal. Writing `8'sd6` gives a signed positive six; `-8'sd6` negates it.

Expression width matters before assignment. An eight-bit destination cannot recover information that was already discarded by a narrower intermediate operation. To calculate an unsigned eight-bit addition with its carry, explicitly widen both operands: `{1'b0, a} + {1'b0, b}` into a nine-bit destination. Unsized decimal constants are signed and at least 32 bits; mixing them with unsigned vectors can change extension and interpretation. Explicit widths and explicit intent make a design easier to review.

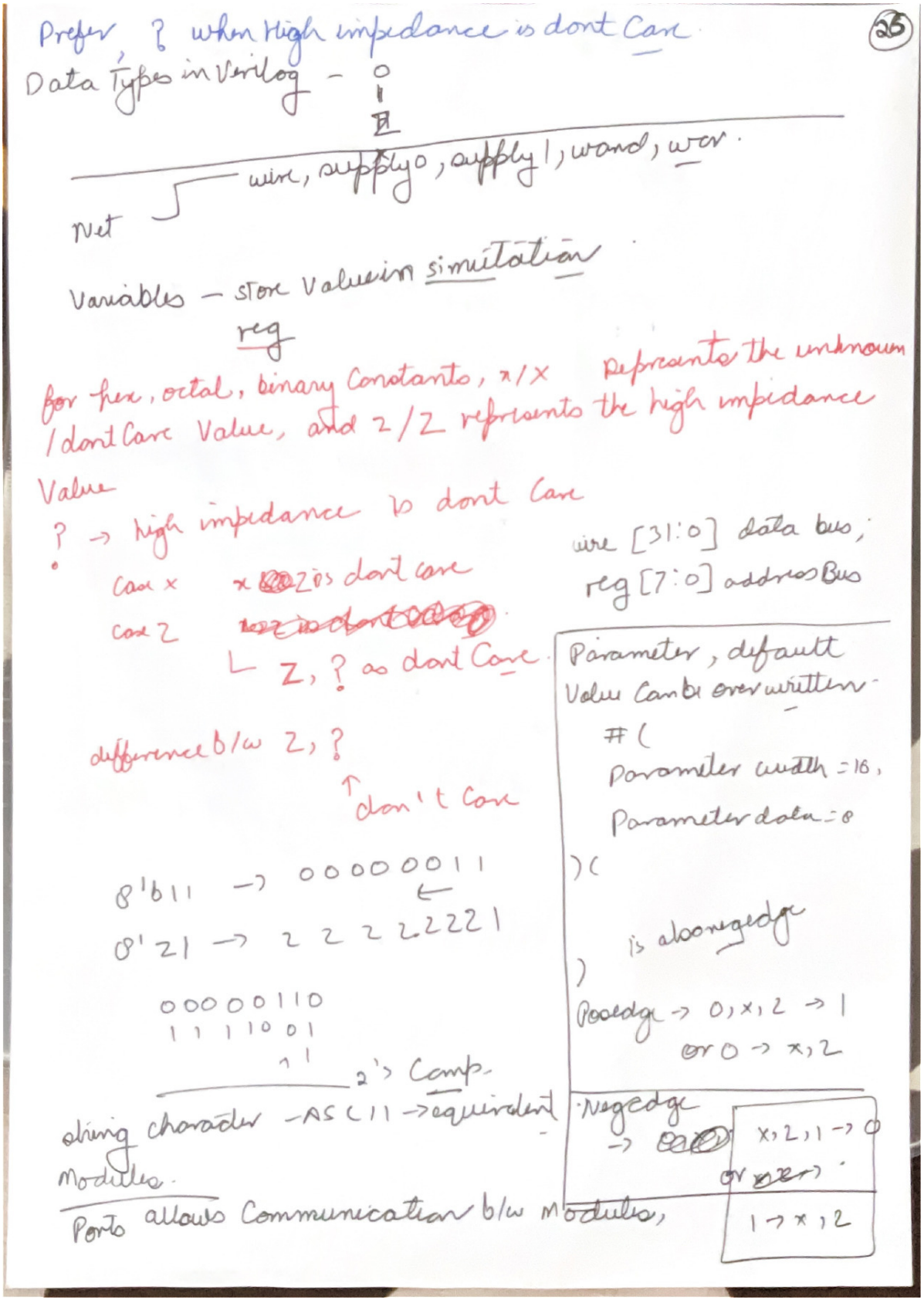


Figure 2.40. *Source: Part 2, PDF page 3, complete values and data-types page.*

The worked truncation example is a **least-significant-bit retention** operation. A declared width is not a request to round a number. The negative-number example uses two's complement correctly once the width is fixed; do not attach a unique decimal meaning to a bit string without also specifying signedness.

What is the difference between z and question mark?

Inside a Verilog **based number literal**, `?` is an alternative spelling of `z`. Thus `4'b10?1` and `4'b10z1` encode the same four-state value. `?` is not a fifth logic state and is not automatically a wildcard everywhere. Separately, the punctuation in `condition ? true_value : false_value` belongs to the conditional operator.

The note “prefer `?` when high impedance is don't care” concerns readability in wildcard **case patterns**. A pattern such as `3'b1??` visually communicates ignored positions, while `3'b1zz` can look like an intentional electrical high-impedance value. The surrounding construct supplies the wildcard behavior:

Construct	Matching rule
`case`	Exact four-state matching, including `x` and `z`
`casez`	`z`/`?` positions in either selector or item are ignored
`casex`	`x`, `z`, and `?` positions in either selector or item are ignored

This rule also affects the **selector**, which is easy to overlook: an unintended `z` in a selector can make `casez` match more broadly than expected. `casex` can hide uninitialized `x` values in control logic. Use intentional patterns and a defined default; do not use wildcard matching to conceal an unknown state. The author's Verilog-2001 reference, decision statements confirms these matching rules.

For equality, `==` can return `x` when unknown or high-impedance bits make the comparison indeterminate. Case equality `===` compares all four states and produces a known Boolean result; it is especially useful in testbenches. It does not magically make an unknown physical signal safe. For example, `1'bx == 1'bx` gives `x`, while `1'bx === 1'bx` gives `1`.

Nets variables vectors arrays and strings

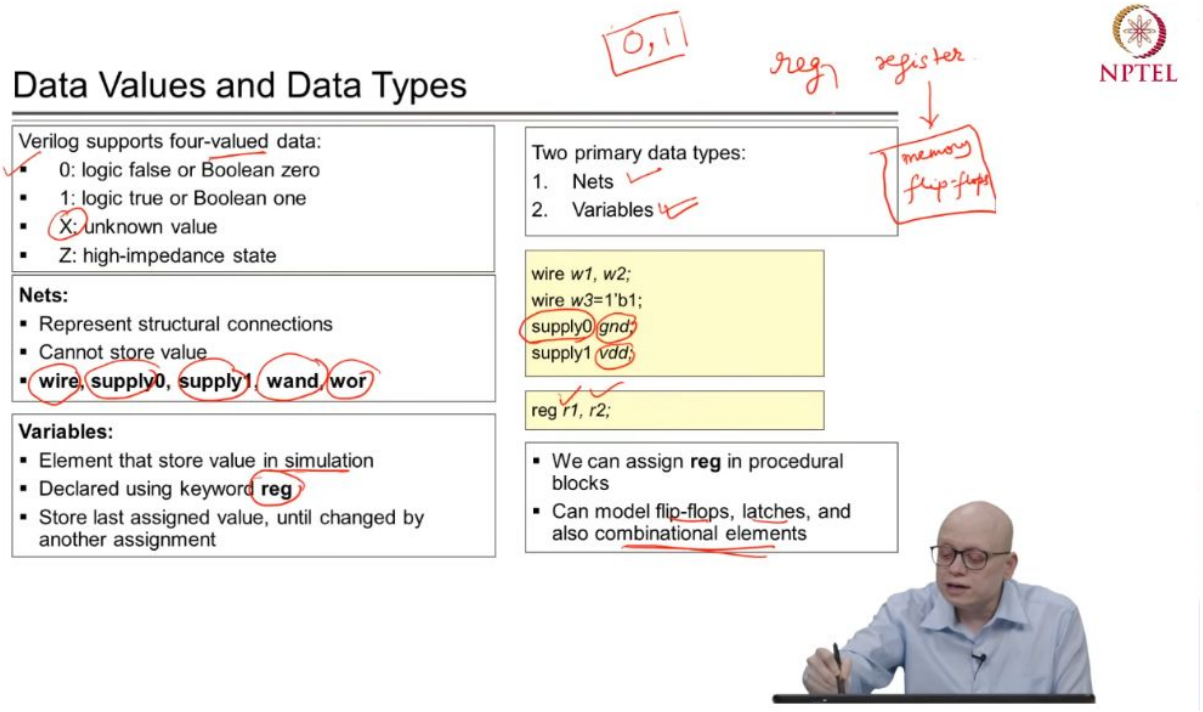


Figure 2.41. *Lecture: 42:14. Net and variable types serve different modeling roles*

A **net** models a connection and takes its value from its drivers. `wire` is the common net type. An undriven ordinary wire reads `z`. Other net types express special resolution or supply behavior: `wand` models wired-AND resolution, `wor` wired-OR resolution, and `supply0`/`supply1` constant supplies. These modeling facilities do not imply that arbitrary internal tri-state or wired logic is supported by every synthesis target.

A **variable** stores its most recently assigned simulation value until another procedural assignment updates it. Traditional Verilog's `reg` is a four-state variable type. Its name does **not** guarantee a hardware register. `reg` assigned in a complete combinational process can represent combinational logic; assigned only on a clock edge, it can describe flip-flop state. An incomplete combinational assignment can infer a latch. The surrounding process determines the inferred storage.

This complete module deliberately uses both forms of combinational description:

```
Verilog
module mux_forms (
    input  wire       sel,
    input  wire [7:0] a, b,
    output wire [7:0] y_wire,
    output reg  [7:0] y_reg
);
    assign y_wire = sel ? b : a;
    always @* begin
        if (sel) y_reg = b;
        else     y_reg = a;
    end
endmodule
```

For known `sel`, the two outputs agree and describe multiplexers, without a clock or a hardware register. With `sel=x`, the conditional operator can merge equal bits from its alternatives, while procedural `if` takes its else branch when the condition is not true. That is a simulation distinction worth understanding; ordinary operation should provide a known control signal. Lesson 12 works through the conditional operator.

`wire [7:0] bus` declares one eight-bit **vector**; `bus[3]` selects a bit and `bus[7:4]` selects four bits. `reg [7:0] memory [0:15]` declares an **array** containing sixteen eight-bit words; `memory[2]` selects a word. A range before a name describes the vector bits, while the array dimension follows the name in this traditional syntax. `[0:7]` is also legal but reverses the index direction; use a consistent convention and never assume that index zero is always the least significant bit.

Traditional types also include `integer` for a signed 32-bit variable, `time` for a 64-bit unsigned time value, and `real` for floating-point simulation values. Declaring `real` does not synthesize a floating-point arithmetic unit. Actual floating-point hardware requires a suitable synthesizable architecture or IP and an explicit representation.

A traditional Verilog string literal packs character codes, eight bits per character, into a vector context. `reg [39:0] text;` can hold five characters such as `"HELLO"`. A destination that is too small truncates the most significant portion, so choose the width deliberately. Strings used for `$display` messages are testbench text, not automatically a hardware text-storage subsystem.

The runnable language examples exercise these values and distinguish variable type from inferred storage. The parameter and edge-event notes at the right of the handwritten page continue in Lesson 12.

Recall checks

1. Why can an undriven wire read `z` while an uninitialized `reg` reads `x`? A net reports its resolved drive; an uninitialized variable has no known stored value.
2. Does `6'h88` equal hexadecimal 88? No: six bits retain only `001000`, decimal eight.
3. Does `reg` imply a flip-flop? No: a clocked assignment can infer a flip-flop; a complete combinational process need not infer storage.
4. Is `?` always a don't-care? No: in a based literal it encodes `z`; wildcard case matching gives that position its don't-care interpretation.
5. Why provide a ninth bit for adding two eight-bit unsigned inputs? The maximum sum is 510, which needs nine bits.

Definitions and mechanisms in Lesson 11

Hardware description and synthesizable subset. An HDL describes concurrent structure and behavior, including bit widths, connections, state and event-driven activity. A simulator executes its modeling semantics. Synthesis interprets supported constructs as hardware. The two tools use the language for different purposes, so code that prints a message or waits for an arbitrary simulation delay need not describe manufacturable logic. Explain what the construct means in simulation and what hardware, if any, the chosen synthesis flow can infer.

Four-state value and physical interpretation. Verilog's four states represent known low, known high, unknown and high impedance. Unknown records insufficiently established information in the model; high impedance records a released drive contribution or an undriven ordinary net. These are not two extra stable digital voltages. Operator evaluation and net resolution use the states differently: zero AND unknown is known zero, while an active driver against a released driver can determine a net's value. Operator evaluation and driver resolution must be distinguished.

Net and procedural variable. A net derives its value from drivers and their resolution. A procedural variable retains its assigned simulation value until another procedural assignment updates it. In traditional Verilog, wire names a common net type and reg names a four-state variable type. The latter does not by itself imply a flip-flop. A complete combinational process can assign a reg without storage; an edge-triggered process describes state; incomplete combinational assignments can require retention and infer a latch.

Bit width and signedness. Width determines how many bits are represented; signedness determines how the pattern participates in numeric interpretation and operations. At eight bits, the pattern 11111010 can represent unsigned 250 or signed negative six. Declaring a six-bit hexadecimal literal retains only six bits even if the written digits describe more. Overflow, extension and expression sizing must be handled deliberately. A wide destination is useful only if the expression rules and operand widths preserve the information being assigned.

Vector and array. A vector groups bits into one multi-bit value. An array groups multiple elements, each of which can itself be a vector. In the traditional declaration with an eight-bit range before memory and a sixteen-element range after it, memory contains sixteen eight-bit words. A bit select, part select and array index identify different objects. The direction of a declared range also affects significance, so a numeric index alone does not identify which end is the least significant.

Wildcard matching and equality. A question mark inside a based literal encodes z. A surrounding casez or casex construct supplies the wildcard interpretation; ordinary case performs exact four-state matching. Logical equality can return unknown when the result is indeterminate, while case equality explicitly compares four-state values and returns a known Boolean result. Those distinctions matter in debugging because an overly permissive match can conceal uninitialized or disconnected control signals rather than fixing them.

Application and validation in Lesson 11

A Verilog declaration establishes object kind, width, index direction and signedness. Its drivers and procedural context establish behavior. Unknown values are traced through initialization, driver resolution and operators; controlling binary inputs can resolve an operator result even when another input is unknown.

Width required by a sum. Two unsigned 8-bit inputs can each reach 255, giving maximum sum 510. Eight result bits represent at most 255, while nine represent 511, so a full sum requires nine bits. For $255 + 1$, truncation to eight bits produces zero and discards the carry; a nine-bit result represents 256. Explicitly extending operands, as in `{1'b0,a}+{1'b0,b}`, makes the intended precision visible before addition. Width checking must include intermediate expressions, not only the final destination. This numeric argument is independent of whether synthesis later implements a ripple or another adder structure.

Complete original notebook pages for Lesson 11

Part 2 original page 03

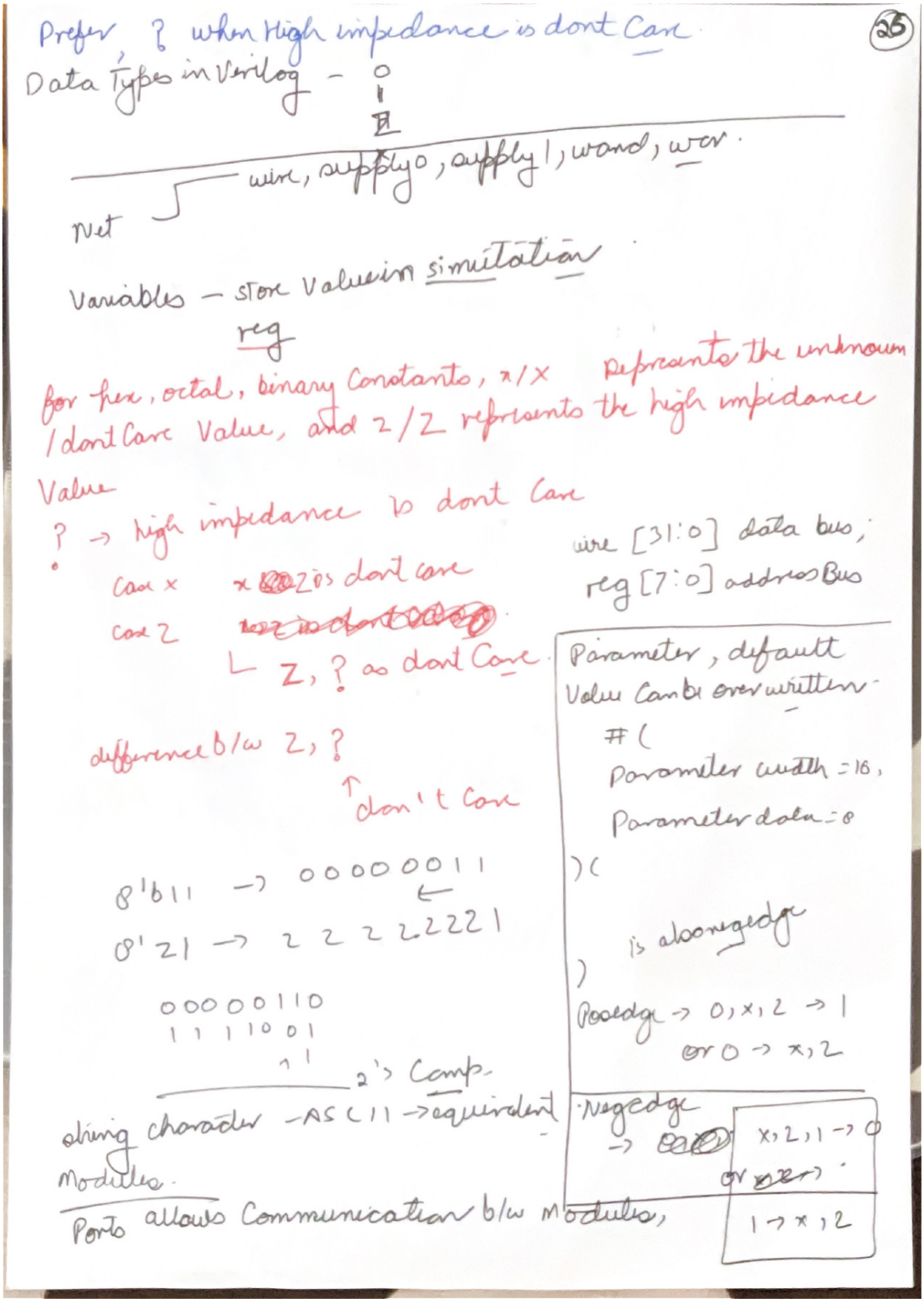


Figure 2.42. *Source: Part 2, PDF page 3. Values, literal widths, types, z/?/cases; parameters and edges.*

Explanation and worked examples: Day 2 · Lesson 11 · Day 2 · Lesson 12.

Lesson 12 Verilog structure events and assignments

Week 3 · Lecture video

Modules ports hierarchy and parameters

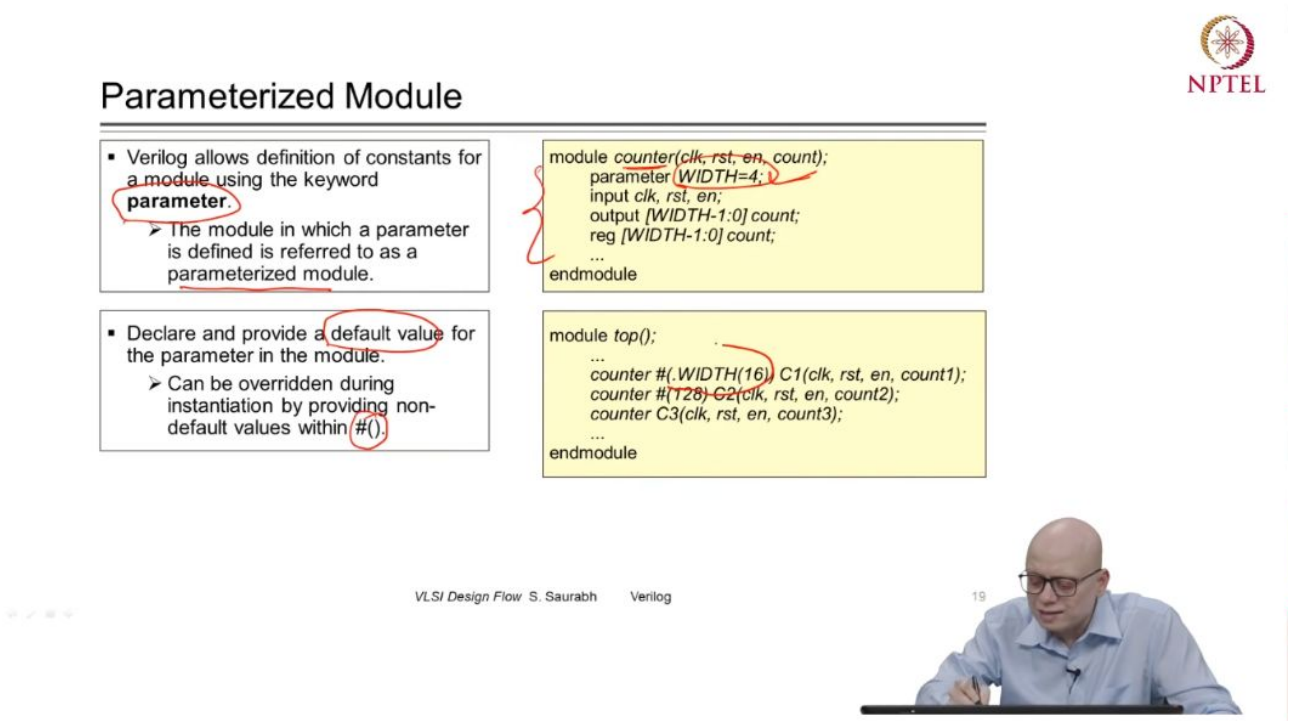


Figure 2.43. *Lecture: 8:41. A module can be reused with different elaboration-time parameters*

A **module** is a named hardware description with an interface and an implementation. Its ports connect it to a surrounding module: `input` receives a value, `output` drives outward, and `inout` represents a bidirectional connection. Instantiating a module creates a particular instance in the design hierarchy. Multiple instances of one definition are distinct pieces of modeled hardware, each with its own connections and state.

Named port connections make the interface explicit: `.clk(clk)` connects the child port `clk` to the parent's signal `clk`. The names need not be identical. An instance name such as `u_counter` is different from its module type `counter`; this is the same type-versus-instance distinction used for library pins in Lesson 06.

A **parameter** is a constant chosen during elaboration, when the simulator or synthesis tool constructs the design hierarchy and sizes. It is not a runtime input. A default can be overridden for an instance. A `localparam` is useful for a derived constant that an instance should not override. Changing an input while a design runs changes a signal; changing a parameter requires a differently elaborated design.

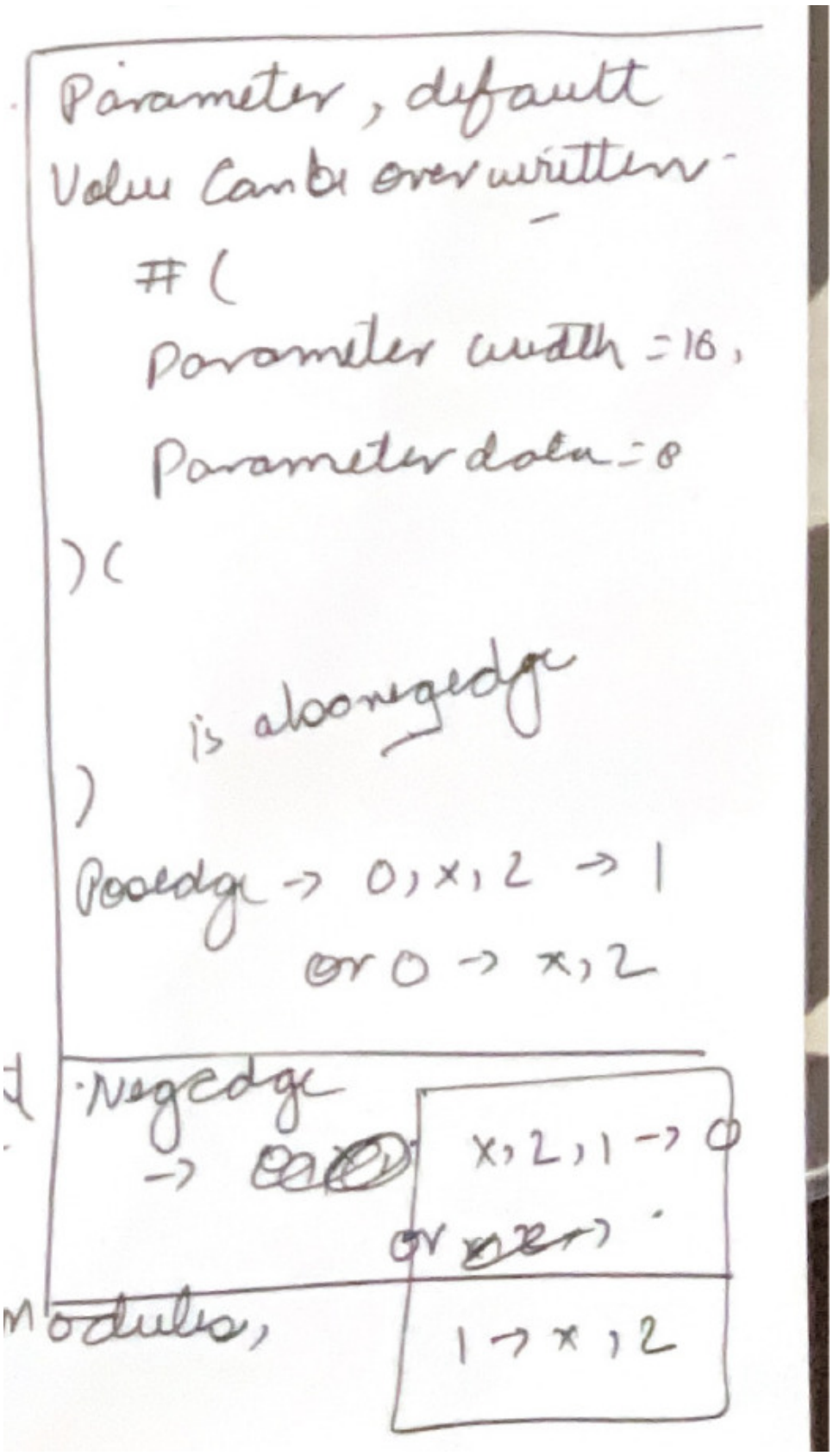


Figure 2.44. *Source: Part 2, PDF page 3, parameter and edge-event notes.*

The default/override example means “use this constant unless this instance supplies another.” The following complete example creates an eight-bit counter from a module whose default width is four. `WIDTH` must be at least one.

```
Verilog
module counter #(parameter WIDTH = 4) (
    input wire clk, rst_n, enable,
    output reg [WIDTH-1:0] count
);
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            count <= {WIDTH{1'b0}};
        else if (enable)
            count <= count + {{(WIDTH-1){1'b0}}, 1'b1};
    end
endmodule

module counter_top (
    input wire clk, rst_n, enable,
    output wire [7:0] count
);
    counter #(.WIDTH(8)) u_counter (
        .clk(clk), .rst_n(rst_n), .enable(enable), .count(count)
    );
endmodule
```

When reset becomes low, `count` clears without waiting for a rising clock; this is an **asynchronous active-low reset**. When reset is high and `enable` is high at a rising edge, the counter increments. When disabled, a clocked variable retaining its old value is intentional register behavior. After 255, the eight-bit counter wraps to zero because the carry exceeds the declared width. This coding example specifies behavior; a real reset network also needs an appropriate reset-release strategy and timing checks.

Operators and bit-level examples

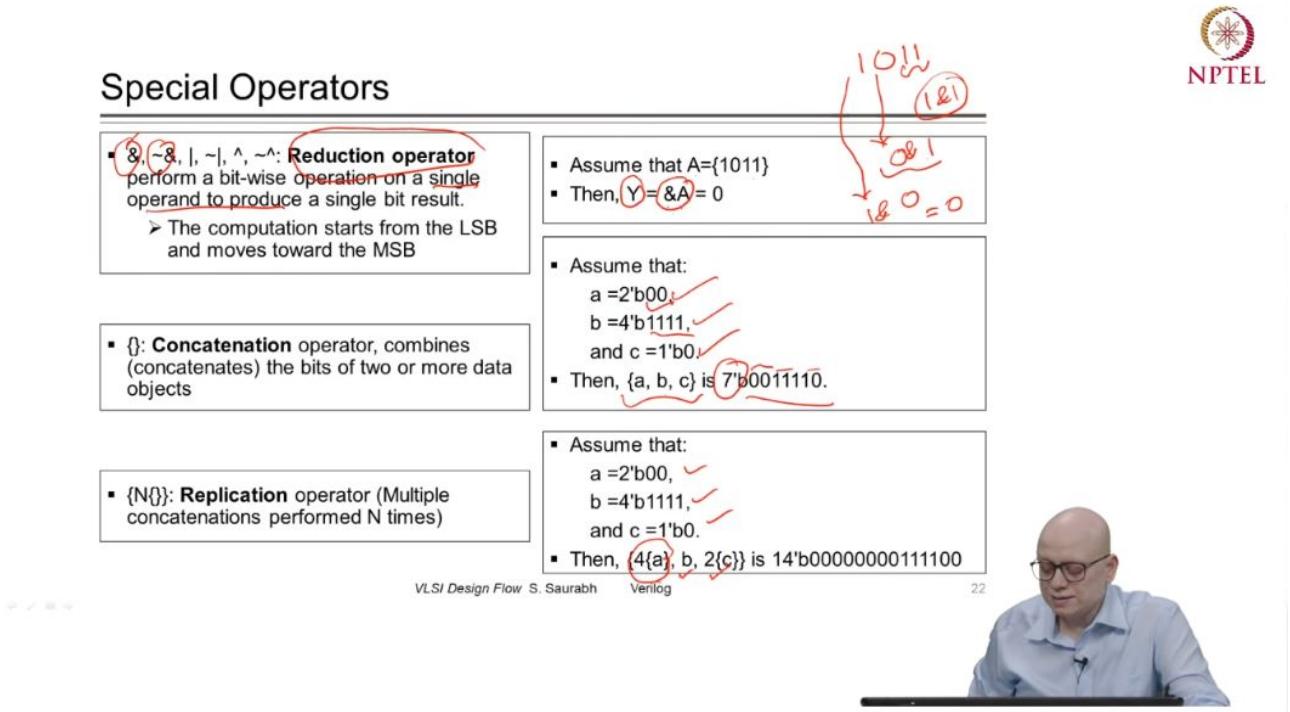


Figure 2.45. *Lecture: 16:19. Bitwise operations, concatenation, replication, and conditional selection*

An operator combines values according to its own width, signedness, and four-state rules. Do not substitute a logical operator for a bitwise one merely because their symbols look similar.

Operation	Example	Result and meaning
Bitwise AND	`4'b1010 & 4'b1100`	`4'b1000`, one AND per bit position
Logical AND	`4'b1010 && 4'b1100`	`1'b1`, both operands are nonzero
Reduction AND	`&4'b1110`	`1'b0`, AND all bits of one operand
Reduction XOR	`^4'b1011`	`1'b1`, odd parity of the operand
Bitwise complement	`~4'b1010`	`4'b0101`
Logical negation	`!4'b1010`	`1'b0`
Concatenation	`{4'b1010, 4'b1100}`	`8'b10101100`
Replication	`{3{2'b10}}`	`6'b101010`
Logical right shift	`4'b1000 >> 1`	`4'b0100`

Arithmetic right shift `>>>` fills with the sign bit when the left operand is signed; an unsigned left operand does not become signed merely because `>>>` is used. Addition, subtraction, multiplication, division, and remainder describe arithmetic operations, but their hardware costs differ greatly. A multiplication by a power of two may simplify to wiring and width handling; a general multiplier requires more logic. Division is not universally a small combinational operation.

Signedness check: the surrounding expression can affect operand interpretation. For a signed eight-bit `s=-6`, assign `s >>> 1` to a signed eight-bit temporary and it holds `11111101` (-3). Comparing the shift directly against an **unsigned** based literal can coerce the comparison operands and alter the shift's interpretation. The runnable checks demonstrate both cases. Keep intermediate signed results explicitly signed, and use deliberate casts or signed constants when combining them with other expressions.

Concatenation places the leftmost item in the most significant portion. Nested replication needs its own braces. For a one-bit `a`, two-bit `b`, and one-bit `c`, `{{4{a}}, b, {2{c}}}` has eight bits: four copies of `a`, then the two bits of `b`, then two copies of `c`. Replication counts must be appropriate constant expressions, such as parameters.

The conditional expression `sel ? b : a` selects `b` for a true condition and `a` for a false condition. For a one-bit unknown condition, equal-width four-state vector alternatives are merged bit by bit: equal known bits survive and differing bits become `x`. For example, `1'bx ? 4'b1010 : 4'b1001` yields `4'b10xx`. This differs from a procedural `if` whose condition evaluates to unknown, which does not execute its true branch.

Operator precedence determines how an unparenthesized expression is grouped. Use parentheses to state the intended comparison and logical grouping, especially when mixing bitwise, equality, shift, and arithmetic operators. Parentheses clarify grouping; they do not by themselves widen an expression or change unsigned operands into signed values.

An unknown mux select can expose different simulation behavior

For a one-bit select and equal-width integral operands, the conditional operator merges corresponding bits when the condition is x or z. Equal known bits remain known; disagreeing bits become x. Thus selecting between 1010 and 1000 with sel=x produces 10x0. It preserves the information that the two possible choices share.

An ordinary procedural `if (sel)` takes its then branch only when the condition is true; x does not make it true. In the complete example below, sel=x therefore produces `by_if=1000`. This can hide an uninitialized control even though the modeled Boolean mux is ambiguous. Both descriptions agree for a known one-bit select.

Verilog
```
module unknown_select (
    input wire sel,
    input wire [3:0] a, b,
    output wire [3:0] by_conditional,
    output reg [3:0] by_if
);
    assign by_conditional = sel ? a : b;
    always @* begin
        if (sel) by_if = a;
        else     by_if = b;
    end
endmodule
```

The useful debugging question is why sel is unknown and whether reset and the control protocol define it before use. Changing syntax alone does not repair that protocol. This example concerns four-state simulation, not a physical x-valued voltage or a synthesis don't-care. See Sutherland and Mills' discussion of X optimism, particularly conditional statements and operators.

Processes event controls and four-state edges

Trace the lecture clock generator

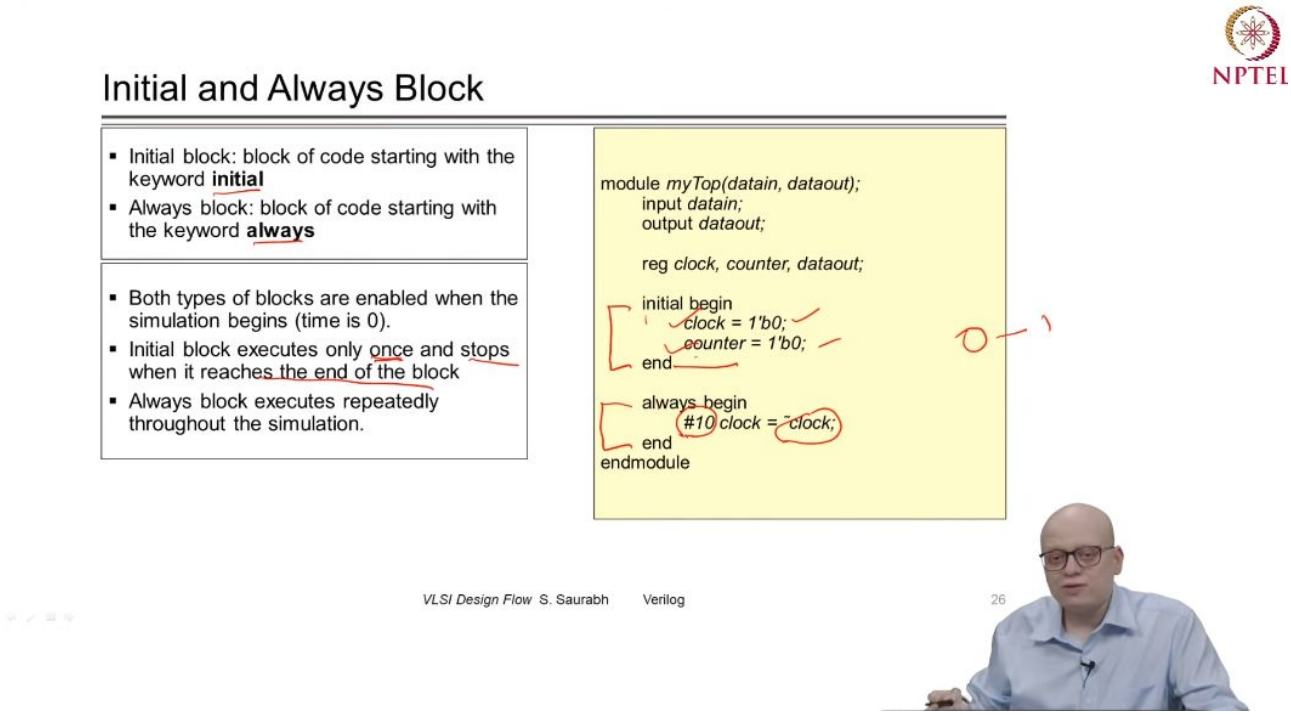


Figure 2.46. *Lecture: 21:55. Initial and always blocks, including the repeated `#10` clock toggle.*

The slide's `initial` block assigns zero to `clock` and `counter`. The separate `always` block waits ten time units, complements `clock`, reaches its end, and starts its body again. Both processes begin at simulation time zero. The clock's first toggle occurs later because the `always` process encounters a delay before its assignment, not because every `initial` block has priority over every `always` block.

The name `counter` does not create a counter circuit. It is a one-bit variable in the slide, and no shown statement increments it, so it remains zero after initialization. Likewise, the slide's unused interface ports do not define a complete input-to-output function. The example illustrates process execution.

This standalone version preserves the initialization and toggle, removes the unused interface, and adds explicit time units, observation, and a finite stopping time:

```verilog
`timescale 1ns/1ps
module initial_always_demo;
    reg clock, counter;

    initial begin
        clock = 1'b0;
        counter = 1'b0;
    end

    always begin
        #10 clock = ~clock;
    end

    initial begin
        $timeformat(-9, 0, " ns", 0);
```

```
Verilog continued
        $monitor("t=%0t clock=%b counter=%b", $time, clock, counter);
        #35 $finish;
    end
endmodule
```

The `timescale` sets a 1 ns delay unit and 1 ps precision. Therefore `#10` means 10 ns here. The third `initial` block starts once but finishes at 35 ns: **executed once** does not mean **completed at time zero**. Its `$monitor` reports changes after the values for that time slot settle.

Simulation time	What has happened	Settled `clock`	Settled `counter`
0 ns	Initialization executes; the toggle process waits	0	0
10 ns	First delayed complement	1	0
20 ns	Second delayed complement	0	0
30 ns	Third delayed complement	1	0
35 ns	`$finish` ends the demonstration	1	0

Ten nanoseconds is the **half-period**. A complete cycle takes 20 ns, giving $f = 1/(20\ \text{ns}) = 50\ \text{MHz}$. Omitting initialization can leave `clock` at x; complementing x does not establish a known zero or one. Omitting the delay creates a repeating zero-time process that can prevent simulation from advancing. The delays generate testbench events; they do not specify a physical on-chip oscillator with a guaranteed 10 ns gate delay.

The editable clock demonstration is included in the example checker. The next frame replaces fixed waiting time with waiting for signal events.

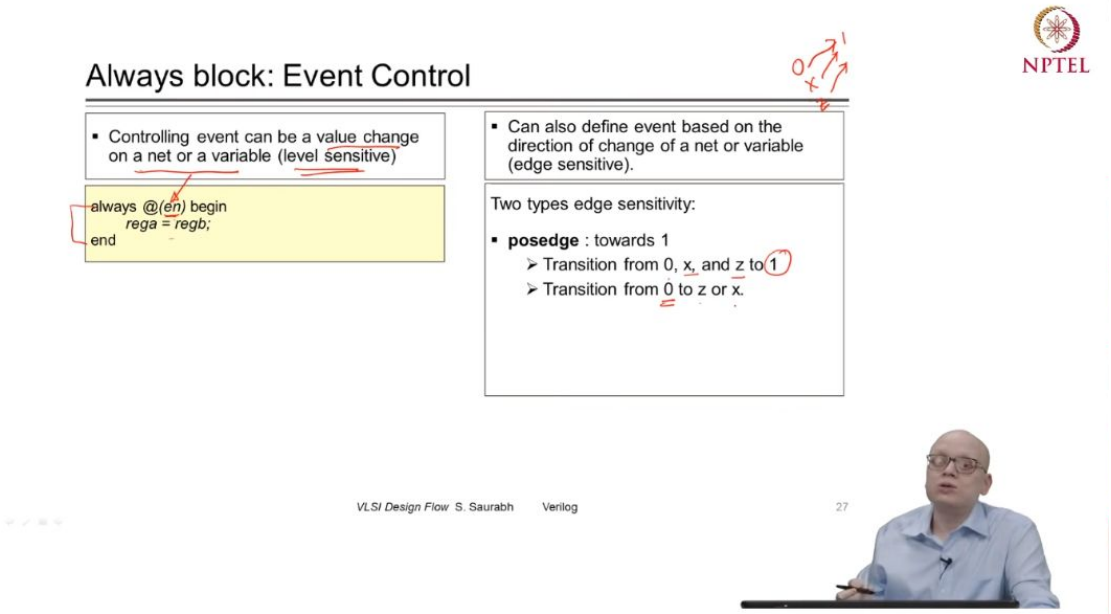


Figure 2.47. *Lecture: 24:13. Event controls suspend a process until a specified signal transition*

Different processes are concurrent; source-code order does not guarantee which separate time-zero process runs first. An event control can suspend one process while other processes continue. Event-driven simulation evaluates the modeled activity at the current time and advances to later scheduled events when that activity permits it; the simulator does not need to check every gate continuously at every possible physical instant.

`begin ... end` groups sequentially executed statements within one process. “Sequentially executed statements” does not automatically mean “sequential hardware”: a combinational process also executes statements in order. `fork ... join` starts concurrent branches and waits for all of them to finish; it is particularly useful in testbenches.

An **event control** suspends a process until the specified event occurs. `@(a or b)` reacts to changes of either operand. `@(posedge clk)` reacts to a positive-edge event. In traditional combinational RTL, `always @*` automatically collects the signals read by that process, avoiding an accidentally incomplete hand-written sensitivity list. Give every combinational output a value on every control-flow path. Otherwise the output must retain an earlier value, which can infer a latch.

The slide's `always @(en)` illustration reacts whenever `en` changes; it does not remain active throughout the time `en` is high. If its body reads another variable, changing that other variable alone does not trigger this particular event control. It is an event-control illustration, not a complete template for a combinational multiplexer or a transparent latch. Use the complete `always @*` example in Lesson 11 when modeling combinational selection.

The handwritten edge list includes unknown and high-impedance transitions. The precise single-bit simulation rules are:

Event	Transitions that trigger it
posedge	0→1, 0→x, 0→z, x→1, z→1
negedge	1→0, 1→x, 1→z, x→0, z→0
Neither edge	x→z, z→x, or no value change

These are language event rules; they do not promise that an uncertain electrical clock transition is safe. Establish a known testbench clock before measuring behavior. For a multi-bit expression used directly as an edge event, the edge detection concerns its least significant bit; ordinary clock signals should be one bit.

A `for` loop does not automatically consume a clock cycle per iteration. A statically bounded loop describing combinational work may be unrolled into parallel logic. A testbench loop containing `@(posedge clk)` explicitly waits for edges. Hardware cost and latency come from the full description and synthesis interpretation, not from the word `for` alone.

Functions and tasks

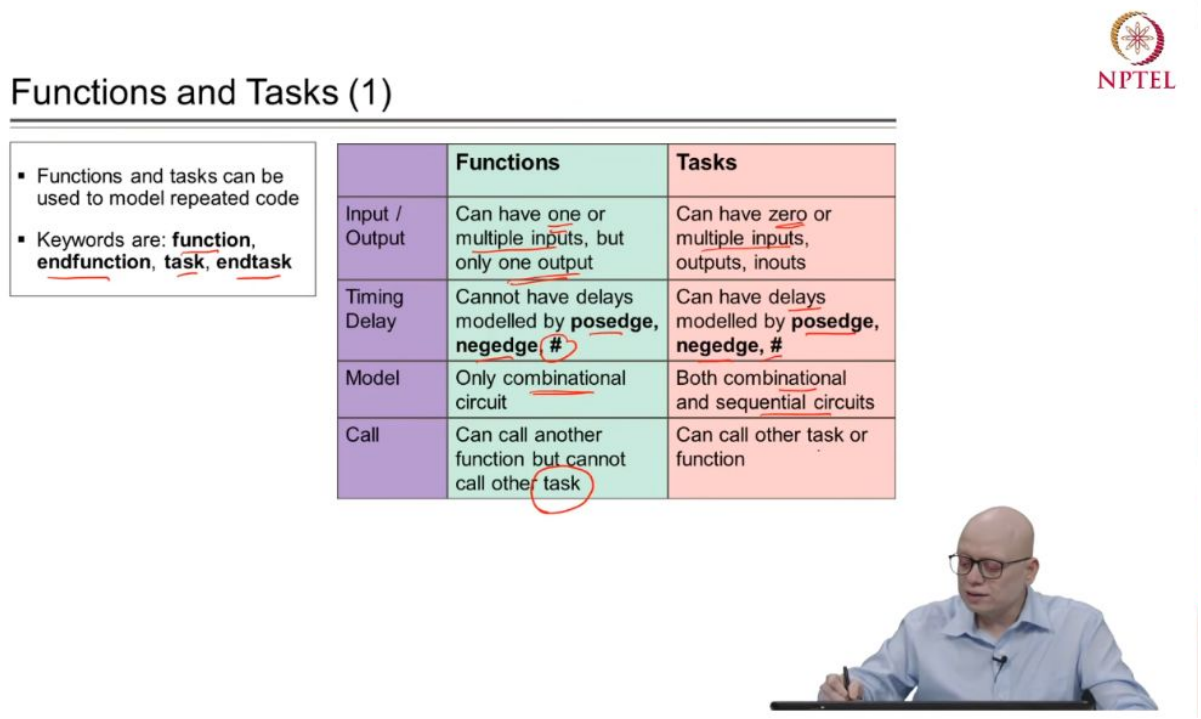


Figure 2.48. *Lecture: 31:54. Traditional Verilog functions and tasks have different timing rules*

Functions and tasks package reusable procedural work. Calling one is not the same as instantiating a module. A function computes a return value for an expression; a task is invoked as a statement and can communicate through output or inout arguments. These lessons use **traditional Verilog** rules; SystemVerilog extends several of them.

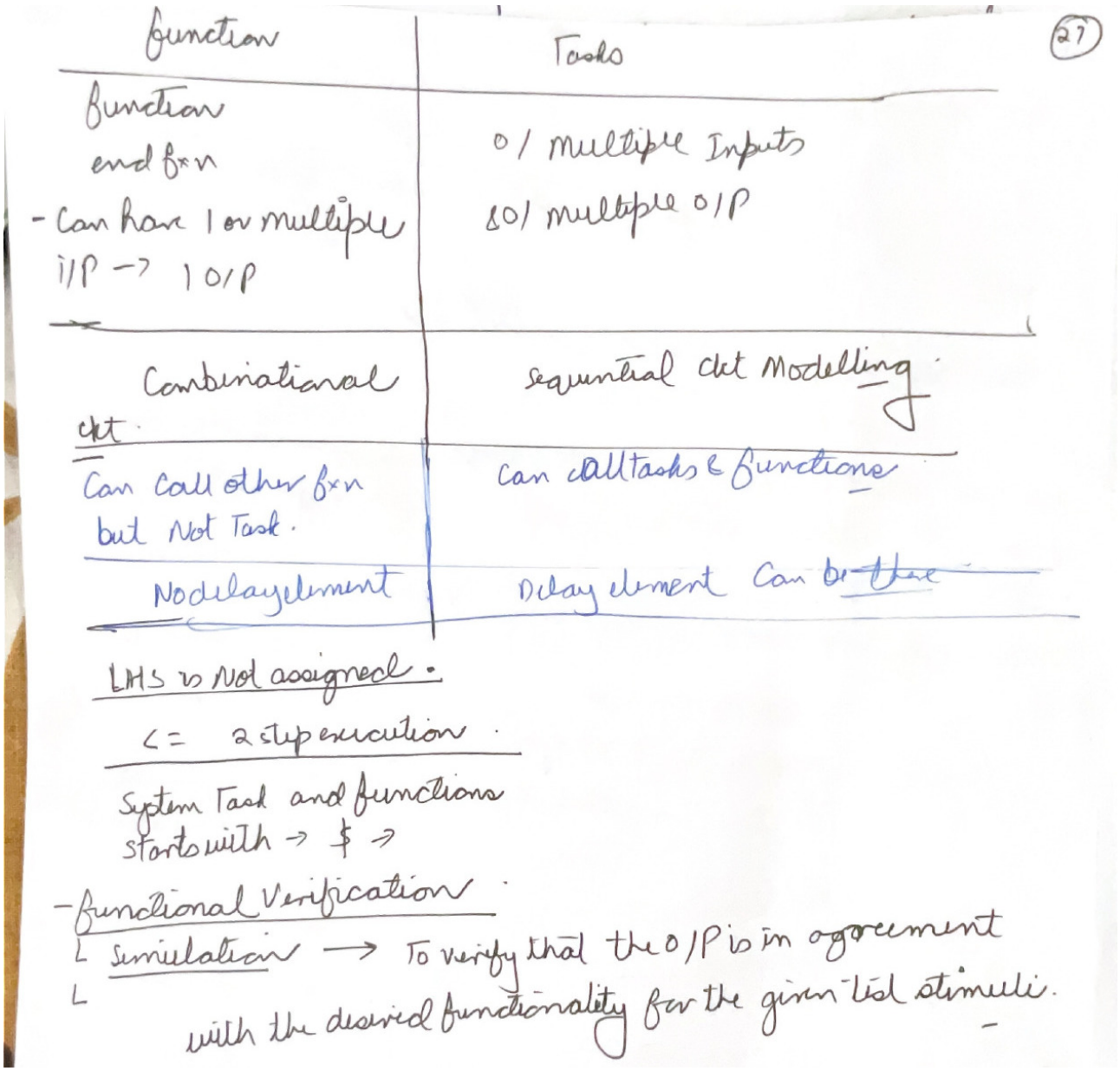


Figure 2.49. *Source: Part 2, PDF page 4, function/task comparison and assignment notes.*

Traditional Verilog feature	Function	Task
Returned result	One function return value	No function-style return value; can use output/inout arguments
Arguments	At least one input; input arguments only	Zero or more input/output/inout arguments
Time-consuming controls	No `#`, `@`, or `wait` that suspends execution	May contain timing/event controls
Calls	Can call other functions, not tasks	Can call functions and tasks
Invocation	Used in an expression	Used as a procedural statement

“Function executes in zero simulation time” means it does not advance the simulator's time; evaluating it still requires computation. “Task can take time” means permission, not obligation: a task without a wait or delay can also finish in the current time slot. These facts do not automatically

classify one as combinational hardware and the other as sequential hardware. Synthesizability depends on the body and how it is called.

By default, traditional task/function locals have static lifetime. `automatic` creates separate storage per invocation, which matters for recursion or overlapping calls. This complete demonstration keeps arithmetic in a function and console output in a task:

```
Verilog
module function_task_demo;
    function [8:0] add8;
        input [7:0] a, b;
        begin
            add8 = {1'b0, a} + {1'b0, b};
        end
    endfunction

    task show_sum;
        input [7:0] a, b;
        reg [8:0] result;
        begin
            result = add8(a, b);
            $display("%0d + %0d = %0d", a, b, result);
        end
    endtask

    initial begin
        show_sum(8'd200, 8'd100);
        $finish;
    end
endmodule
```

The printed result is 300. Widening the inputs before adding preserves the ninth carry bit. The module as a whole is a demonstration testbench because it prints and finishes simulation; the arithmetic operation can separately be used in suitable synthesizable logic.

Continuous blocking and nonblocking assignment

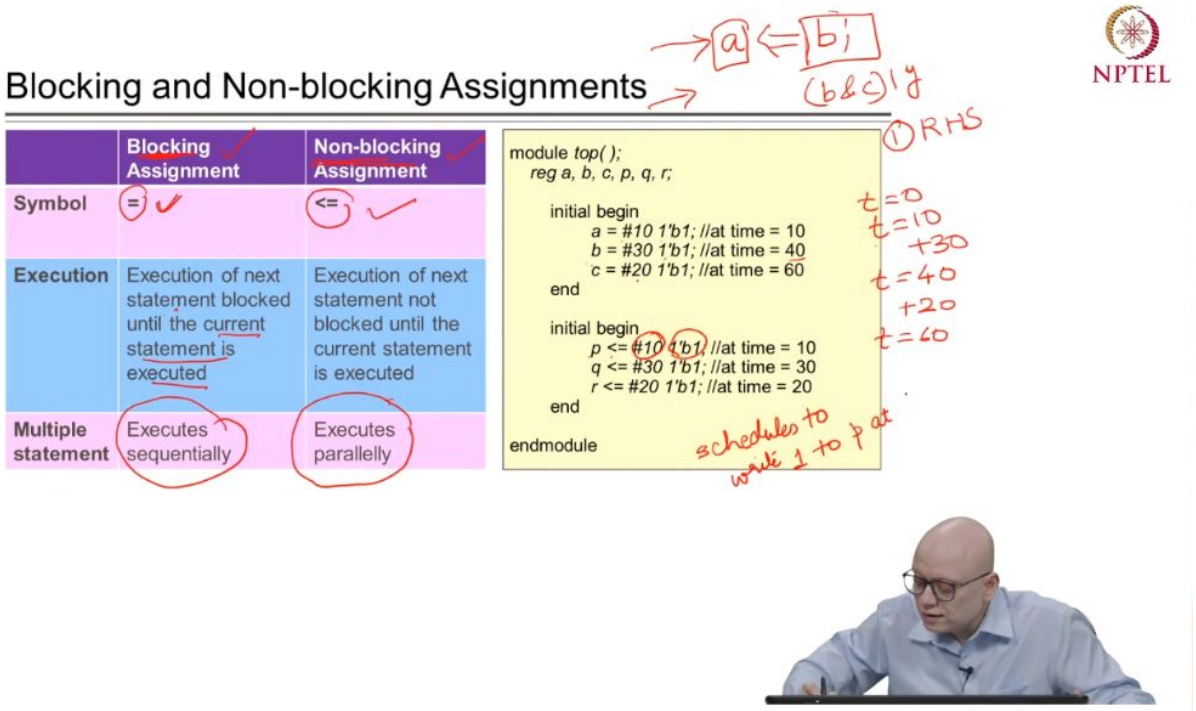


Figure 2.50. *Lecture: 43:36. Blocking delays accumulate while delayed nonblocking updates are scheduled independently*

A **continuous assignment**, such as `assign y = a & b;`, continuously drives a net as its source expression changes. A **procedural assignment** runs when execution reaches it inside a process. Traditional combinational procedural code commonly uses blocking `=`, while clocked state updates commonly use nonblocking `<=`.

With **blocking assignment**, the process completes that assignment before moving to its next statement. Without an explicit delay, the new value is available to following statements in that process immediately. With **nonblocking assignment**, the process evaluates the right-hand expression when the statement executes and schedules the destination update. With no explicit delay, that update occurs in the nonblocking-assignment region of the **current simulation time slot**, after the relevant active-region execution. It does not wait until the entire simulation ends or automatically wait for another clock edge.

This last distinction refines the two-step RHS/LHS note and the lecture's informal “end of simulation time” language. The simulator can process many events at one timestamp. Event ordering within that timestamp is different from advancing time to the next timestamp. The next dedicated simulation lesson develops scheduling in more detail; here we need enough to read RTL correctly.

Consider two registers in a pipeline:

```verilog
module pipeline2 (
    input wire clk, rst_n,
    input wire [7:0] d,
    output reg [7:0] q1, q2
);
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            q1 <= 8'd0;
```

Verilog continued

```
            q2 <= 8'd0;
        end else begin
            q1 <= d;
            q2 <= q1;
        end
    end
endmodule
```

At a normal rising clock edge, `q1` schedules the current input, while `q2` schedules the **old** value of `q1`. If old `q1=10` and `d=25`, the settled post-edge values are `q1=25`, `q2=10`. Both right-hand expressions were evaluated before their pending updates. If both lines instead used blocking assignment in this same order, the second line would observe the newly assigned `q1`, changing the model's behavior. Nonblocking assignment expresses the simultaneous sampling behavior expected from clocked registers.

Follow one sample through both registers

Assume reset has established `q1=q2=0`, reset is released away from a sampling edge, and each input below is stable before its rising edge. Read the last two columns after the nonblocking updates for that edge have completed:

Rising edge	**Input `d` before the edge**	**Old `q1` sampled by `q2`**	**New `q1`**	**New `q2`**
First	25	0	25	0
Second	42	25	42	25
Third	9	42	9	42

The sample 25 enters `q1` on the first edge and reaches `q2` on the second. Once the pipeline is filled, a new sample can emerge at every edge even though each sample passes through two registers. This separates **latency**, the delay experienced by a particular sample, from **throughput**, the rate of completed samples. When stating a cycle latency, identify the reference event: here `q2` changes one clock period after the edge that first captures the sample into `q1`.

The assignments run at the same simulation timestamp as the edge; the table does not require time to advance by a full period before `q1` updates. It requires the pending nonblocking assignments to settle. An observation made immediately in an active-region process at `posedge clk` can still see the old register values. This is why a waveform's timestamp and its event ordering both matter.

The lecture's delayed-assignment example

The following is a complete, explicitly timed simulation example. These # delays are for understanding the language; they are not a synthesizable implementation of arbitrary hardware delays.

```
Verilog
`timescale 1ns/1ps
module assignment_delays;
    reg a, b, c, p, q, r;
    initial begin
        a = 0; b = 0; c = 0;
        a = #10 1'b1;
        b = #30 1'b1;
        c = #20 1'b1;
    end
    initial begin
        p = 0; q = 0; r = 0;
        p <= #10 1'b1;
        q <= #30 1'b1;
        r <= #20 1'b1;
    end
    initial begin
        $monitor("t=%0t a=%b b=%b c=%b p=%b q=%b r=%b",
                 $time, a, b, c, p, q, r);
        #61 $finish;
    end
endmodule
```

Destination	Becomes one at	Why
`a`	10 ns	First blocking assignment delays its completion by ten
`b`	40 ns	Execution reaches it at ten, then waits another thirty
`c`	60 ns	Execution reaches it at forty, then waits another twenty
`p`	10 ns	Scheduled from time zero with a delay of ten
`q`	30 ns	Scheduled from time zero with a delay of thirty
`r`	20 ns	Scheduled from time zero with a delay of twenty

Here the delay is **inside** the assignment: `p <= #10 expression` samples the expression now and schedules the write ten time units later. Moving the delay in front, `#10 p <= expression`, first suspends the process and then evaluates the expression. These forms can produce different results if inputs change during the delay. The example uses constant RHS values to isolate scheduling.

Change the RHS to reveal when sampling happens

The lecture uses constants, which conceal the difference between delaying a sample and delaying a write. In this complete simulation, d changes at 5 ns, away from the two sampling times:

```
Verilog
`timescale 1ns/1ps
module delayed_samples;
    reg d = 0;
    reg inside_delay = 0, before_delay = 0;
    initial #5 d = 1;
    initial inside_delay <= #10 d;
    initial #10 before_delay <= d;
    initial begin
        #11;
        $display("inside=%b before=%b", inside_delay, before_delay);
        $finish;
    end
endmodule
```

`inside_delay <= #10 d` evaluates d=0 at time zero and schedules that sampled zero for 10 ns. `#10 before_delay <= d` first waits until 10 ns, then evaluates d=1 and schedules its nonblocking update in that time slot. After both updates settle, the display reports `inside=0 before=1`.

Neither form samples continuously during the waiting interval. Each reads the RHS at one execution event. Observing at 11 ns also avoids confusing active-region execution at 10 ns with the later nonblocking update at that same timestamp. These explicit delays are simulation models; a synthesizable pipeline expresses delayed samples with registers and a clock, as the earlier pipeline example does.

System tasks and the next study boundary

System task/function names start with `$`. `$display` prints when it executes; `$monitor` reports changes to its argument values at the end of the time step; `$time` returns simulation time; `$finish` ends simulation and `$stop` requests a simulation stop, whose interactive behavior depends on the tool. Waveform dumping commonly uses `$dumpfile` and `$dumpvars` in tools supporting VCD output. Use the selected simulator's manual for nonstandard names and extensions; do not assume every `$` example on a slide is portable.

The final handwritten definition of **functional verification using simulation** says that a testbench applies stimuli and checks responses against expected behavior. That introductory definition connects to Lesson 08. The dedicated *Functional Verification Using Simulation* lesson is documented in Day 03, Lesson 13.

The Verilog example folder contains the complete modules and a check script. It checks arithmetic widths, four-state matching, edge-event behavior, parameter overrides, pipeline state, and the timed blocking/nonblocking example. The examples support these explanations; they do not claim exhaustive verification of a production design.

Definitions and mechanisms in Lesson 12

Module instance and elaboration. A module definition describes a reusable hardware unit. An instance is one occurrence with particular parameters and connections. Elaboration constructs that hierarchy, resolves widths and applies constant configuration. Two counter instances can use the same definition while having distinct state. A parameter changes the elaborated design; a runtime input changes a signal during its operation. Named connections identify the child port and the parent signal explicitly, which helps prevent positional wiring mistakes.

Process, event and simulation time slot. A process is an independently executing procedural activity, such as an initial or always block. An event can awaken a suspended process, and several activities may occur at the same simulation timestamp. The simulator's event ordering within that time slot is separate from advancing to a later time. An initial block starts once and may finish much later if it waits. An always block repeatedly executes its body and needs a meaningful wait or delay to avoid an uncontrolled zero-time loop.

Event control and sensitivity. An event control suspends execution until a specified change or edge occurs. A sensitivity list identifies changes that trigger a process; it is not a statement that the process continuously runs whenever a signal is high. For combinational behavior, every relevant input change must be reflected and every output must be assigned along every path. An incomplete sensitivity list can make simulation stale, while incomplete assignments can require stored state. These are different mistakes even if both cause surprising waveforms.

Blocking and nonblocking assignment. A blocking assignment completes before the process proceeds to its next statement. A nonblocking assignment evaluates its right-hand expression when reached and schedules its destination update. With no explicit delay, that scheduled write occurs in the current time slot's nonblocking update region. This allows several clocked registers to sample old state before their new values are committed. Nonblocking does not mean next clock, and blocking does not mean that the whole simulator or all other processes stop.

Assignment delay and event delay. In a delayed nonblocking assignment, the right-hand value is sampled when the statement executes and the write is scheduled for later. A delay placed before the assignment instead suspends the process before expression evaluation. Changing inputs during the wait can therefore make the forms produce different values. Timing controls in the demonstration model simulator behavior; they do not compel synthesis to manufacture a logic path with an arbitrary specified delay.

Function, task and module reuse. A traditional Verilog function supplies a value for an expression without a time-consuming suspension. A task is called procedurally and may use timing controls and output or inout arguments. A module instance creates a structural instance in the design hierarchy. These are different forms of reuse. Neither the word function nor the word task alone determines the inferred hardware; the body, context and tool support matter. The lesson's printed arithmetic demonstration is a simulation example even though its addition can also be described in synthesizable logic.

Reset, enable and validity. Reset establishes a defined state according to its synchronous or asynchronous behavior. Enable determines whether an allowed state update occurs. Validity indicates whether stored data should be interpreted as a completed or meaningful transaction. They answer different interface questions. A register may contain a known reset value that is not a completed calculation; a shared datapath may temporarily hold an intermediate value. Correct communication about a block identifies when it accepts input and when its output is valid.

Application and validation in Lesson 12

A sequential trace records state immediately before an event, evaluates right-hand sides using the applicable statement rules, lists scheduled updates and then records settled state. For nonblocking pipeline assignments, the receiving stage samples the preceding stage's old value. Delay controls determine when evaluation or updating occurs and must be interpreted in their exact statement position.

Pipeline latency versus throughput. With `q1<=d; q2<=q1;`, reset state $(q1, q2) = (0, 0)$ and successive sampled inputs 5, 7, 9, the settled states are $(5, 0)$, $(7, 5)$ and $(9, 7)$. Each accepted sample reaches `q2` on the next rising edge after entering `q1`; the pipeline accepts a new sample every rising edge. Changing both assignments to blocking statements in the same process makes `q2` receive the newly assigned `q1` at that edge and removes the modeled interstage delay. A waveform checker must observe after nonblocking updates; observing in the active region can mistake old state for a design failure.

Complete original notebook pages for Lesson 12

Part 2 original page 04

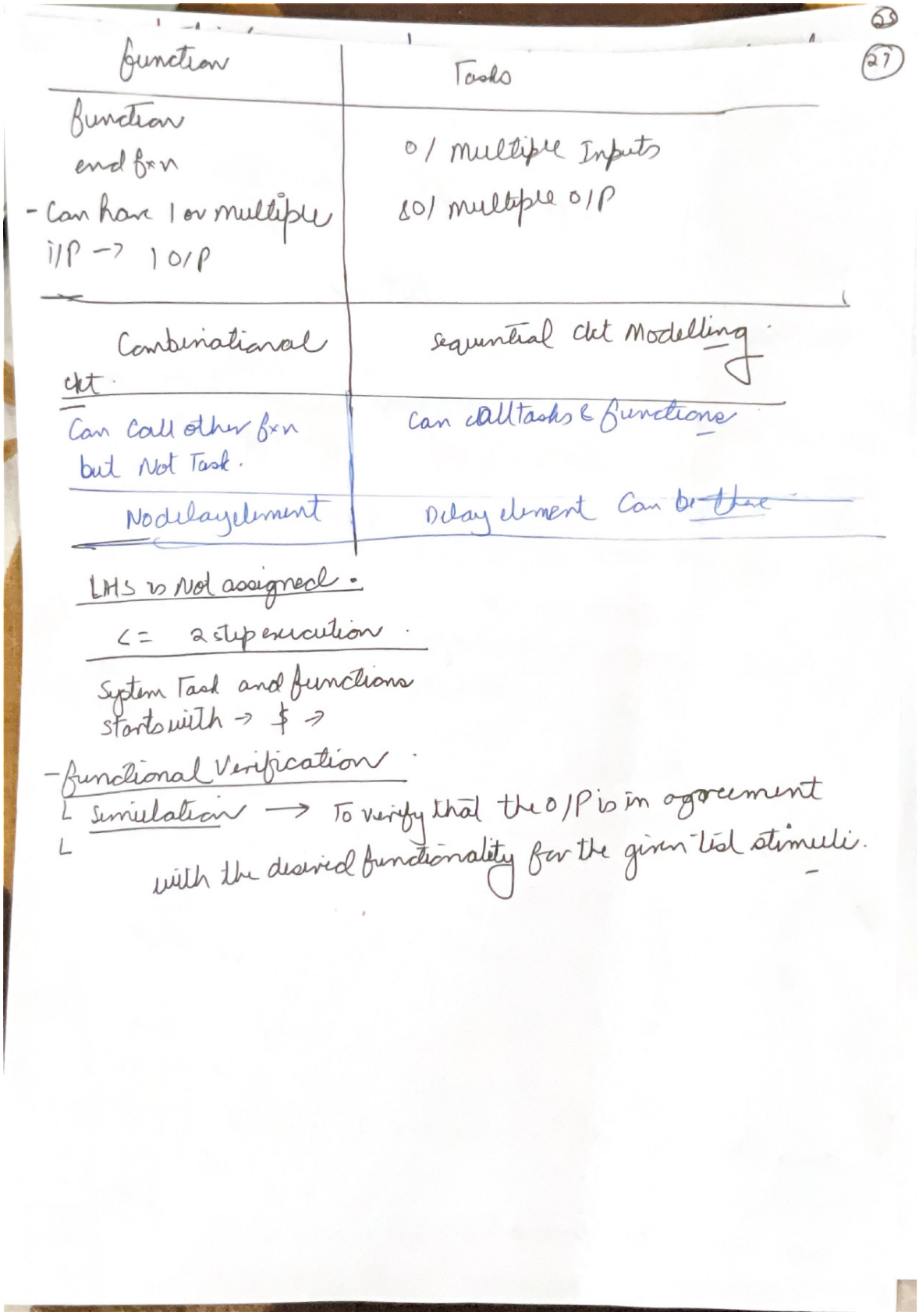


Figure 2.51. *Source: Part 2, PDF page 4. Functions/tasks, nonblocking assignment, system tasks, simulation definition.*

Explanation and worked examples: Day 2 · Lesson 12 · Day 2 · Lesson 08.

Revision checks and explained answers

Try each question before reading its answer. Revisit the linked lesson and its original handwritten page when an assumption or step is unclear.

Check 01: Utilization does not prove routability

Question. A 100 by 100 micrometre core contains 6,000 square micrometres of standard cells. What is nominal utilization, and what does it leave unproved?

Explained answer. Core area is 10,000 square micrometres, so nominal standard-cell utilization is 60%. The remaining area is not a guarantee of accessible pins, legal power connections or enough routing tracks. A local hotspot can fail routing even when the global ratio looks comfortable. State whether macros or blockages have already been removed from the denominator before comparing utilization figures.

Check 02: Clock delay and skew are separate

Question. Two sinks receive the same clock edge after 90 ps and 130 ps. Find their pairwise skew. Does making both paths 30 ps slower change that skew?

Explained answer. The arrival difference is 40 ps. Changing the arrivals to 120 ps and 160 ps preserves that difference, while increasing insertion delay. This example assumes the added delays affect both paths equally. Actual clock cells and loads can alter slew and relative delay. A clock-tree report must therefore expose both absolute arrivals and the relevant launch/capture difference.

Check 03: Yield is not fault coverage

Question. Of 1,000 manufactured devices, 800 are good under a stated test criterion. Separately, a pattern set detects 180 of 200 modeled faults. Compare the two percentages.

Explained answer. The observed good-device fraction is 80%; the modeled fault coverage is 90%. Their denominators describe different populations. The coverage result alone cannot establish a 90% shipped yield or a particular escape rate. A modeled fault can represent many physical defects, and the defect distribution and screening process affect what ships.

Check 04: Follow Tcl substitution

Question. Let `a` be 2. Compare `set b {$a+3}`, `expr {$a+3}`, and `set c [expr {$a+3}]`.

Explained answer. Braces in the first command preserve the literal text `$a+3` as `b`'s value. In the second command, `expr` interprets its braced expression and obtains 5. In the third, square brackets execute `expr` and substitute its result into `set`, so `c` receives 5. Bracing an expression controls Tcl's first substitution stage; it does not prevent `expr` from evaluating variables.

Check 05: Truncate a sized literal deliberately

Question. What value remains when hexadecimal CA is represented by the six-bit literal 6'hCA?

Explained answer. Hexadecimal CA is decimal 202. A six-bit result keeps the low six bits: $202 - 3 \times 64 = 10$, represented as 001010. A tool warning about truncation is useful evidence, not a reason to silently reinterpret the literal as eight bits. Keep operand width, signedness and expression context explicit before extending this example to arithmetic.

Check 06: Predict nonblocking updates

Question. At an edge, old q is 0 and d is 1. The same sequential process executes q <= d followed by r <= q. What values are scheduled?

Explained answer. q is scheduled to become 1; r is scheduled to become the old q, which is 0. Right-hand sides are evaluated before those nonblocking updates take effect. Replacing the first statement with blocking assignment changes the procedural value seen by the second statement and may change the modeled pipeline. Observe after the nonblocking update region when checking the resulting state.

Handwritten page finder

Every entry opens the complete original page embedded in this PDF, beside its explanation. Original scan page numbers and this PDF's page numbers are different.

Original handwritten page	Open in this PDF
Part 1, page 17	Figure 2.08
Part 1, page 18	Figure 2.09
Part 1, page 19	Figure 2.10
Part 1, page 20	Figure 2.11
Part 1, page 21	Figure 2.22
Part 1, page 22	Figure 2.23
Part 1, page 23	Figure 2.24
Part 1, page 24	Figure 2.25
Part 2, page 1	Figure 2.32
Part 2, page 2	Figure 2.33
Part 2, page 3	Figure 2.42
Part 2, page 4	Figure 2.51

Sources and references

Primary course: **VLSI Design Flow: RTL to GDS**, Prof. Sneh Saurabh, IIIT Delhi, NPTEL. Unix and Tcl tutorials: Jasmine Kaur. Lecture captions provide video timestamps. Additional derivations and practice examples are study annotations.

Open the course video playlist.

Original handwritten sources: **Part 1 PDF pages 17 to 24 and Part 2 PDF pages 1 to 4**. Source identifiers are separate from the page numbers of this document.

Primary references for the technical clarifications

1. OpenROAD placement

2. SKY130 clock-gating cell model

3. ASML - Lithography principles

4. Tcl 8.6 - expr

5. Tcl 8.6 - Syntax and substitution rules

6. Tcl 8.6 - lset

7. Tcl 8.6 - foreach

8. Sutherland HDL - Verilog-2001 reference

9. Sutherland and Mills - X behavior in simulation

Numerical examples use stated teaching assumptions. Technology-specific library values and manufacturing rules require their corresponding source models. Verilog examples use traditional Verilog semantics unless otherwise stated.